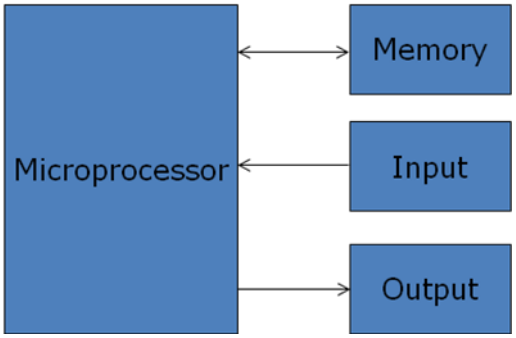
CONTENTS

CHAPTER -1 INTRODUCTION

Microprocessor



A Microprocessor is a multipurpose programmable, clock driven, register based electronic device that reads binary instructions from a storage device called memory, accepts binary data as input, processes data according to those instructions and provide results as output. The microprocessor operates in binary 0 and 1 known as bits are represented in terms of electrical voltages in the machine that means 0 represents low voltage level and 1 represents high voltage level. Each microprocessor recognizes and processes a group of bits called the word and microprocessors are classified according to their word length such as 8 bits microprocessor with 8 bit word and 32 bit microprocessor with 32 bit word etc.

Fig 1.1: A Programmable Machine

Terms used:

- CPU: - Central processing unit which consists of ALU and control unit.
- Microprocessor: - Single chip containing all units of CPU.
- Microcomputer: - Computer having microprocessor as CPU.
- Microcontroller: single chip consisting of MPU, memory, I/O and interfacing circuits.
- MPU: - Micro processing unit – complete processing unit with the necessary control signals.

Advantages of Microprocessor:

- Computational/Processing speed is high
- Intelligence has been brought to systems
- Automation of industrial process and office automation
- Flexible
- Compact in size
- Maintenance is easier

Applications of Microprocessors:

- Microcomputer: Microprocessor is the CPU of the microcomputer.
- Embedded system: Used in microcontrollers.

- Measurements and testing equipment: used in signal generators, oscilloscopes, counters, digital voltmeters, x-ray analyzer, blood group analyzers baby incubator, frequency synthesizers, data acquisition systems, spectrum analyzers etc.
- Scientific and Engineering research.
- Industry: used in data monitoring system, automatic weighting, batching systems etc.
- Security systems: smart cameras, CCTV, smart doors etc.
- Automatic system
- Communication system Some

  Examples are:

  – Calculators
  – Accounting system
  – Games machine
  – Complex Industrial Controllers
  – Traffic light Control
  – Data acquisition systems
  – Military applications

Historical Background of the Development of Computers

The most efficient and versatile electronic machine computer is basically a development of a calculator which leads to the development of the computer. The older computer were mechanical and newer are digital. The mechanical computer namely difference engine and analytical engine developed by Charles Babbage the father of the computer can be considered as the forerunners of modern digital computers.

The difference engine was a mechanical device that could add and subtract and could only run a single algorithm. Its output system was incompatible to write on punched cards and early optical disks. The 'analytical engine' provided more advanced features. It consisted mainly four components the store (memory), the mill (computation unit), input section (punched card reader) and output section (punched and printed output). The store consisted of 1000s of words of 50 decimal digits used to hold variables and results. The mill could accept operands from the store, add, subtract, multiply or divide them and return a result to the store.

The evolution of the vacuum tubes led the development of computer into a new era. The world's first general purpose electronic digital computer was ENIAC (Electronic Numerical Integrator and Calculator) built by using vacuum tubes was enormous in size and consumed very high power. However it was faster than mechanical computers. The ENIAC was decimal machine and performed only decimal numbers. Its memory consisted of 20 'accumulators' each capable of holding 10 digits decimal numbers. Each digit was represented by a ring of 10 vacuum tubes. ENIAC had to be programmed

manually by setting switches and plugging and unplug a cable which was the main drawback of it.

Automated calculator:

It is a data processing device that carries out logic and arithmetic operations but has limited programming capability for the user. It accepts data from a small keyboard one digit at a time performs required arithmetic and logical calculations and stores the result on visual display like LCD or LED. The calculator's programs are stored in ROM's while the data is stored in RAM.

Some important features of automated calculations:

- The ability to interface easily with keyboards and displays.
- The ability to handle decimal digits, the device is able to handle more than 4 bits at a time.
- Ability to execute the standard programs stored in read only memory.
- Extendibility, so that mathematical functions such as %, √, trigonometric, statistical etc. can be easily executed.
- Flexibility so it can be used in engineering business or programming without a complete new design.
- Low cost, small size and low power consumptions

Stored Program Concept and Von-Neumann Machine:

The simplest way to organize a computer is to have one processor, register and instruction code format with two parts op-code and address/operand. The memory address tells the control where to find an operand in memory. This operand is read from memory and used as data to be operated on together with the data stored in the processor register. Instructions are stored in one section of same memory. It is called stored program concept.

The task of entering and altering the programs for ENIAC was tedious. It could be facilitated if the program could be represented in a form suitable for storing in memory alongside the data. So the computer could get its instructions by reading from the memory and program could be set or altered by setting the values of a portion of memory. This approach is known as 'stored- program concept' was first adopted by John Von Neumann and such architecture is named as von-Neumann architecture and shown in figure below.

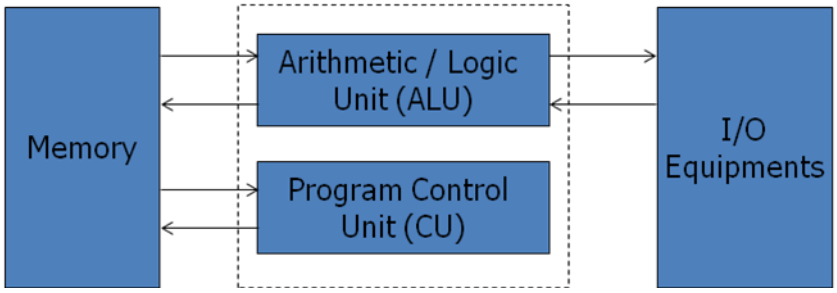


Fig: Von –Neumann Architecture

The main memory is used to stare both data and instructions. The arithmetic and logic unit is capable of performing arithmetic and logical operation on binary data. The program control unit interprets the instruction in memory and causes them to be executed. The I/O unit gets operated from the control unit.

The Von–Neumann architecture is the fundamental basis for the architecture of modern digital computers. It consisted of 1000 storage locations which can hold words of 40 binary digits and both instructions as well as data are stored in it. The storage location of control unit and ALU are called registers and the various models of registers are:

MAR – memory address register – contains the address in memory of the word to be written into or read from MBR.
MBR – memory buffer register – consists of a word to be stored in or received from memory.
IR – instruction register – contains the 8-bit op-code instruction to be executed. **IBR** – instruction buffer register – used to temporarily hold the instruction from a word in memory.
PC - program counter - contains the address of the next instruction to be fetched from memory. **AC & MQ** (Accumulator and Multiplier Quotient) - holds the operands and results of ALU after processing.

Harvard Architecture:

In von-Neumann architecture, the same memory is used for storing instructions and data. Similarly, a single bus called data bus or address bus is used for reading data and instructions from or writing to memory. It also had limited the processing speed for computers.

The Harvard architecture based computer consists of separate memory spaces for the programs (instructions) and data. Each space has its own address and data buses. Therefore, instructions and data can be fetched from memory concurrently and provides significance processing speed improvement.

In figure below, there are two data and two address buses multiplexed for data bus and address bus. Hence, there are two blocks of RAM chips one for program memory and another for data memory addresses.

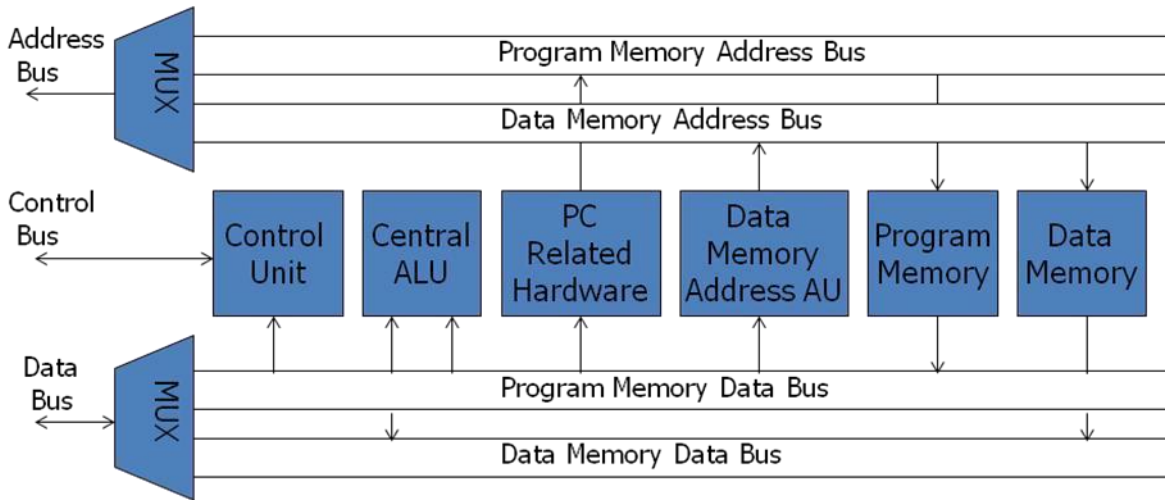


Fig: Harvard Architecture Based Microprocessor

The control unit controls the sequence of operations. Central ALU consists of ALU, multiplier, accumulator and scaling chief register. The PC used to address program memory and always contains the address of next instruction to be executed. Here data and control buses are bidirectional and address bus is unidirectional.

Evolution of Microprocessors (Intel series):

The CPU of a computer consists of ALU, CU and memory. If all these components can be organized on a single chip by means of SSI, MSI, LSI, VLSI, ULSI, ELSI technology, then such chip is called microprocessor. It can fetch instructions from memory, decode and execute them, perform logical and arithmetic functions, accept data from input devices and send results to the output devices. The evolution of microprocessor is dependent on the development of integrated circuit technology from single scale integration (SSI) to giga scale integration (GSI).

Date	**Microprocessor**	**Data bus**	**Address Bus**	**Memory**
1971	4004	4-bit	10-bit	640 Bytes
1972	8008	8-bit	14-bit	16k
1974	8080	8bit	16bit	64k
1976	8085	8bit	16b it	64k
1978	8086	16bit	20bit	1M
1979	8088	8bit	20bit	1M
1982	80286	16bit	24bit	16M
1985	80386	32bit	32bit	4G
1989	80486	32bit	32bit	4G
1993	Pentium	32/64bit	32bit	4G
1995	Pentium pro	32/64bit	36bit	64G
1997	Pentium II	64bit	36bit	64G
1998	Celeron	64bit	36bit	64G
1999	Pentium III	64bit	36bit	64G
2000	Pentium IV	64bit	36bit	64G
2001	Itanium	128 bit	64bit	64G
2002	Itanium 2	128 bit	64bit	64G
2003	Pentium M/Centrino (wireless capability) for Mobile version e.g. Laptop			
	Core 2: X86 – 64 Architecture			

Components of microprocessor

Microprocessor based system includes there components microprocessor, input/output and memory (read only and read/write). These components are organized around a common communication path called a bus.

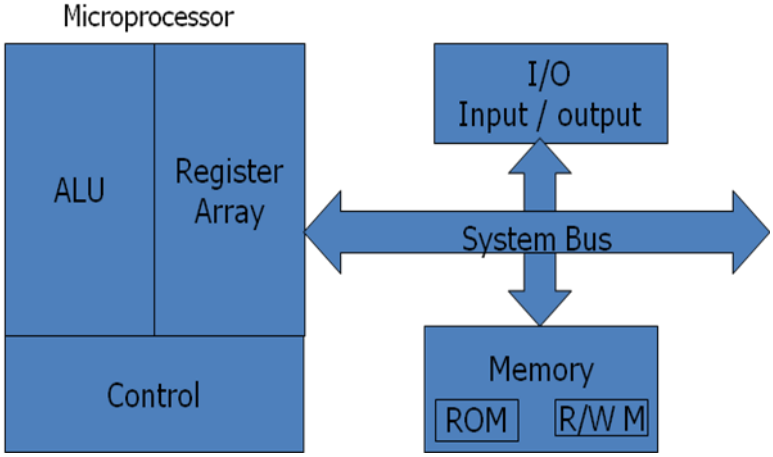


Fig 1.3: Microprocessor Based System with Bus Architecture

Microprocessor:

It is clock driven semiconductor device consisting of electronic logic circuits manufactured by using either a large scale integration (LSI) or very large scale integration (VLSI) technique. It is capable of performing various computing functions and making decisions to change the sequence of program execution. It can be divided in to three segments.

A. Arithmetic/Logic unit: It performs arithmetic operations as addition and subtraction and logic operations as AND, OR & XOR.
B. Register Array: The registers are primarily used to store data temporarily during the execution of a program and are accessible to the user through instruction. The registers can be identified by letters such as B, C, D, E, H and L.
C. Control Unit: It provides the necessary timing and control signals to all the operations in the microcomputer. It controls the flow of data between the microprocessor and memory & peripherals.

Memory:

Memory stores binary information such as instructions and data, and provides that information to the up whenever necessary. To execute programs, the microprocessor reads instructions and data from memory and performs the computing operations in its

ALU. Results are either transferred to the output section for display or stored in memory for later use. Memory has two sections.

A. Read only Memory (ROM): Used to store programs that do not need alterations and can only read.
B. Read/Write Memory (RAM): Also known as user memory which is used to store user programs and data. The information stored in this memory can be easily read and altered.

Input/Output:

- It communicates with the outside world using two devices input and output which are also Known as peripherals.
- The input device such as keyboard, switches, and analog to digital converter transfer binary information from outside world to the microprocessor.
- The output devices transfer data from the microprocessor to the outside world. They include the devices such as LED, CRT, digital to analog converter, printer etc.

System Bus:

It is a communication path between the microprocessor and peripherals; it is nothing but a group of wires to carry bits.

Data Bus :

A collection of wires through which data is transmitted from one part of a computer to another is called Data Bus. Data Bus can be thought of as a highway on which data travels within a computer. This bus connects all the computer components to the CPU and main memory. The size (width) of bus determines how much data can be transmitted at one time.

E.g.:

 A 16-bit bus can transmit 16 bits of data at a time.
32-bit bus can transmit 32 bits at a time.

Address Bus

A collection of wires used to identify particular location in main memory is called Address Bus. In other words, the information used to describe the memory locations travels along the address bus.The size of address bus determines how many unique memory locations can be addressed.

E.g.:

- A system with 4-bit address bus can address 24 = 16 Bytes of memory.
- A system with 16-bit address bus can address 216 = 64 KB of memory. A system with 20-bit address bus can address 220 = 1 MB of memory.

Control Bus

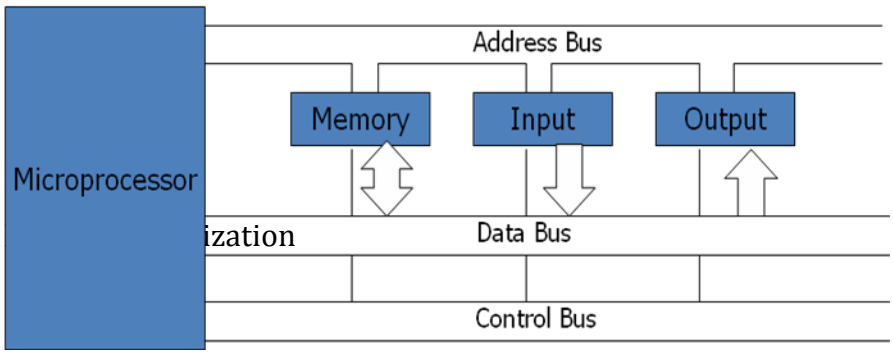
The connections that carry control information between the CPU and other devices within the computer is called Control Bus. The control bus carries signals that report the status of various devices.

E.g.:

 This bus is used to indicate whether the CPU is reading from memory or writing to memory.

Bus organization

Bus is a common channel through which bits from any sources can be transferred to the destination. A typical digital computer has many registers and paths must be provided to transfer instructions from one register to another. The number of wires will be excessive if separate lines are used between each register and all other registers in the system. A more efficient scheme for transferring information between registers in a multiple register configuration is a common bus system. A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time. Control signals determine which register is selected by the bus during each particular register transfer.



A very easy way of constructing a common bus system is with multiplexers. The multiplexers select the source register whose binary information is then pleased on the bus.

A system bus consists of about 50 to 100 of separate lines each assigned a particular meaning or function. Although there are many different bus designers, on any bus, the lines can be classified into three functional groups; data, address and control lines. In addition, there may be power distribution lines as well.

- The data lines provide a path for moving data between system modules. These lines are collectively called data bus.
- The address lines are used to designate the source/destination of data on data bus.
- The control lines are used to control the access to and the use of the data and address lines. Because data and address lines are shared by all components, there must be a means of controlling their use. Control signals transmit both command and timing signals indicate the validity of data and address information. Command signals specify operations to be performed. Control lines include memory read/write, i/o read/write, bus request/grant, clock, reset, interrupt request/acknowledge etc.

CHAPTER-2 BASIC COMPUTER ARCHITECTURE

2.1 8085 Microprocessor Architecture and Operations

The Intel 8085 A is a complete 8 bit parallel central processing unit. The main components of 8085A are array of registers, the arithmetic logic unit, the encoder/decoder, and timing and control circuits linked by an internal data bus. The block diagram is shown below:

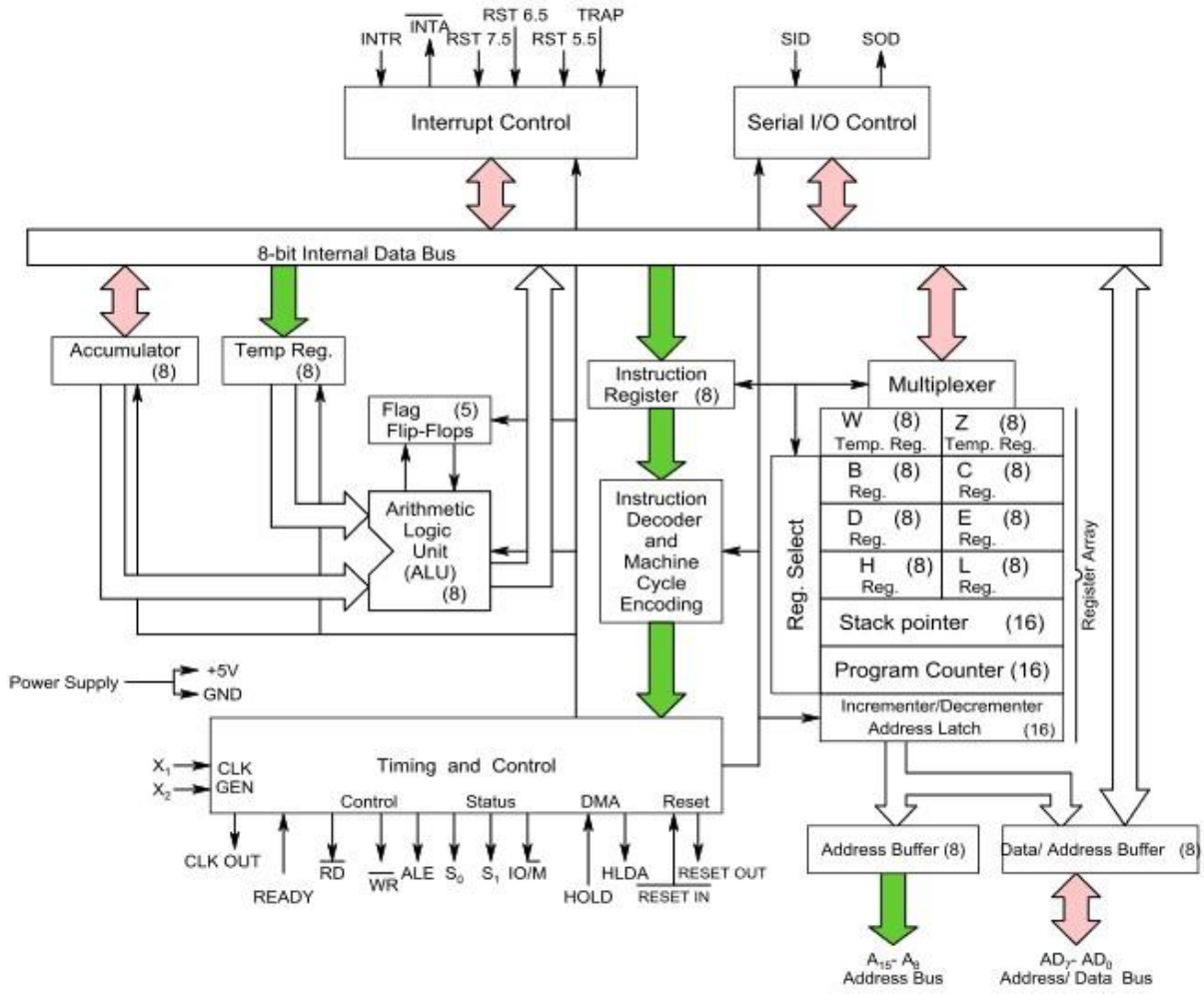


Fig: The 8085A microprocessor Functional Block Diagram

1: ALU: The arithmetic logic unit performs the computing functions, it includes the accumulator, the temporary register, the arithmetic and logic circuits and five flags. The temporary register is used to hold data during an arithmetic/logic operation. The result is

stored in the accumulator; the flags (flip-flops) are set or reset according to the result of the operation.

2. **Accumulator (register A):** It is an 8 bit register that is the part of ALU. This register is used to store the 8-bit data and to perform arithmetic and logic operations and 8085 microprocessor is called accumulator based microprocessor. When data is read from input port, it first moved to accumulator and when data is sent to output port, it must be first placed in accumulator.

3. **Temporary registers (W & Z):** They are 8 bit registers not accessible to the programmer. During program execution, 8085A places the data into it for a brief period.

4. **Instruction register (IR):** It is a 8 bit register not accessible to the programmer. It receives the operation codes of instruction from internal data bus and passes to the instruction decoder which decodes so that microprocessor knows which type of operation is to be performed.

5. **Register Array:** (Scratch pad registers B, C, D, E): It is a 8 bit register accessible to the programmers. Data can be stored upon it during program execution. These can be used individually as 8-bit registers or in pair BC, DE as 16 bit registers. The data can be directly added or transferred from one to another. Their contents may be incremented or decremented and combined logically with the content of the accumulator.

Register H & L: - They are 8 bit registers that can be used in same manner as scratch pad registers.

Stack Pointer (SP): - It is a 16 bit register used as a memory pointer. It points to a memory location in R/W memory, called the stack. The beginning of the stack is defined by loading a 16- bit address in the stack pointer.

Program Counter (PC): - Microprocessor uses the PC register to sequence the execution of the instructions. The function of PC is to point to the memory address from which the next byte is to be fetched. When a byte is being fetched, the PC is incremented by one to point to the next memory location.

6. **Flag register: -** Register consists of five flip-flops, each holding the status of different states separately is known as flag register and each flip-flop are called flags. 8085A can set or reset one or more of the flags and are sign(S), Zero (Z), Auxiliary Carry (AC) and Parity (P) and Carry (CY). The state of flags indicates the result of arithmetic and logical operations, which in turn can be used for decision making processes. The different flags are described as:

- **Carry (CY)**: - If the last operation generates a carry its status will 1 otherwise 0. It can handle the carry or borrow from one word to another.

- **Zero (Z):** - If the result of last operation is zero, its status will be 1 otherwise o. It is often used in loop control and in searching for particular data value.

- **Sign (S)**: - If the most significant bit (MSB) of the result of the last operation is 1 (negative), then its status will be 1 otherwise 0.

- **Parity (P)**: - If the result of the last operation has even number of 1's (even parity), its status will be 1 otherwise 0.

- **Auxiliary carry (AC)**: - If the last operation generates a carry from the lower half word (lower nibble), its status will be 1 otherwise 0. Used for performing BCD arithmetic.

Its bit position is shown in the following diagram:

D7	D6	D5	D4	D3	D2	D1	D0
S	Z	-	AC	-	P	-	CY

7. Timing and Control Unit:
This unit synchronizes all the microprocessor operations with the clock and generates the control signals necessary for communication between the microprocessor and peripherals. The control signals are similar to the sync pulse in an oscilloscope. The $\overline{RD}$ and $\overline{WR}$ signals are sync pulses indicating the availability of data on the data bus.

8. Interrupt controls: The various interrupt controls signals (INTR, RST 5.5, RST 6.5, RST 7.5 and TRAP) are used to interrupt a microprocessor.

9. Serial I/O controls: Two serial I/O control signals (SID and SOD) are used to implement the serial data transmission.

Addressing modes:

Instructions are command to perform a certain task in microprocessor. The instruction consists of op-code and data called operand. The operand may be the source only, destination only or both of them. In these instructions, the source can be a register, a memory or an input port. Similarly, destination can be a register, a memory location, or an output port. The various format (way) of specifying the operands are called addressing mode. So addressing mode specifies where the operands are located rather than their nature. The 8085 has 5 addressing mode:

1) Direct addressing mode:

The instruction using this mode specifies the effective address as part of instruction. The instruction size either 2-bytes or 3-bytes with first byte op-code followed by 1 or 2 bytes of address of data.

E. g. LDA 9500H A ← [9500]

IN 80H A ← [80]

This type of addressing is called absolute addressing.

2) Register Direct addressing mode:

This mode specifies the register or register pair that contains the data.

E g. MOV A, B

Here register B contains data rather than address of the data.

Other examples are: ADD, XCHG etc.

3) Register Indirect addressing mode:

In this mode the address part of the instruction specifies the memory whose contents are the address of the operand. So in this type of addressing mode, it is the address of the address rather than address itself. (One operand is register) e. g. MOV R, M

MOV M, R STAX,

LDAX etc.

STAX B B= 95 C =00 [9500] ← A

4) Immediate addressing mode:

In this mode, the operand position is the immediate data. For 8-bit data, instruction size is 2 bytes and for 16 bit data, instruction size is 3 bytes.

E.g. MVI A, 32H

LXI B, 4567H

5) Implied or Inherent addressing mode: The instructions of this mode do not have operands.

E.g. NOP: No operation HLT:

Halt

EI: Enable interrupt

DI: Disable interrupt

PIN Configuration of 8085

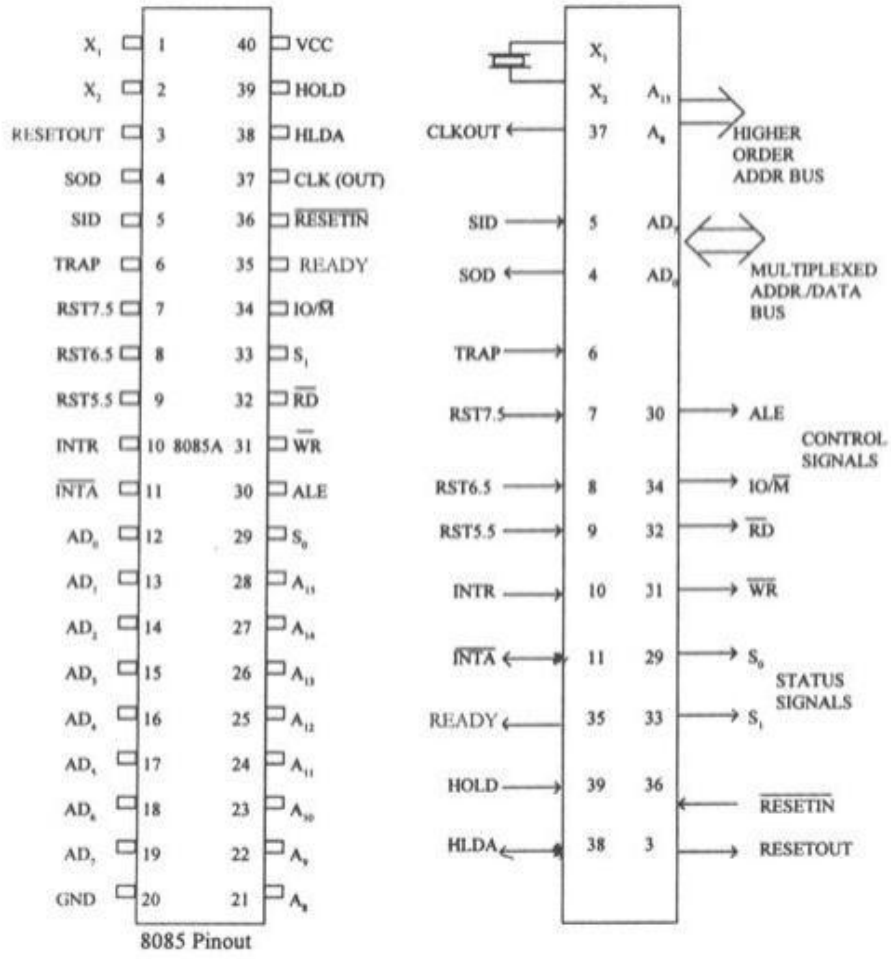


Fig (a) - Pin Diagram of 8085 & Fig(b) - logical schematic of Pin diagram

- The microprocessor is a clock-driven semiconductor device consisting of electronic logic circuits manufactured by using either a large-scale integration (LSI) or very-large-scale integration (VLSI) technique.

- The microprocessor is capable of performing various computing functions and making decisions to change the sequence of program execution.

- In large computers, a CPU implemented on one or more circuit boards performs these computing functions.

- The microprocessor is in many ways similar to the CPU, but includes the logic circuitry, including the control unit, on one chip.

- The microprocessor can be divided into three segments for the sake clarity, arithmetic/logic unit (ALU), register array, and control unit.

- 8085 is a 40 pin IC, DIP package. The signals from the pins can be grouped as follows:

1. Power supply and clock signals
2. Address bus
3. Data bus
4. Control and status signals
5. Interrupts and externally initiated signals
6. Serial I/O ports

1. Power supply and Clock frequency signals:

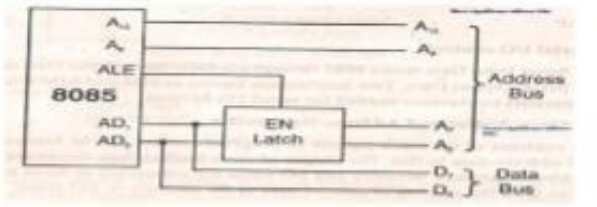
- Vcc: + 5 volt power supply
- Vss: Ground
- X1, X2: Crystal or R/C network or LC network connections to set the frequency of internal clock generator.
- The frequency is internally divided by two. Since the basic operating timing frequency is 3 MHz, a 6 MHz crystal is connected externally.
- CLK (output)-Clock Output is used as the system clock for peripheral and devices interfaced with the microprocessor.

2. Address Bus:

- A8 - A15
- It carries the most significant 8 bits of the memory address or the 8 bits of the I/O address.

3. Multiplexed Address / Data Bus:

- AD0 - AD7
- These multiplexed set of lines used to carry the lower order 8 bit address as well as data.
- During the opcode fetch operation, in the first clock cycle, the lines deliver the lower order address A0 - A7.
- In the subsequent IO / memory, read / write clock cycle the lines are used as data bus.
- The CPU may read or write out data through these lines.



4. **Control and Status signals:** These signals include two control signals (RD & WR) three status signals (IO/M, S1 and S0) to identify the nature of the operation and one special signal (ALE) to indicate the beginning of the operations.

   - **ALE (output)** - Address Latch Enable. This signal helps to capture the lower order address presented on the multiplexed address / data bus. When it is the pulse, 8085 begins an operation. It generates AD0 - AD7 as the separate set of address lines A0 – A7.

   - **RD (active low)** - Read memory or IO device. This indicates that the selected memory location or I/O device is to be read and that the data bus is ready for accepting data from the memory or I/O device.

   - **WR (active low)** - Write memory or IO device. This indicates that the data on the data bus is to be written into the selected memory location or I/O device.

   - **IO/M (output)** - Select memory or an IO device. This status signal indicates that the read / write operation relates to whether the memory or I/O device. It goes high to indicate an I/O operation. It goes low for memory operations.

$IO/\bar{M}$	S_1	S_0	States
0	0	1	Memory write
0	1	0	Memory read
1	0	1	I/O write
1	1	0	I/O read
0	1	1	Opcode fetch

5. **Interrupts & externally initiated signals:** Interrupts are the signals generated by external devices to request the microprocessor to perform a task. There are 5 interrupt

signals, i.e. TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR. We will discuss interrupts in detail in interrupts section.

- INTA: It is an interrupt acknowledgment signal.
- RESET IN: This signal is used to reset the microprocessor by setting the program counter to zero.
- RESET OUT: This signal is used to reset all the connected devices when the microprocessor is reset.
- READY: This signal indicates that the device is ready to send or receive data. If READY is low, then the CPU has to wait for READY to go high.
- HOLD: This signal indicates that another master is requesting the use of the address and data buses.
- HLDA (HOLD Acknowledge): It indicates that the CPU has received the HOLD request and it will relinquish the bus in the next clock cycle. HLDA is set to low after the HOLD signal is removed.

6. **Serial I/O signals:** There are 2 serial signals, i.e. SID and SOD and these signals are used for serial communication.
   - SOD (Serial output data line): The output SOD is set/reset as specified by the SIM instruction.
   - SID (Serial input data line): The data on this line is loaded into accumulator whenever a RIM instruction is executed.

Interrupts in 8085:

Interrupts are the signals generated by the external devices to request the Microprocessor to perform a task. There are 5 interrupt signals, i.e. TRAP, RST 7.5, RST 6.5, RST 5.5, and INTR.

Interrupt are classified into following groups based on their parameter:

- **Vector interrupt:** In this type of interrupt, the interrupt address is known to the processor. For example: RST7.5, RST6.5, RST5.5, TRAP.

- **Non-Vector interrupt:** In this type of interrupt, the interrupt address is not known to the processor so, the interrupt address needs to be sent externally by the device to perform interrupts. For example: INTR.

- **Maskable interrupt:** In this type of interrupt, we can disable the interrupt by writing some instructions into the program. For example: RST7.5, RST6.5, RST5.5.

- **Non-Maskable interrupt:** In this type of interrupt, we cannot disable the interrupt by writing some instructions into the program. For example: TRAP.

- **Software interrupt:** In this type of interrupt, the programmer has to add the instructions into the program to execute the interrupt. There are 8 software interrupts in 8085, i.e. RST0, RST1, RST2, RST3, RST4, RST5, RST6, and RST7.

- **Hardware interrupt:** There are 5 interrupt pins in 8085 used as hardware interrupts, i.e.
  TRAP, RST7.5, RST6.5, RST5.5, INTA.

Note: NTA is not an interrupt; it is used by the microprocessor for sending acknowledgement. TRAP has the highest priority, then RST7.5 and so on.

Interrupt Service Routine

A small program or a routine that when executed, services the corresponding interrupting source is called an ISR.

- **TRAP:** It is a non-maskable interrupt, having the highest priority among all interrupts. By default, it is enabled until it gets acknowledged. In case of failure, it executes as ISR and sends the data to backup memory. This interrupt transfers the control to the location 0024H.

- **RST7.5:** It is a maskable interrupt, having the second highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 003CH address.

- **RST 6.5:** It is a maskable interrupt, having the third highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 0034H address.

- **RST 5.5:** It is a maskable interrupt. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 002CH address.

- **INTR:** It is a maskable interrupt, having the lowest priority among all interrupts. It can be disabled by resetting the microprocessor.

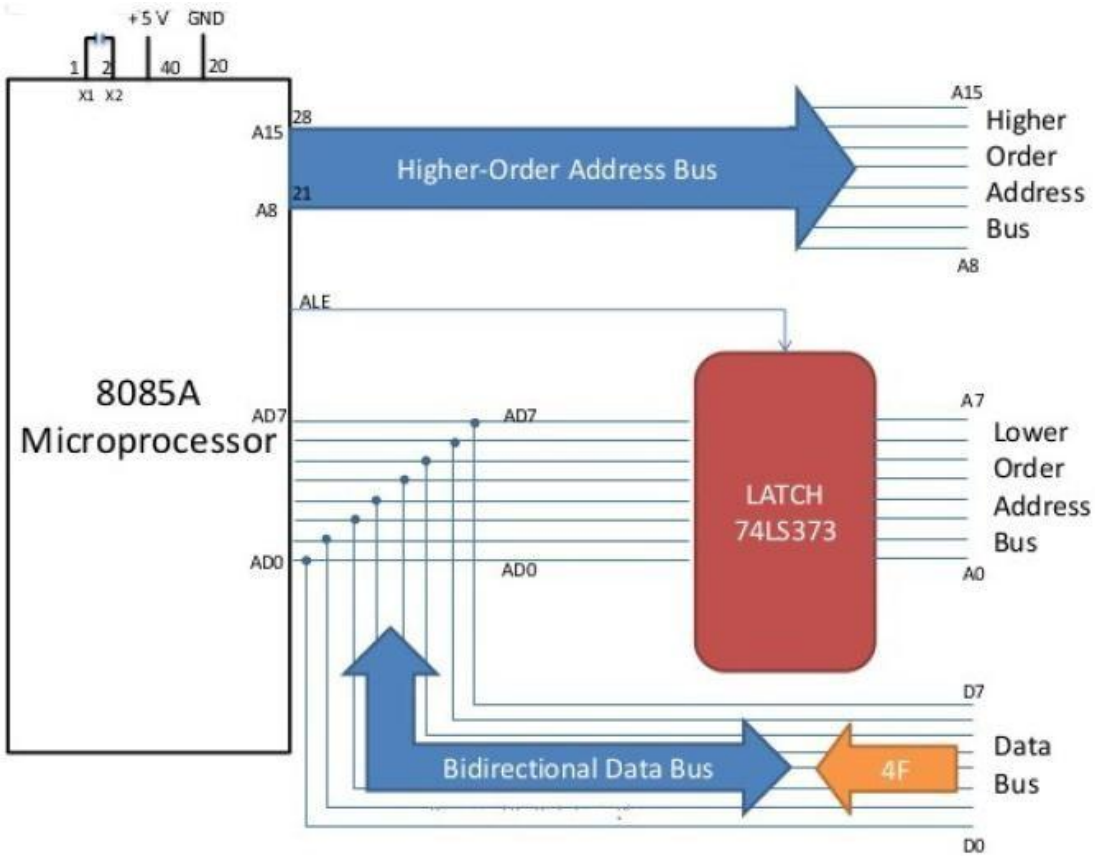
When **INTR signal goes high**, the following events can occur:

- The microprocessor checks the status of INTR signal during the execution of each instruction.

- When the INTR signal is high, then the microprocessor completes its current instruction and sends active low interrupt acknowledge signal.

- When instructions are received, then the microprocessor saves the address of the next instruction on stack and executes the received instruction.

Multiplexing and De-multiplexing of Address / Data bus:



- The address bus has 8 signal lines A8 -A15 which are unidirectional.

- The other 8 address bits are multiplexed (time shared) with the 8 data bits. So, the bits AD0 -AD7 are bi-directional and serve as A0-A7 and D0-D7 at the same time.

During the execution of the instruction, these lines carry the address bits during the early part, then during the late parts of the execution, they carry the 8 data bits.

- In order to separate the address from the data, we can use a latch to save the value before the function of the bits changes.

- The high order bits of the address remain on the bus for three clock periods. However, the low order bits remain for only one clock period and they would be lost if they are not saved externally. Also, notice that the low order bits of the address disappear when they are needed most.

- To make sure we have the entire address for the full three clock cycles, we will use an external latch to save the value of AD7-AD0 when it is carrying the address bits. We use the ALE(Address Enable Latch) signal to enable this latch.
- Given that ALE operates as a pulse during T1, we will be able to latch the address. Then when ALE goes low, the address is saved and the AD7-AD0 lines can be used for their purpose as the bi-directional data lines.

2.2 8086 Microprocessor Architecture and Operations

8086 Microprocessor is an enhanced version of 8085Microprocessor that was designed by Intel in 1976.It is a 16-bit Microprocessor having 20 address lines and 16 data lines that provides up to 1MB storage. It consists of powerful instruction set, which provides operations like multiplication and division easily.
It supports two modes of operation, i.e. Maximum mode and Minimum mode. Maximum mode is suitable for system having multiple processors and Minimum mode is suitable for system having a single processor.

Features of 8086

The most prominent features of a 8086 microprocessor are as follows:

- It has an instruction queue, which is capable of storing six instruction bytes from the memory resulting in faster processing.

- It was the first 16-bit processor having 16-bit ALU, 16-bit registers, internal data bus, and 16-bit external data bus resulting in faster processing.

- It is available in 3 versions based on the frequency of operation:

    - 8086 -> 5MHz
    - 8086-2 ->8MHz
    - 8086-1 ->10 MHz

- It uses two stages of pipelining, i.e. Fetch Stage and Execute Stage, which improves performance.

- Fetch stage can prefetch up to 6 bytes of instructions and stores them in the queue.

- Execute stage executes these instructions.

- It has 256 vectored interrupts.

- It consists of 29,000 transistors

Comparison between 8085 &8086 Microprocessor

- **Size:** 8085 is 8-bit microprocessor, whereas 8086 is 16-bit microprocessor.

- **Address Bus:** 8085 has 16-bit address bus while 8086 has 20-bit address bus.

- **Memory:** 8085 can access up to 64Kb, whereas 8086 can access up to 1 Mb of memory.

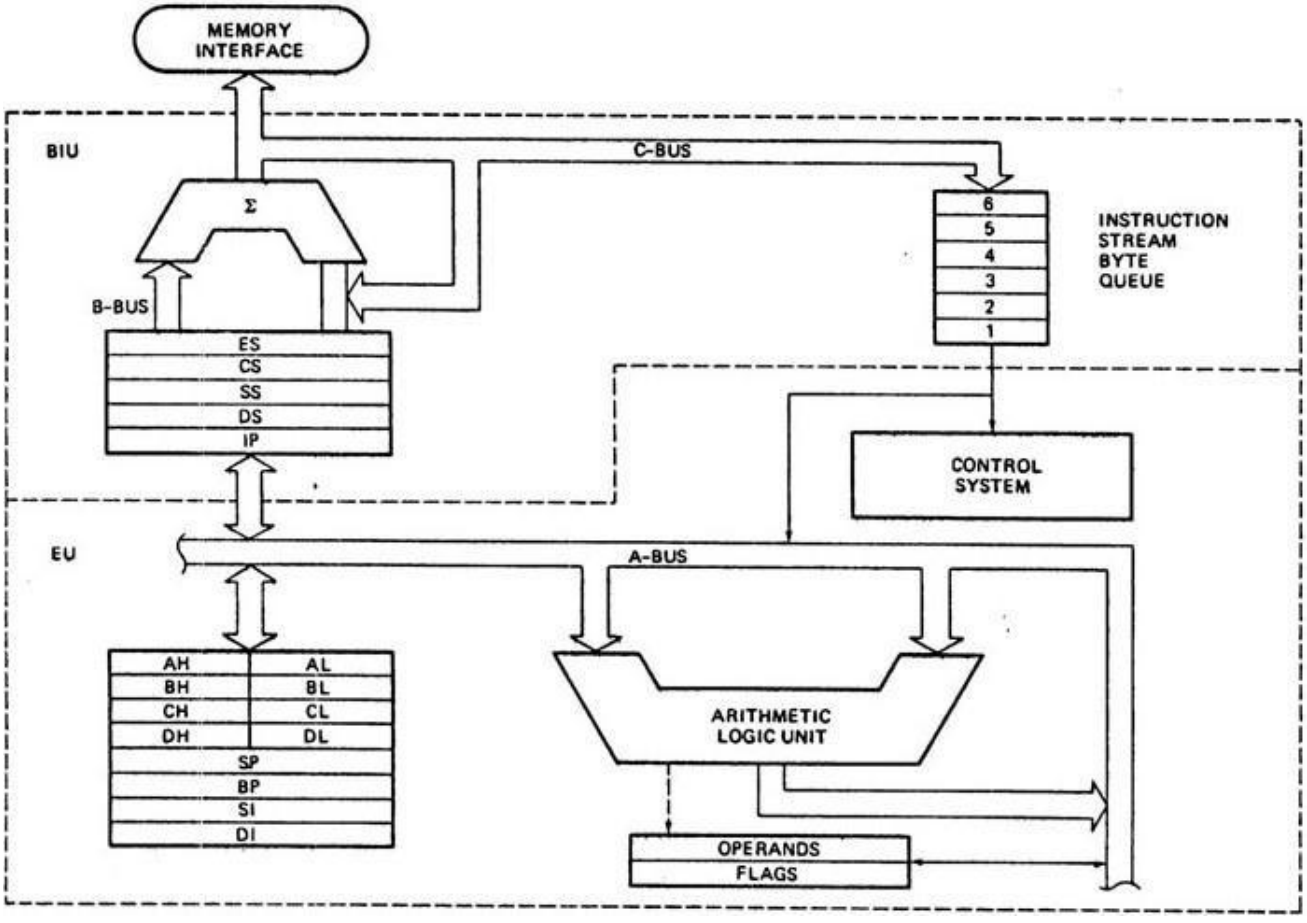
- **Instruction:** 8085 does not have an instruction queue, whereas 8086 has an instruction queue.
- **Pipelining:** 8085 does not support a pipelined architecture while 8086 supports a pipelined architecture.

- **I/O:** 8085 can address 2^8 = 256 I/O's, whereas 8086 can access 2^{16} = 65,536 I/O's.

- **Cost:** The cost of 8085 is low whereas that of 8086 is high.

Architecture of 8086

The following diagram depicts the architecture of an 8086 Microprocessor:



8086 Microprocessor is divided into two functional units, i.e., EU (Execution Unit) and BIU (Bus Interface Unit).

EU (Execution Unit)

Execution unit gives instructions to BIU stating from where to fetch the data and then decode and execute those instructions. Its function is to control operations on data using the instruction decoder & ALU. EU has no direct connection with system buses as shown in the above figure, it performs operations over data through BIU. Let us now discuss the functional parts of 8086 microprocessors.

ALU: It handles all arithmetic and logical operations, like +,-, ×, /, OR, AND, NOT operations.

Flag Register: It is a 16-bit register that behaves like a flip-flop, i.e. it changes its status according to the result stored in the accumulator. It has 9 flags and they are divided into 2 groups: Conditional Flags and Control Flags.

Conditional Flags: It represents the result of the last arithmetic or logical instruction executed. Following is the list of conditional flags:

- **Carry flag:** This flag indicates an overflow condition for arithmetic operations.

- **Auxiliary flag:** When an operation is performed at ALU, it results in a carry/barrow from lower nibble (i.e. D0 – D3) to upper nibble (i.e. D4 – D7), then this flag is set, i.e. carry given by D3 bit to D4 is AF flag. The processor uses this flag to perform binary to BCD conversion.

- **Parity flag:** This flag is used to indicate the parity of the result, i.e. when the lower order 8-bits of the result contains even number of 1's, then the Parity Flag is set. For odd number of 1's, the Parity Flag is reset.

- **Zero flag:** This flag is set to 1 when the result of arithmetic or logical operation is zero else it is set to 0.

- **Sign flag:** This flag holds the sign of the result, i.e. when the result of the operation is negative, then the sign flag is set to 1 else set to 0.

- **Overflow flag:** This flag represents the result when the system capacity is exceeded.

Control Flags: Control flags controls the operations of the execution unit. Following is the list of control flags:

- **Trap flag:** It is used for single step control and allows the user to execute one instruction at a time for debugging. If it is set, then the program can be run in a single step mode.

- **Interrupt flag:** It is an interrupt enable/disable flag, i.e. used to allow/prohibit the interruption of a program. It is set to 1 for interrupt enabled condition and set to 0 for interrupt disabled condition.

- **Direction flag:** It is used in string operation. As the name suggests when it is set then string bytes are accessed from the higher memory address to the lower memory address and vice-a-versa.

General purpose register:

There are 8 general purpose registers, i.e., AH, AL, BH, BL, CH, CL, DH, and DL. These registers can be used individually to store 8-bit data and can be used in pairs to store 16bit data. The valid register pairs are AH and AL, BH and BL, CH and CL, and DH and DL. It is referred to the AX, BX, CX, and DX respectively.

- **AX register:** It is also known as accumulator register. It is used to store operands for arithmetic operations.

- **BX register:** It is used as a base register. It is used to store the starting base address of the memory area within the data segment.

- **CX register:** It is referred to as counter. It is used in loop instruction to store the loop counter.

- **DX register:** This register is used to hold I/O port address for I/O instruction.

Stack pointer register:

It is a 16-bit register, which holds the address from the start of the segment to the memory location,
Where a word was most recently stored on the stack.

BIU (Bus Interface Unit)

BIU takes care of all data and addresses transfers on the buses for the EU like sending addresses, fetching instructions from the memory, reading data from the ports and the memory as well as writing data to the ports and the memory. EU has no direction connection with System Buses so this is possible with the BIU. EU and BIU are connected with the Internal Bus. It has the following functional parts:

- **Instruction queue:** BIU contains the instruction queue. BIU gets upto 6 bytes of next instructions and stores them in the instruction queue. When EU executes instructions and is ready for its next instruction, then it simply reads the instruction from this instruction queue resulting in increased execution speed.

- Fetching the next instruction while the current instruction executes is called **pipelining.**

- **Segment register:** BIU has 4 segment buses, i.e. CS, DS, SS & ES. It holds the addresses of instructions and data in memory, which are used by the processor to access memory locations. It also contains 1 pointer register IP, which holds the address of the next instruction to execute by the EU.

- **CS:** It stands for Code Segment. It is used for addressing a memory location in the code segment of the memory, where the executable program is stored.

    - **DS:** It stands for Data Segment. It consists of data used by the program and is accessed in the data segment by an offset address or the content of other register that holds the offset address.

    - **SS:** It stands for Stack Segment. It handles memory to store data and addresses during execution.

    - **ES:** It stands for Extra Segment. ES is additional data segment, which is used by the string to hold the extra destination data.

- **Instruction pointer (IP):** It is a 16-bit register used to hold the address of the next instruction to be executed.

8086 MEMORY ORGANIZATION

1. INTRODUCTION

- The Intel 8086 is a 16 bit Microprocessor that is intended to be used as the CPU in a Microcomputer.

- The 8086 has a 20 bit address bus so it can address any one of 2^{20} or 1,048,576 memory locations.

- Each of the 1,048,576 memory address of the 8086 represents a byte-wide location.

- 16 bit word will be stored in two consecutive memory locations.

- If the first byte of a word is at an even address, the 8086 can read the entire word in one operation.

- If the first byte of a word is at an odd address, the 8086 will read the first byte with one bus cycle and the second byte with another bus cycle.

2. ACCESSING DATA IN MEMORY

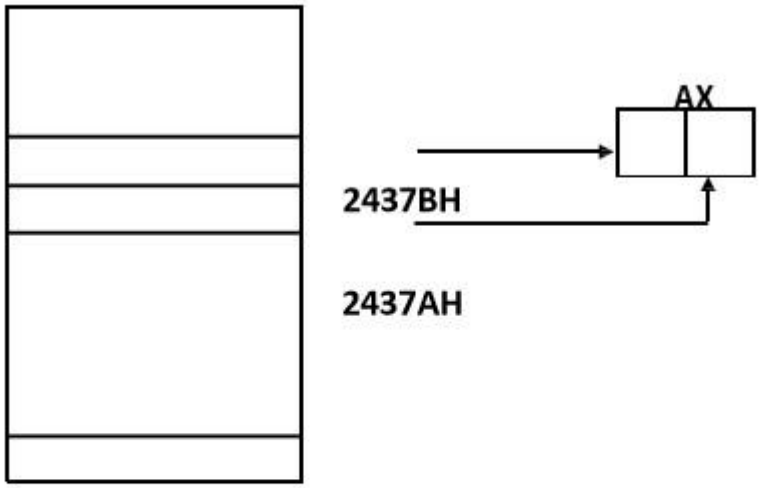
- An important point here is that an 8086 always stores the low byte of word in lower address and stores high byte of word in higher address.

- Low Byte – Low Address : High Byte – High Address
- MOV AX,[437AH]

- Assume DS=2000H

- To Compute the physical address Add 20000 H and 437AH 20000 H + 437A H = 2437AH

- 2437A H is the physical address.



20000 H [Start of Data Segment i.e. DS=2000H]

3. SEGMENTED MEMORY

- The 8086 BIU sends out 20-bit address so it can address any of 2^{20} or 1,048,576 bytes in memory.
- However at any given time the 8086 works with only four 65536 bytes (64 Kbyte) segment within this 1,048,576 byte (1 Mbyte) Range
- Four segments are : Code Segment, Stack Segment, Data Segment and Extra Segment
- Four segment registers in BIU are used to hold the upper 16 bits of the starting address of 4 memory segments that the 8086 is working with at a particular time.
- The 4 segment registers are code segment register (CS), stack segment register (SS), data segment register (DS) and the extra segment register (ES).
- For small programs which do not need all 64 Kbytes in each segment can overlap.
- For example, the code segment holds the upper 16 bits of the starting address for the segment from which the BIU is currently fetching instruction code bytes.
- The BIU always inserts zero for the lowest 4 bits of the 20-bit starting address.

- If the code segment register contains 348A H then the code segment will start at address 348A0 H

- A 64 Kbytes segment can be located anywhere within the 1 Mbyte address space, but the segment will always start at an address with zeros in the lowest 4 bits

ADVANTAGES OF SEGMENTATION [SEGMENT: OFFSET SCHEME]

Intel designed the 8086 family devices to access memory using the segment: offset approach rather than accessing memory directly with 20 bit. The advantages are listed below.

- The segment: offset scheme requires only a 16-bit number to represent the base address for a segment and only a 16 bit offset to access any location in a segment. This means that 8086 has to manipulate and store only 16-bit quantities instead of 20-bit quantities.

- This makes easier interface with 8 and 16-bit wide memory boards and with 16 bit registers in the 8086.

- It allows programs to be relocated in memory system. A relocatable program is one that can be placed in any area of memory and executed without change.

- It allows programs written to function in the real mode to operate in protected mode.

- Segmentation also makes easy to keep user's program and data separate from one another and segmentation makes it easy to switch from one user's program to another user's program.

DISADVANTAGE OF SEGMENT: OFFSET APPROACH

- The segment: offset scheme introduces complexity in hardware and software design.

DIFFERENT SEGMENT OFFSET COMBINATION

SEGMENT	OFFSET	SPECIAL PURPOSE
CS [CODE SEGMENT]	IP [INSTRUCTION POINTER]	INSTRUCTION
SS [STACK SEGMENT]	SP [STACK POINTER] BP [BASE POINTER]	ADDRESS
DS [DATA SEGMENT]	BX [BASE REGISTER] DI [DESTINATION INDEX] SI [SOURCE INDEX] 8 BIT NUMBER 16 BIT NUMBER	STACK ADDRESS
ES [EXTRA SEGMENT]		

Memory Banking:

- A memory bank is a designated section of computer memory used for storing data.
- In 8086 there is 20 bit address bus, so it can address 1,048,576 addresses. At each address we can store 8 bit address (1-byte)but if want to write a word(16-bit)into a memory segment to store data in byte form then we write the data in two consecutive memory address which are even(low) and odd(high) memory.
- The 8086 memory address space can be viewed as a sequence of one million bytes in which any byte may contain an 8-bit data element and any two consecutive bytes may contain a 16-bit data element.
- There is no constraint on byte or word address boundaries.
- The address space is physically connected to a 16-bit data bus by dividing the address space into two 8-bit banks of up to 512K bytes each.

Even Memory bank

- One bank is connected to the lower half of the 16-bit data bus (D0 – D7) and contains even address bytes. i.e., when A0 bit is low, the bank is selected.
- To access memory bytes from Even address, information is transferred over the lower half of the data bus (D0 - D7). The A0 is output LOW and BHE is output HIGH enabling only the even address bank.

Odd Memory bank

- The other bank is connected to the upper half of the data bus (D8 - D15) and contains odd address bytes. i.e., when A0 is high and BHE (Bus High Enable) is low, the odd bank is selected.
- To access memory byte from an odd address information, is transferred over the higher half of the data bus (D8 - D15). The BHE output low enables the upper memory bank. A0 is output high to disable the lower memory bank.

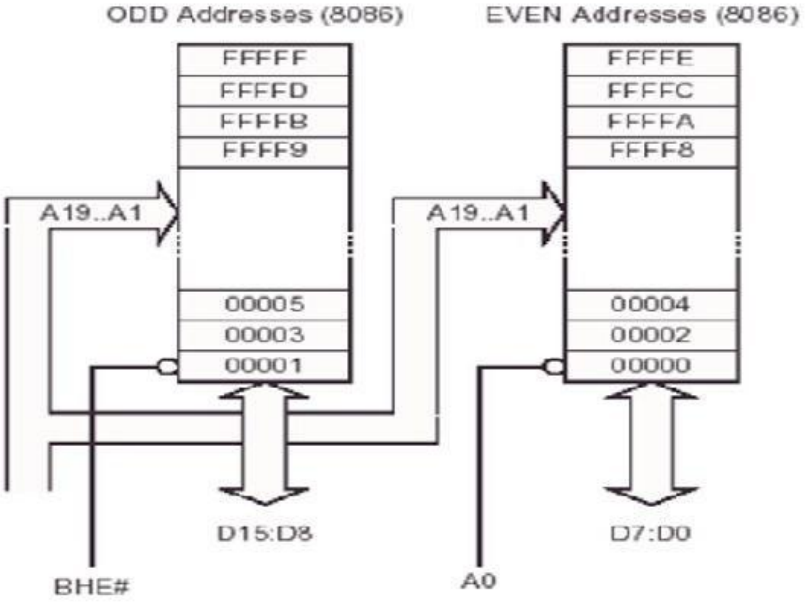
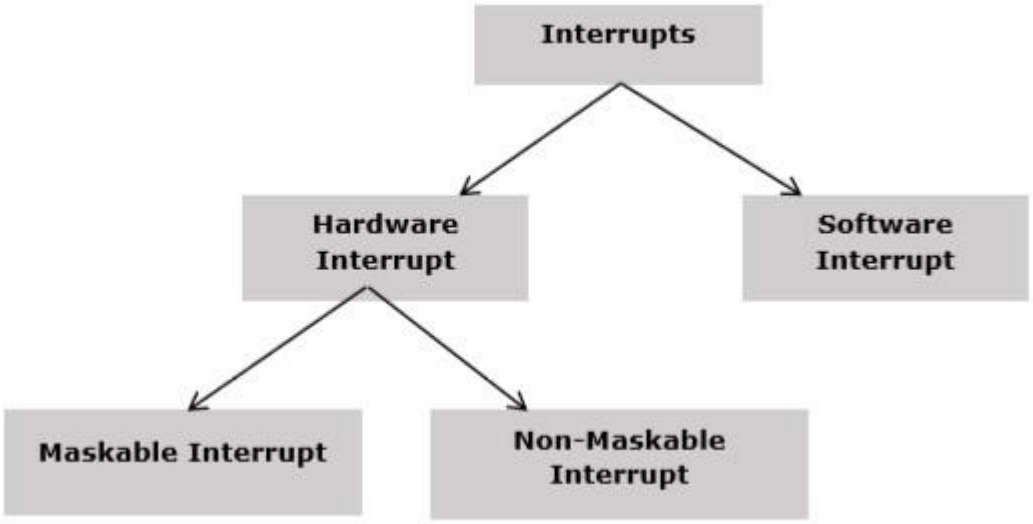


Figure: Memory Bank

8086 – Interrupts

Interrupt is the method of creating a temporary halt during program execution and allows peripheral devices to access the microprocessor. The microprocessor responds to that interrupt with an ISR (Interrupt Service Routine), which is a short program to instruct the microprocessor on how to handle the interrupt.
The following image shows the types of interrupts we have in an 8086 microprocessor:



Hardware Interrupts:

Any peripheral device causes hardware interrupt by sending a signal through a specified pin to the microprocessor. The 8086 has two hardware interrupt pins, i.e. NMI and INTR. NMI is a non-maskable interrupt and INTR is a maskable interrupt having lower priority. One more interrupt pin associated is INTA called interrupt acknowledge.

NMI:
It is a single non-maskable interrupt pin (NMI) having higher priority than the maskable interrupt request pin (INTR) and it is of type 2 interrupt.
When this interrupt is activated, these actions take place:

- Completes the current instruction that is in progress.
- Pushes the Flag register values on to the stack.
- Pushes the CS (code segment) value and IP (instruction pointer) value of the return address on to the stack.

- IP is loaded from the contents of the word location 00008H.
- CS is loaded from the contents of the next word location 0000AH.
- Interrupt flag and trap flag are reset to 0.

INTR:

The INTR is a maskable interrupt because the microprocessor will be interrupted only if interrupts are enabled using set interrupt flag instruction. It should not be enabled using clear interrupt Flag instruction.

The INTR interrupt is activated by an I/O port. If the interrupt is enabled and NMI is disabled, then the microprocessor first completes the current execution and sends '0' on INTA pin twice. The first '0' means INTA informs the external device to get ready and during the second '0' the microprocessor receives the 8 bit, say X, from the programmable interrupt controller.

These actions are taken by the microprocessor:

- First completes the current instruction.
- Activates INTA output and receives the interrupt type, say X.
- Flag register value; CS value of the return address and IP value of the return address are pushed on to the stack.
- IP value is loaded from the contents of word location X x 4
- CS is loaded from the contents of the next word location.
- Interrupt flag and trap flag is reset to 0

SOFTWARE INTERRUPTS:

Some instructions are inserted at the desired position into the program to create interrupts. These interrupt instructions can be used to test the working of various interrupt handlers. It includes:

INT- Interrupt instruction with type number It is 2-byte instruction. First byte provides the op-code and the second byte provides the interrupt type number. There are 256 interrupt types under this group.

Its execution includes the following steps:

- Flag register value is pushed on to the stack.
- CS value of the return address and IP value of the return address are pushed on to the stack.

- IP is loaded from the contents of the word location 'type number' x 4

- CS is loaded from the contents of the next word location.

- Interrupt Flag and Trap Flag are reset to 0.

The starting address for type0 interrupt is 000000H, for type1, interrupt is 00004H similarly for type2 is 00008H and …so on. The first five pointers are dedicated interrupt pointers. i.e.:

- TYPE 0 interrupt represents division by zero situation.

- TYPE 1 interrupt represents single-step execution during the debugging of a program.

- TYPE 2 interrupt represents non-maskable NMI interrupt.

- TYPE 3 interrupt represents break-point interrupt.

- TYPE 4 interrupt represents overflow interrupt.

The interrupts from Type 5 to Type 31 are reserved for other advanced microprocessors, and interrupts from 32 to Type 255 are available for hardware and software interrupts.

INT 3-Break Point Interrupt Instruction:

It is a 1-byte instruction having op-code is CCH. These instructions are inserted into the program so that when the processor reaches there, then it stops the normal execution of program and follows the break-point procedure.
Its execution includes the following steps:

- Flag register value is pushed on to the stack.

- CS value of the return address and IP value of the return address are pushed on to the stack.

- IP is loaded from the contents of the word location 3x4 = 0000CH

- CS is loaded from the contents of the next word location.

- Interrupt Flag and Trap Flag are reset to 0.

INTO - Interrupt on overflow instruction:

It is a 1-byte instruction and their mnemonic INTO. The op-code for this instruction is CEH. As the name suggests it is a conditional interrupt instruction, i.e. it is active only when the overflow flag is set to 1 and branches to the interrupt handler whose interrupt type number is 4. If the overflow flag is reset then, the execution continues to the next instruction.
Its execution includes the following steps:

- Flag register values are pushed on to the stack.
- CS value of the return address and IP value of the return address are pushed on to the stack.
- IP is loaded from the contents of word location 4x4 = 00010H
- CS is loaded from the contents of the next word location.
- Interrupt flag and Trap flag are reset to 0 .

8086 – Addressing Modes

The different ways in which a source operand is denoted in an instruction is known as **addressing modes.** Different addressing modes are required in programs for processing implied numbers, constants, variables, and arrays and hence the flexibility to the programmer to access the operand from memory in different ways. There are 7 different addressing modes in 8086 programming:

1. Register Addressing Mode
2. Immediate Addressing Mode
3. Direct Addressing Mode
4. Register Indirect Addressing Mode
5. Base plus Index Addressing Mode
6. Register Relative Addressing Mode
7. Base Relative Plus Index Addressing Mode

1. REGISTER ADDRESSING MODE

- It is the most common form of data addressing.
- Transfers a copy of a byte/word from source register to destination register.

INSTRUCTION	SOURCE	DESTINATION
MOV AX,BX	REGISTER BX	REGISTER AX

- It is carried out with 8 bit registers AH,AL,BH,BL,CH,CL,DH & DL or with 16 bit registers AX,BX,CX,DX,SP,BP,SI and DI.
- It is important to use registers of same size.
- Never mix an 8 bit register with a 16 bit register i.e. MOV AX,BL

EXAMPLES
MOV AL,BL : Copys BL into AL MOV ES,DS
: Copys DS into ES
MOV AX,CX : Copys CX into AX

2. IMMEDIATE ADDRESSING MODE

- The term immediate implies that the data immediately follow the hexadecimal opcode in the memory.
- Note that immediate data are constant data.
- It transfers the source immediate byte/word of data in destination register or memory location.

INSTRUCTION	SOURCE	DESTINATION
MOV CH,3AH	DATA 3AH	REGISTER AX

EXAMPLES
MOV AL,90 : Copys 90 into AL
MOV AX,1234H : Copys 1234H into AX
MOV CL,10000001B : Copys 100000001 binary value into CL

3. DIRECT ADDRESSING MODE

- In this scheme, the address of the data is defined in the instruction itself.
- When a memory location is to be referenced, its offset address must be specified

INSTRUCTION	SOURCE	DESTINATION

MOV AL,[1234H]	ASSUME DS=1000H 10000 H + 1234 H 11234H MEMORY LOCATION	REGISTER AL

EXAMPLES
MOV AL,[1234H] : Copys the byte content of data segment memory location 11234H into AL.

MOV AL, NUMBER : Copys the byte content of data segment memory location NUMBER into AL.

4. REGISTER INDIRECT ADDRESSING MODE

- Register Indirect Addressing allows data to be addressed at any memory location through an offset address held in any of the following registers: BP, BX, DI and SI.
- The Index and Base registers are used to specify the address of data.
- It transfers byte/word between a register and a memory location addressed by an index or base registers.
- The symbol [ ] denote indirect addressing.
- The data segment is used by default with register indirect addressing or any other addressing mode that uses BX,DI or SI to address memory. If BP register addresses memory, the stack segment is used by default.

INSTRUCTION	SOURCE	DESTINATION
MOV CL,[BX]	ASSUME DS=1000H ASSUME BX=0300H 10000 H + 0300 H 10300H	REGISTER CL

MEMORY LOCATION

EXAMPLES
MOV CX,[BX] : Copys the word contents of the data segment memory location addressed by BX into CX.

MOV [DI],BH : Copys BH into the data segment memory location addressed by DI.

MOV [DI],[BX] : Memory to Memory moves are not allowed except with string instructions.

5. BASE PLUS INDEX ADDRESSING MODE

- Base plus index addressing is similar to indirect addressing because it indirectly addresses memory data
- This type of addressing uses one base register (BP or BX) and one Index Register (DI or SI) to indirectly address memory.

INSTRUCTION	SOURCE	DESTINATION
MOV [BX+SI],CL	REGISTER CL	ASSUME DS=1000H ASSUME BX=0300H ASSUME SI =0200H 10000H + 0300H + 0200H 10500H MEMORY LOCATION

EXAMPLES

MOV CX,[BX+DI] : Copys the word contents of the data segment memory location addressed by BX plus DI into CX.

MOV CH,[BP+SI] : Copys the byte contents of the stack segment memory location addressed by BP plus SI into CH

6. REGISTER RELATIVE ADDRESSING MODE

- In this case, the data in a segment of memory are addressed by adding the displacement to the content of base or an index register (BP,BX ,DI or SI).

- Transfers a byte/word between a register and the memory location addressed by an index or base register plus a displacement.

INSTRUCTION	SOURCE	DESTINATION
MOV [BX+4],CL	REGISTER CL	ASSUME DS=1000H ASSUME BX=0300H 10000H + 0300H + 4H 10304H

EXAMPLES

MOV ARRAY[SI],BL : Copys BL into the data segment memory location addressed by ARRAY plus SI.

MOV LIST[SI+2],CL : Copys CL into the data segment memory location addressed by sum of LIST, SI and 2.

7. BASE RELATIVE PLUS INDEX ADDRESSING MODE

- The base relative plus index addressing mode is similar to the base plus index addressing mode but it adds a displacement to form a memory address.
- Transfers a byte or word between a register and the memory location addressed by a base and an index register plus a displacement

INSTRUCTION	SOURCE	DESTINATION
MOV [BX+SI+05],CL	REGISTER CL	ASSUME DS=1000H ASSUME BX=0300H ASSUME SI =0200H 10000H + 0300H + 0200H +05H 10505H

EXAMPLES

MOV LIST[BP+DI],CL : Copys CL into the stack segment memory location addressed by the sum of LIST, BP and DI

MOV DH,[BX+DI+20H] : Copys the byte contents of the data segment memory location addressed by the sum of BX,DI and 20H into DH

Pipelining:

Pipelining is an implementation technique where multiple instructions are overlapped in execution. The computer pipeline is divided in**stages**. Each stage completes a part of an instruction in parallel. The stages are connected one to the next to form a pipe - instructions enter at one end, progress through the stages, and exit at the other end.

Pipelining does not decrease the time for individual instruction execution. Instead, it increases instruction throughput. The **throughput** of the instruction pipeline is determined by how often an instruction exits the pipeline.

Because the pipe stages are hooked together, all the stages must be ready to proceed at the same time. We call the time required to move an instruction one step further in the pipeline ***a machine cycle*** . The ***length*** of the machine cycle is determined by the time required for the slowest pipe stage.

The pipeline designer's *goal is to balance the length of each pipeline stage* . If the stages are perfectly balanced, then the time per instruction on the pipelined machine is equal to

$$\frac{\text{Time per instruction on nonpipelined machine}}{\text{Number of pipe stages}}$$

Under these conditions, the speedup from pipelining equals the number of pipe stages. Usually, however, the stages will not be perfectly balanced; besides, the pipelining itself involves some overhead.

Instruction Pipeline:

Any architecture can be pipelined by making each clock cycle into a pipe stage.

Clock #	1	2	3	4	5	6	7
Instruction *i*	Fetch	Decode	Execute				
Instr. *i*+1		Fetch	Decode	Execute			
Instr. *i*+2			Fetch	Decode	Execute		
Instr. *i*+3				Fetch	Decode	Execute	
Instr. *i*+4					Fetch	Decode	Execute

Here instruction i is fetched in clock #1. After it has been fetched it is decoded in clock #2 and at the same time next instruction i.e. instr. i+1 is fetched. At Clock#3, instruction i is executed, instr. i+1 is decoded and instr. i+2 is fetched and so on.

Hence at the end of clock #3, instruction i is executed. Sometime later at the end of clock #4 instruction i+1 is executed.
If we assume that unpipelined architecture took 3ns then this pipelined architecture will take 1ns to finish a stage (fetch, decode and execute).

Advantages of pipelining:

- The execution unit always reads the next instruction byte from the queue in BIU. This is faster than sending out an address to the memory and waiting for the next instruction byte to come.

- In short pipelining eliminates the waiting time of EU and speeds up the processing. The 8086 BIU will not initiate a fetch unless and until there are two empty bytes in queue. 8086 BIU normally obtains two instruction bytes per fetch.

CHAPTER 3 - INSTRUCTION CYCLE

Instruction cycle:

The necessary steps that the CPU carries out to fetch an instruction and necessary data from the memory and to execute it constitute and instruction cycle it is defined as the time required to complete the execution of an instruction.

An instruction cycle consists of fetch cycle and execute cycle. In fetch cycle CPU fetches opcode from the memory. The necessary steps which are carried out to fetch an opcode from memory constitute a fetch cycle. The necessary steps which are carried out to get data if any from the memory and to perform the specific operation specified in an instruction constitute and execute cycle. The total time required to execute an instruction given by IC = Fc+ Ec the 8085 consists of 1-6 machine cycles or operations.

Fetch cycle:

The first byte of an instruction is its opcode. The program counter keeps the memory address of the next instruction to be executed in the beginning of fetch cycle the content of the program counter, which is the address of the memory location where opcode is available, is send to the memory. The memory places the opcode on the data bus so as to transfer it to CPU. The entire process takes 3 clock cycle.

Execute cycle/Operation:

The opcode fetched from the memory goes to IR from the IR it goes to the decoder which decodes instruction. After the instruction is decoded execution begins.

- If the operand is in general purpose register, execution is performed immediately. I,e in one clock cycle.
- If an instruction is contains data or operand address, then CPU has to perform some read operations to get the desired data.
- In some instruction write operation is performed. In write cycle data are sent from the CPU to the memory of an o/p device.
- In some cases execute cycle may involve one or more read or write cycle or both.

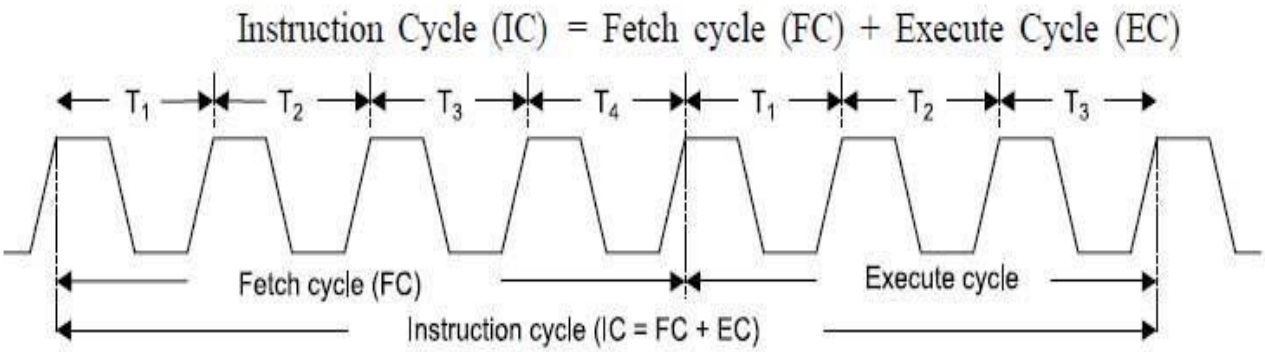


Fig. 5.2 (*a*) Processor cycle

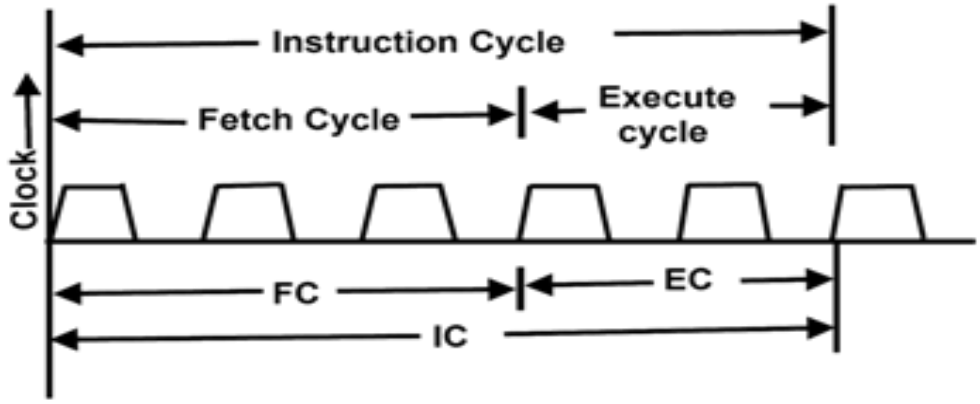


Figure (a) Instruction cycle showing FC, EC and IC

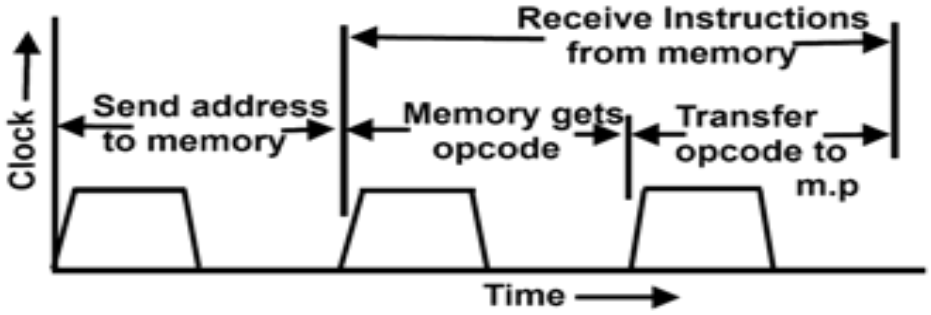


Figure (b) A Typical Fetch Cycle

Figure 4

Machine cycle:

It is defined as the time required to complete one operation of accessing memory i/p, o/p or acknowledging and external request. This cycle may consists of 3 to 6 T states. T-states: It is defined as one sub division of the operation performed in one clock period. These sub division are internal states synchronized with system clock and each T states precisely equal to one clock period.

Machine cycles of 8085

The 8085 microprocessor has 5 (seven) basic machine cycles. They are

- ✓ Opcode fetch cycle (4T)
- ✓ Memory read cycle (3 T)
- ✓ Memory write cycle (3 T)
- ✓ I/O read cycle (3 T)
- ✓ I/O write cycle (3 T)
- ✓ Interrupt

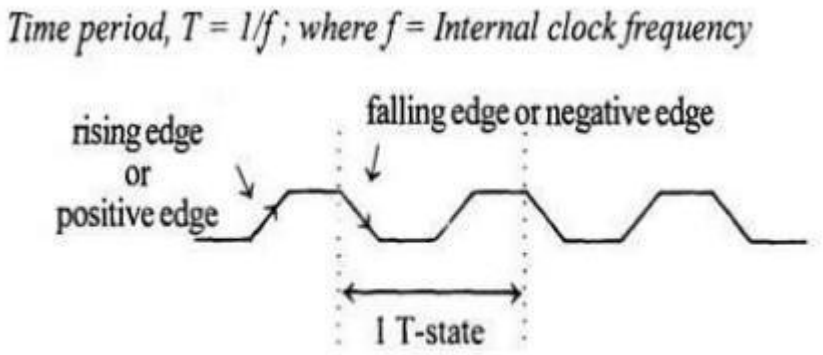


Fig 1.7 Clock Signal

MACHINE CYCLE STATUS AND CONTROL SIGNALS

Table 5.1(*a*) Machine cycle status and control signals

Machine cycle	*Status*			*Controls*		
	IO / $\overline{M}$	S_1	S_0	$\overline{RD}$	$\overline{WR}$	$\overline{INTA}$
Opcode Fetch (OF)	0	1	1	0	1	1
Memory Read	0	1	0	0	1	1
Memory Write	0	0	1	1	0	1
I/O Read (I/OR)	1	1	0	0	1	1
I/O Write (I/OW)	1	0	1	1	0	1
Acknowledge of INTR (INTA)	1	1	1	1	1	0
BUS Idle (BI) : DAD	0	1	0	1	1	1
ACK of RST, TRAP	1	1	1	1	1	1
HALT	Z	0	0	Z	Z	1
HOLD	Z	X	X	Z	Z	1
X ⇒ Unspecified, and Z ⇒ High impedance state						

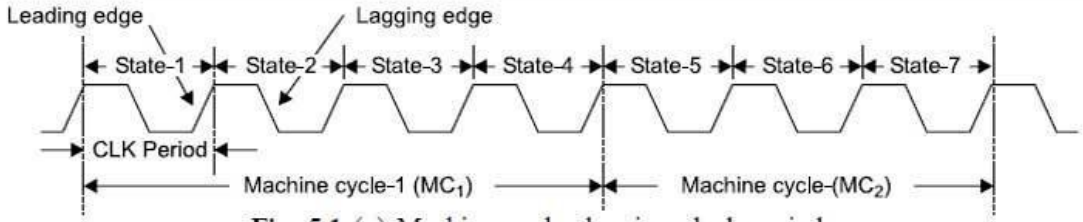


Fig. 5.1 (*a*) Machine cycle showing clock periods

1. Opcode Fetch Cycle :

The first machine cycle of every instruction is opcode fetch cycle in which the 8085 finds the nature of the instruction to be executed. In this machine cycle, processor places the contents of the Program Counter on the address lines, and through the read process, reads the opcode of the instruction. Fig. 1.15 (a) (See Fig. on next page) shows flow of data (opcode) from memory to the microprocessor and Fig. 1,15 (b) shows the timing diagram for opcode fetch machine cycle. The length of this cycle is not fixed. It varies from 4T states to 6T states as per the instruction. The following section describes the opcode fetch cycle in step by step manner.

Step 1 : (State T_1) In T_1 state, the 8085 places the contents of program counter on the address bus. The high-order byte of the PC is placed on the A_8-A_{15} lines. The low-order byte of the PC is placed on the AD_0 – AD_7 lines which stays on only during T_1. Thus

microprocessor activates ALE (Address Latch Enable) which is used to latch the low-order byte of the address in external latch before it disappears.

In T_1, 8085 also sends status signals IO/M, S_1, and S_0. IO/M specifies whether it is a memory or I/O operation, S_i status specifies whether it is read/write operation; S_1 and S_0 together indicates read, write, opcode fetch, machine cycle operation, or whether it is in HALT state. In opcode fetch machine cycle status signals are : IO/M = 0, S_1 = 1, S_0 = 1.

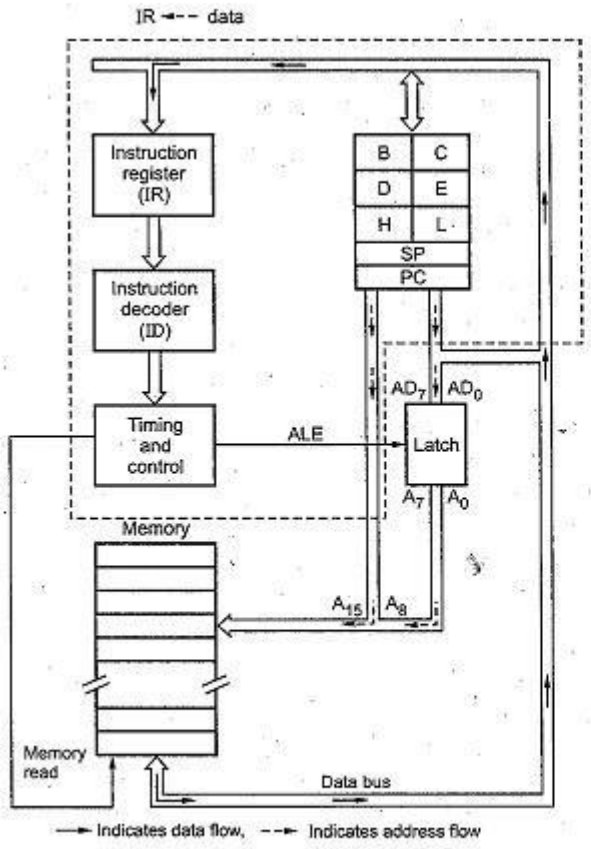


Fig. 1.15 (a) Data (opcode) flow from memory to micrprocessor

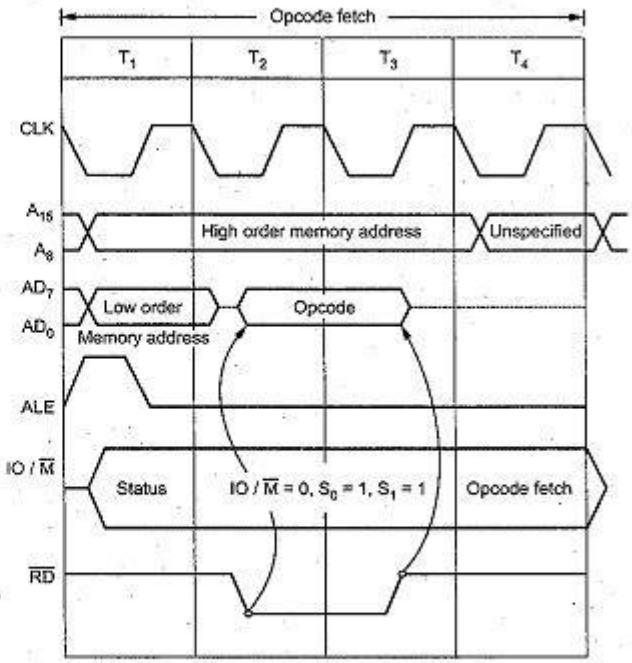


Fig. 1.15 (b) Opcode fetch machine cycle

Step 2 : (State T_2) In T_2, low-order address disappears from the AD_0 – AD_7 lines. (However A_o – A_7 remain available as they were latched during T_1). In T_2, 8085 sends RD signal low to enable the addressed memory location. The memory device then places the contents of addressed memory location on the data bus (AD_o – AD_7).

Step 3 : (State T_3) During T_3, 8085 loads the data from the data bus in its Instruction Register and raises RD to high which disables the memory device.

Step 4 : (State T_4) In T_4, microprocessor decodes the opcode, and on the basis of the instruction received, it decides whether to enter state T_5 or to enter state T_1 of the next

machine cycle. One byte instructions those operate on eight bit data (8 bit operand) are executed in T_4.

For example : MOV A, B, ANA D, ADD B, INR L, DCR C, RAL and many more.

Note : For one byte instructions which operate on eight bit data, data is always available in the internal memory of 8085 i.e. registers.

Step 5 : (State T_5 and T_6)

State T_5 and T_6, when entered, are used for internal microprocessor operations required by the instruction. During T_5 and T_6 8085 performs stack write, internal 16 bit; and conditional return operations depending upon the type of instruction. One byte instructions those operate on sixteen bit data (16 bit operand) are executed in T_5 and T_6.
For example DCX H, PCHL, SPHL, INX H, etc.

2. Memory Read Cycle :

The 8085 executes the memory read cycle to read the contents of R/W memory or ROM. The length of this machine cycle is 3-T states (T_1 – T_3). In this machine cycle, processor places the address on the address lines from the stack pointer, general purpose register pair or program counter, and through the read process, reads the data from the addressed memory location. Fig. 1.16 (a) (See Fig. on next page) shows flow of data from memory to the microprocessor and Fig. 1.16 (b) shows the timing diagram for memory read machine cycle. Memory read machine cycle is similar to the opcode fetch machine cycle. However, they use only states T_1 to T_3, and the status signal values (IO/M = 0, S_1 = 1, S_0 = 0) appropriate for memory read machine cycle are issued in T_1.

The following section describes the memory read machine cycle in step by step manner.

Step 1 : (State T_1.) In T_1 state, microprocessor places the address on the address lines from stack pointer, general purpose register pair or program counter and activates ALE signal in order to latch low-order byte of address.

During T_1, 8085 sends status signals : IO/M = 0, S_1 = 1, and S_0 = 0 for memory read machine cycle.

Step 2 : (State T_2) In T_2, 8085 sends RD signal low to enable the addressed memory location. The memory device then places the contents of addressed memory location on the data bus (AD_0 -AD_7).

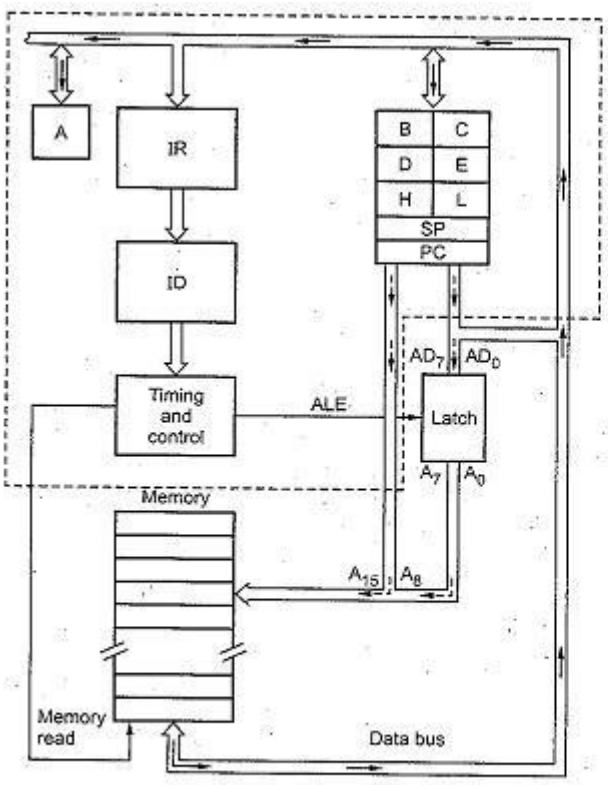


Fig. 1.16 (a) Data flow from memory to microprocessor

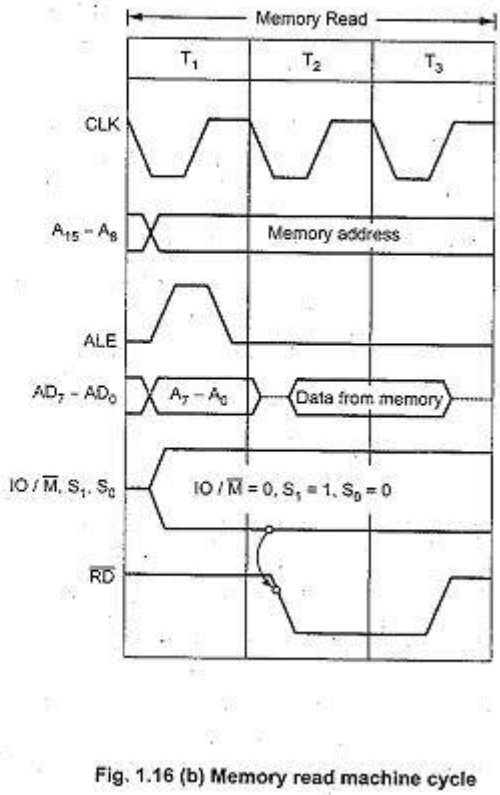


Fig. 1.16 (b) Memory read machine cycle

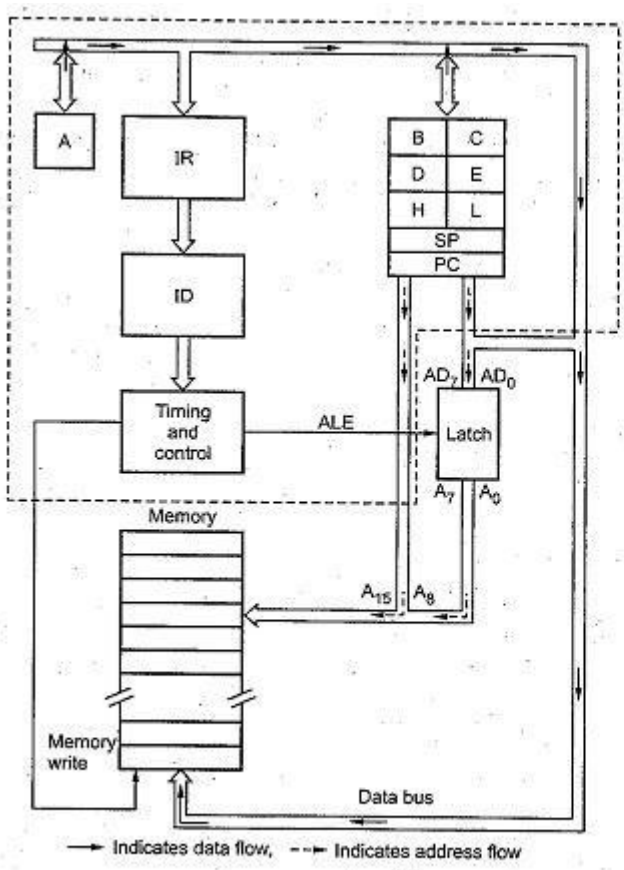


Fig. 1.17 (a) Data flow microprocessor to memory

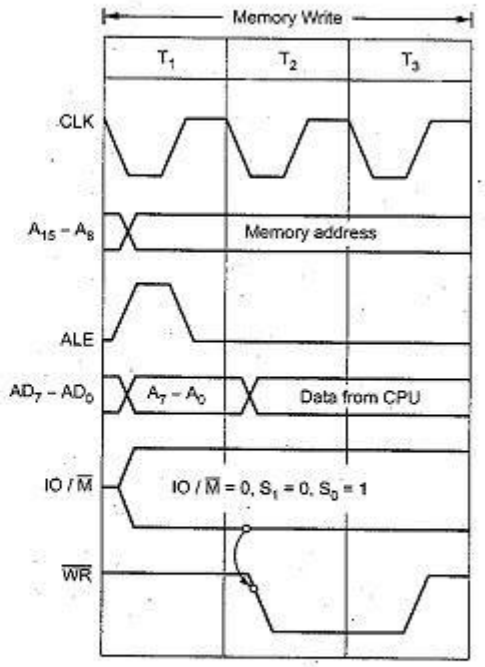


Fig. 1.17 (b) Memory write machine cycle

Step 3 : (State T_3) During T_3, 8085 loads the data from the data bus into specified register (F, A, B, C, D, E, H, and L) and raises RD to high which disables the memory device.

3. Memory Write Cycle

The 8085 executes the memory write cycle to store the data into data memory or stack memory. The length of this machine cycle is 3T states. (T_1 — T_3). In this machine cycle, processor places the address on the address lines from the stack pointer or general purpose register pair and through the write process, stores the data into the addressed memory location. Fig. 1.17 (See Fig. on previous page) shows the timing diagram for memory write machine cycle. The memory write timing diagram is similar to the memory read timing diagram, except that instead of RD, WR signal goes low during T_2 and T_3. The status signals for memory write cycle are : IO/M = 0, S_1 = 0, S_0 = 1. The following section describes the memory write machine cycle in step by step manner.

Step 1 : (State T_1) In T_1 state, the 8085 places the address on the address lines from stack pointer or general purpose register pair and activates ALE signal in order to latch loworder byte of address. During T_1, 8085 sends status signals :

IO/M = 0, S_1 = 0 and S_0 = 1 for memory write machine cycle.

Step 2 : (State T_2) In T_2, 8085 places data on the data bus and sends WR signal low for writing into the addressed memory location.

Step 3 : (State T_3) During T_3, WR signal goes high, which disables the memory device and terminates the write operation.

4, 5. I/O Read and I/O Write cycles

The I/O read and I/O write machine cycles are similar to the memory read and memory write machine cycles, respectively, except that the 10/M signal is high for I/O read and I/O write machine cycles. High IO/M signal indicates that it is an I/O operation. Fig. 1.18 (b) and Fig. 1.19 (b) show the timing diagrams for I/O read and I/O write cycles, respectively.

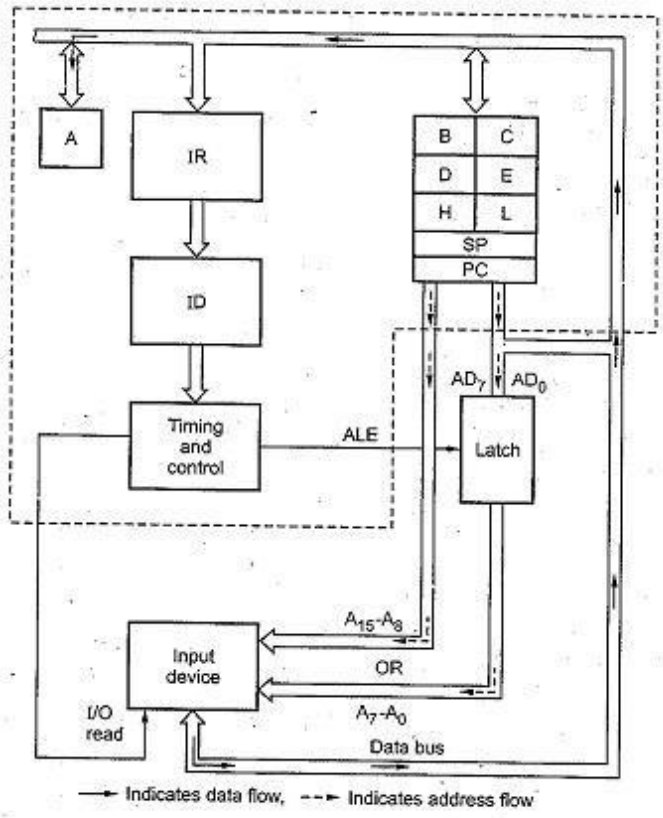


Fig. 1.18 (a) Data flow from input device to microprocessor

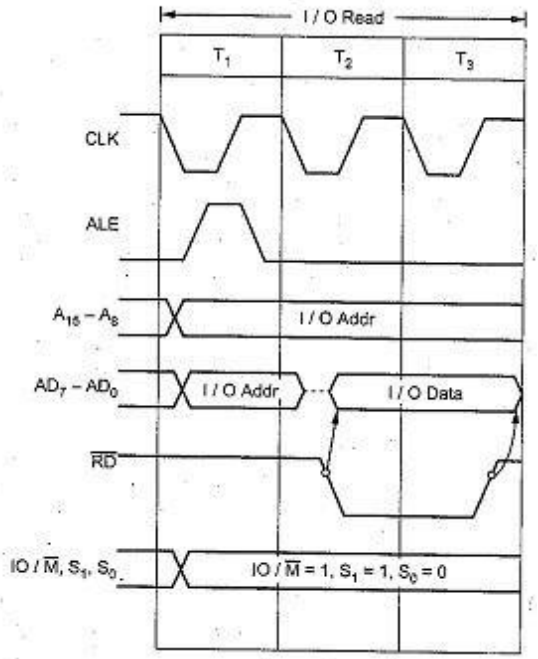


Fig. 1.18 (b) I/O read memory cycle

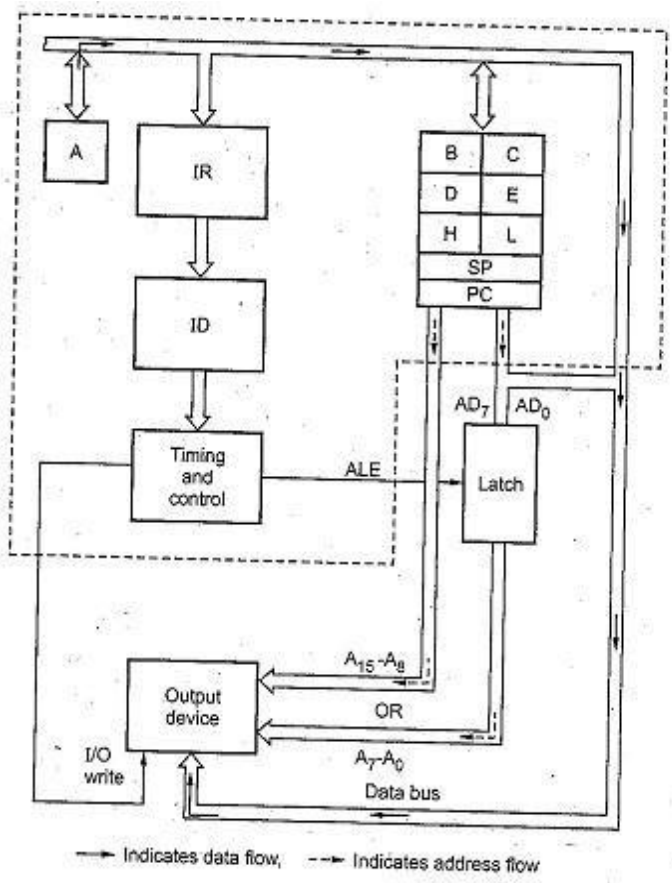


Fig. 1.19 (a) Data flow from microprocessor to output device

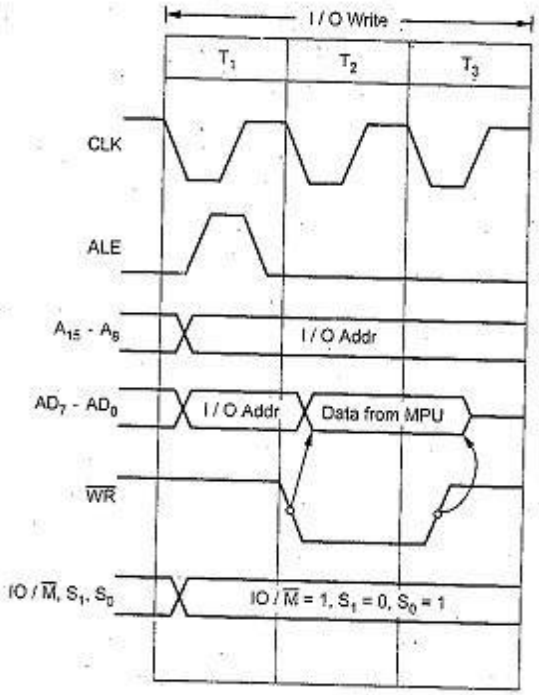


Fig. 1.19 (b) I/O write machine cycle

6. Interrupt Acknowledge Cycle

In response to INTR signal, 8085 executes interrupt acknowledge machine cycle to read an instruction from the external device. Theoretically, the external device can place any instruction on the data bus in response to INTA. However, only RST and CALL, save the PC contents (return address) before transferring control to the interrupt service routine. The next sections explain interrupt acknowledge cycles for RST and CALL instructions.

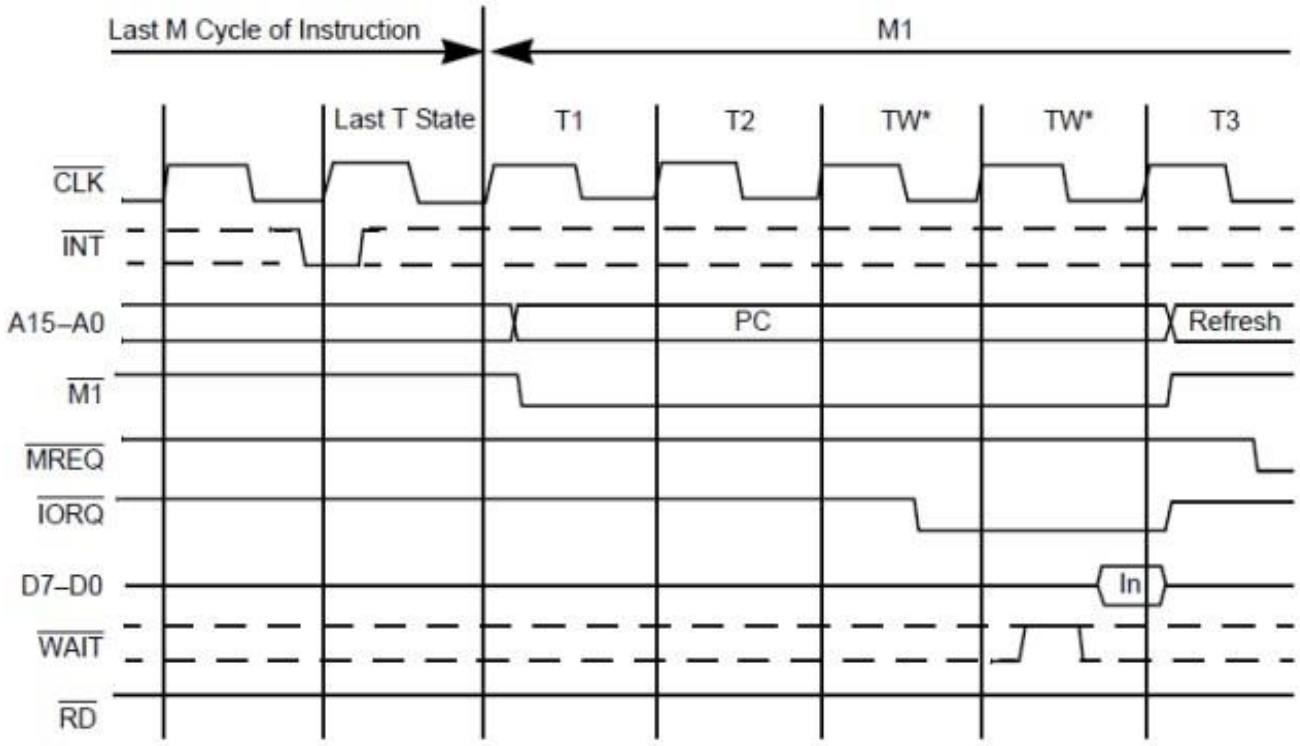


Figure 9. Interrupt Request/Acknowledge Cycle

Timing Diagrams:

1. MOV

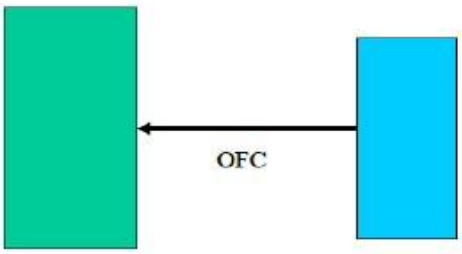
Timing Diagram

Instruction:

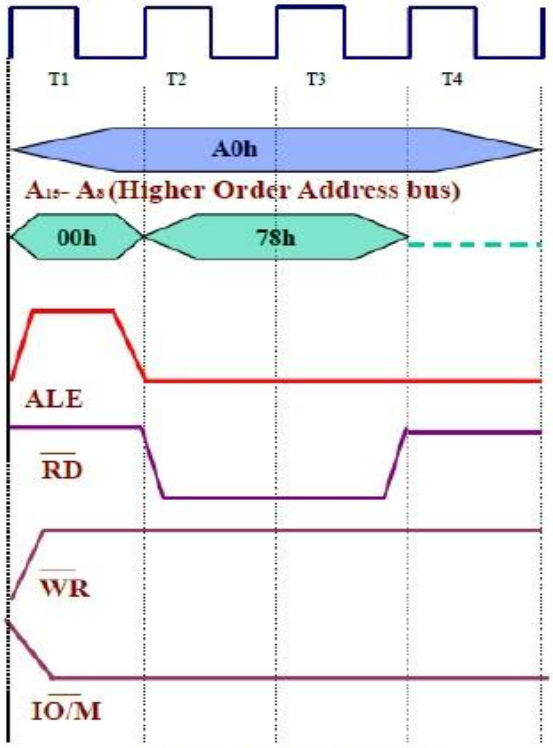
A000h MOV A,B

Corresponding Coding:

A000h 78



8085 Memory



Op-code fetch Cycle

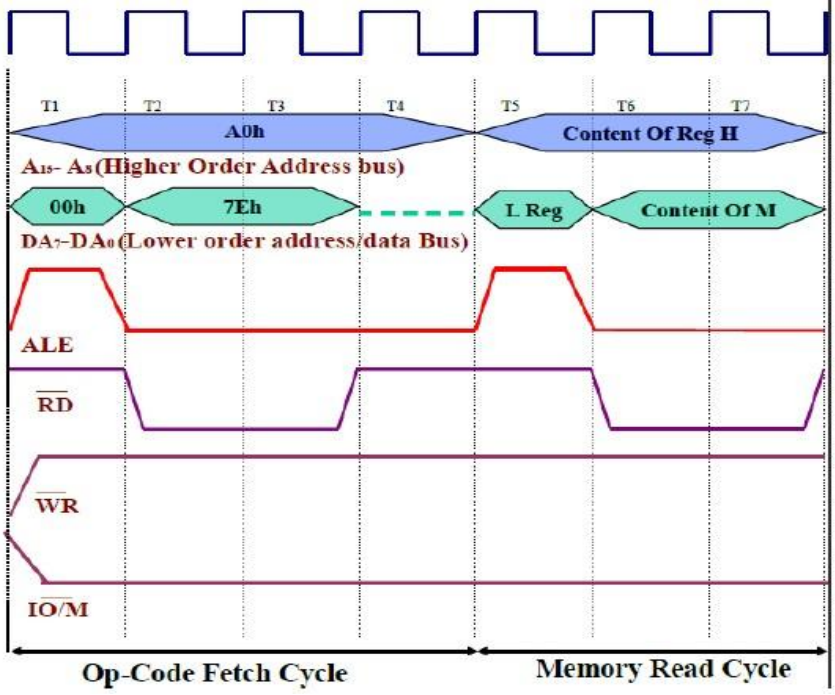
Timing Diagram

Instruction:

A000h MOV A,M

Corresponding Coding:

A000h 7E

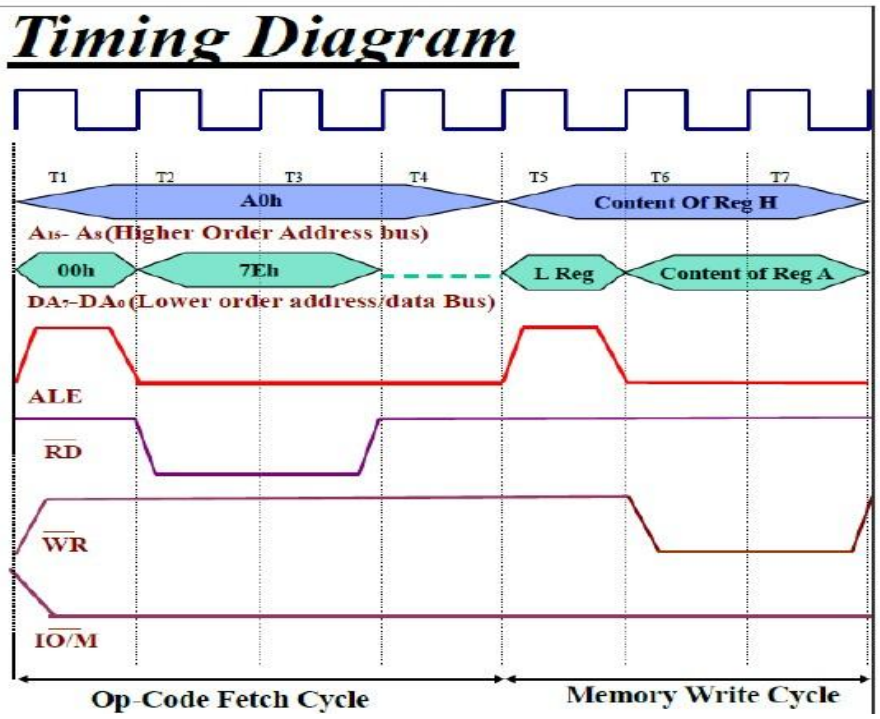


Instruction:

A000h MOV M,A

Corresponding Coding:

A000h 77



2. MVI

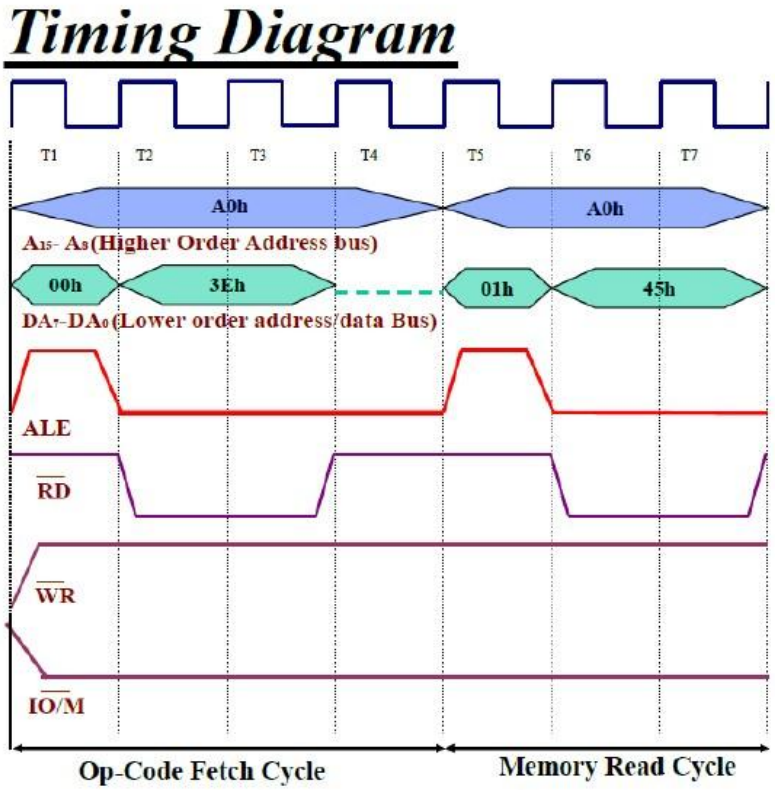
Instruction:

A000h MVI A,45h

Corresponding Coding:

A000h 3E

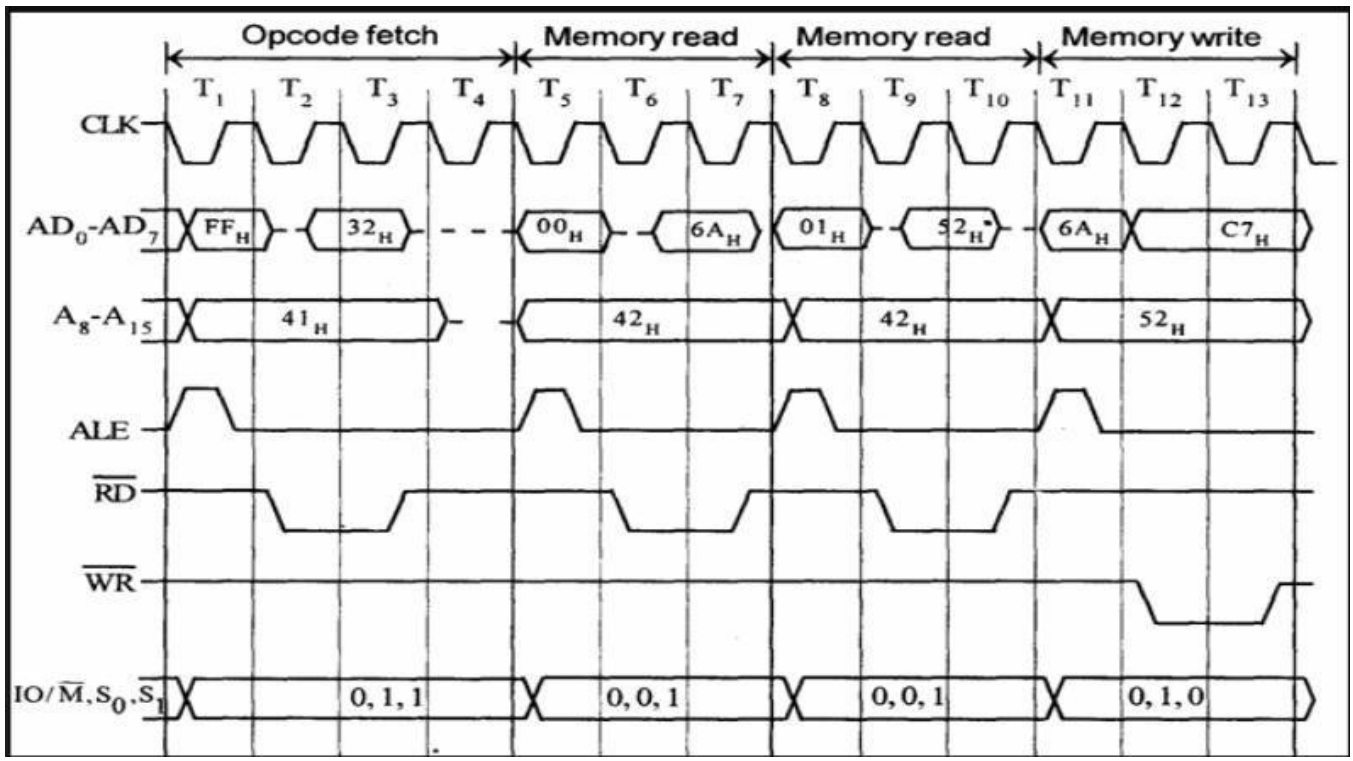
A001h 45



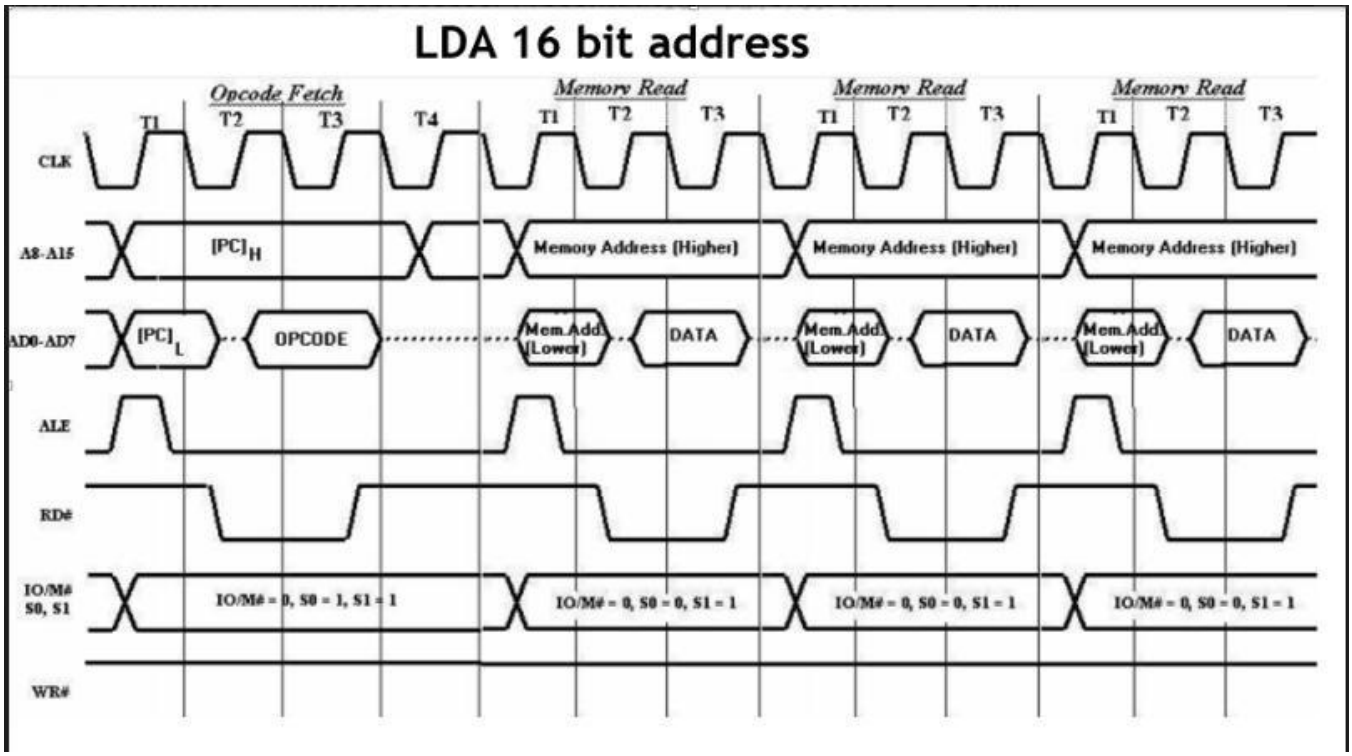
3. STA 526AH

- STA means Store Accumulator -The contents of the accumulator is stored in the specified address(526A).
- The opcode of the STA instruction is said to be 32H. It is fetched from the memory 41FFH(see fig). - *OF machine cycle*
- Then the lower order memory address is read(6A). - *Memory Read Machine Cycle*
- Read the higher order memory address (52).- *Memory Read Machine Cycle*
- The combination of both the addresses are considered and the content from accumulator is written in 526A. - *Memory Write Machine Cycle*
- Assume the memory address for the instruction and let the content of accumulator is C7H. So, C7H from accumulator is now stored in 526A.

Address	Mnemonics	Opcode
41FF	STA 526A$_H$	32$_H$
4200		6A$_H$
4201		52$_H$


Opcode fetch
Memory read
Memory read
Memory write
T1 T2 T3 T4 T5 T6 T7 T8 T9 T10 T11 T12 T13
CLK
AD0-AD7
FFH 32H 00H 6AH 01H 52H 6AH C7H
A8-A15
41H 42H 42H 52H
ALE
RD
WR
IO/M, S0, S1
0, 1, 1 0, 0, 1 0, 0, 1 0, 1, 0

4. LDA



5. IN CO H

- Fetching the Opcode DBH from the memory 4125H.
- Read the port address C0H from 4126H.
- Read the content of port C0H and send it to the accumulator. Let the content of port is 5EH.

Address	Mnemonics	Opcode
4125	IN $C0_H$	DB_H
4126		$C0_H$

Instruction:

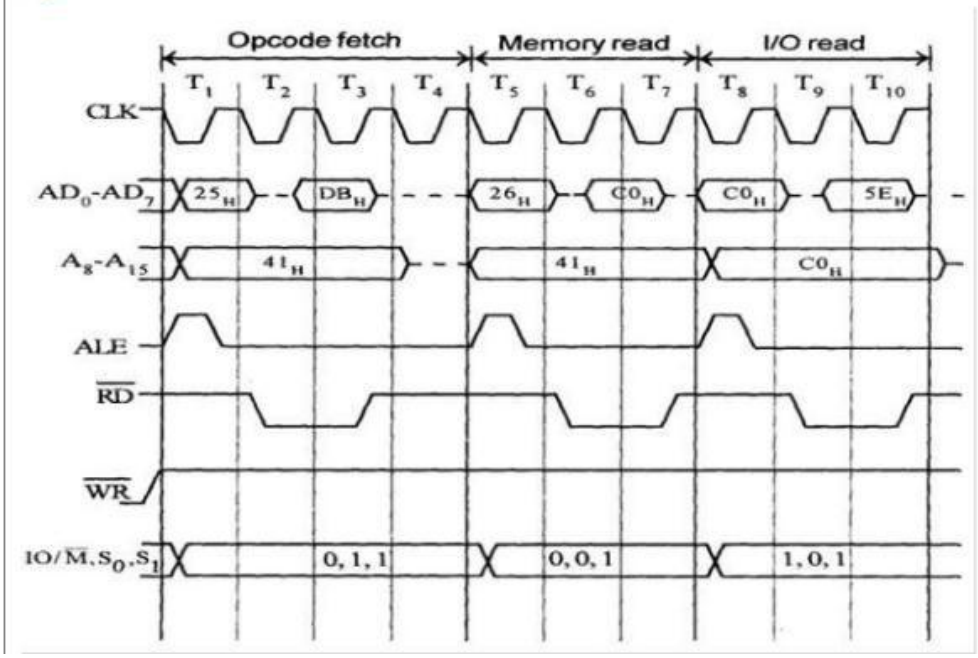
IN C0H

IN
Opcode: DBH
Address: 4125H

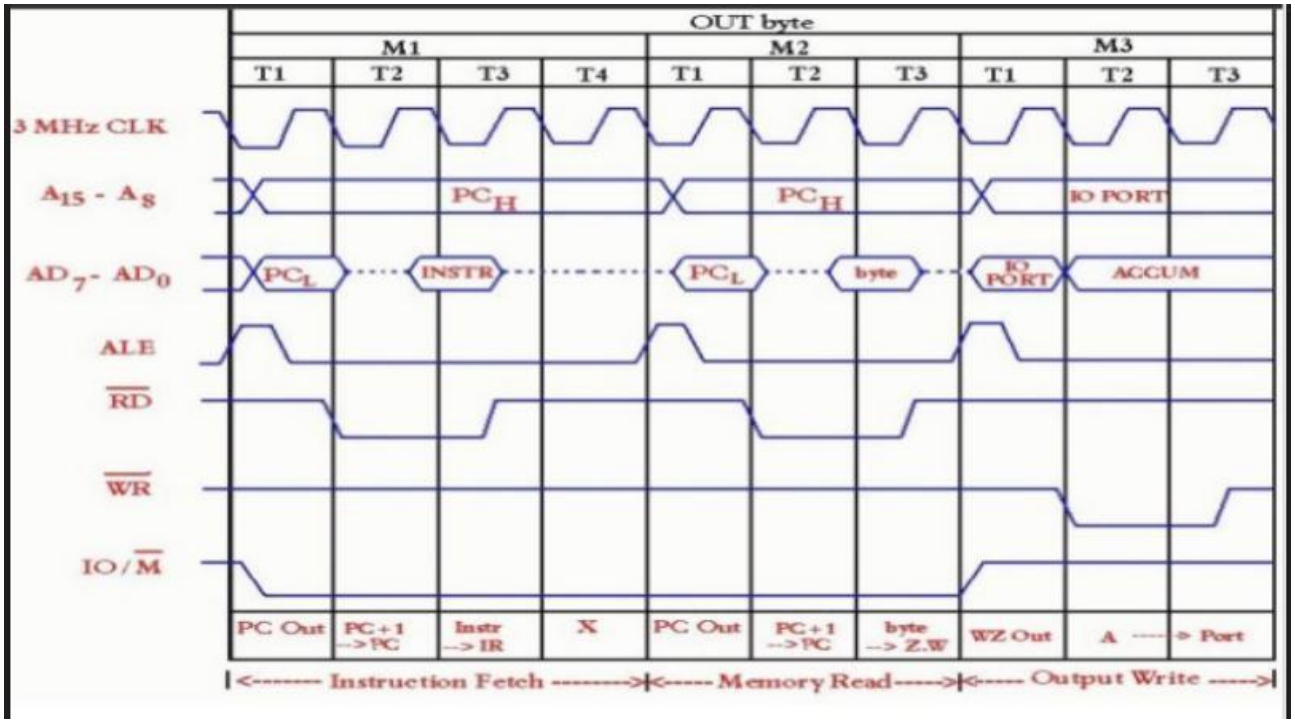
C0H
Address: 4126H

Content of the port
C0H: 5EH

Eg:



6. OUT



Memory Interfacing and Generation of Chip Select Signal

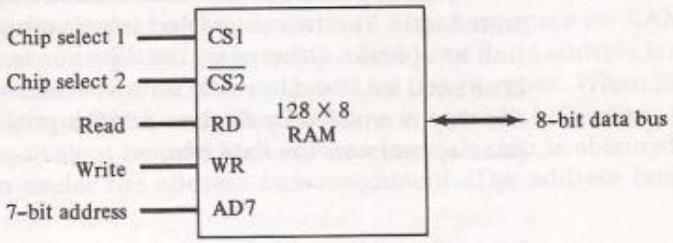
RAM and ROM Chips

bidirectional bus

A RAM chip is better suited for communication with the CPU if it has one or more control inputs that select the chip only when needed. Another common feature is a bidirectional data bus that allows the transfer of data either from memory to CPU during a read operation, or from CPU to memory during a write operation. A bidirectional bus can be constructed with three-state buffers. A three-state buffer output can be placed in one of three possible states: a signal equivalent to logic 1, a signal equivalent to logic 0, or a high-impedance state. The logic 1 and 0 are normal digital signals. The high-impedance state behaves like an open circuit, which means that the output does not carry a signal and has no logic significance.

The block diagram of a RAM chip is shown in Fig. 12-2. The capacity of the memory is 128 words of eight bits (one byte) per word. This requires a 7-bit

Figure 12-2 Typical RAM chip.



(a) Block diagram

CS1	$\overline{CS2}$	RD	WR	Memory function	State of data bus
0	0	×	×	Inhibit	High-impedance
0	1	×	×	Inhibit	High-impedance
1	0	0	0	Inhibit	High-impedance
1	0	0	1	Write	Input data to RAM
1	0	1	×	Read	Output data from RAM
1	1	×	×	Inhibit	High-impedance

(b) Function table

address and an 8-bit bidirectional data bus. The read and write inputs specify the memory operation and the two chips select (CS) control inputs are for enabling the chip only when it is selected by the microprocessor. The availability of more than one control input to select the chip facilitates the decoding of the address lines when multiple chips are used in the microcomputer. The read and write inputs are sometimes combined into one line labeled R/W. When the chip is selected, the two binary states in this line specify the two operations of read or write.

The function table listed in Fig. 12-2(b) specifies the operation of the RAM chip. The unit is in operation only when CS1 = 1 and $\overline{CS2} = 0$. The bar on top of the second select variable indicates that this input is enabled when it is equal to 0. If the chip select inputs are not enabled, or if they are enabled but the read or write inputs are not enabled, the memory is inhibited and its data bus is in a high-impedance state. When CS1 = 1 and $\overline{CS2} = 0$, the memory can be placed in a write or read mode. When the WR input is enabled, the memory stores a byte from the data bus into a location specified by the address input lines. When the RD input is enabled, the content of the selected byte is placed into the data bus. The RD and WR signals control the memory operation as well as the bus buffers associated with the bidirectional data bus.

A ROM chip is organized externally in a similar manner. However, since a ROM can only read, the data bus can only be in an output mode. The block diagram of a ROM chip is shown in Fig. 12-3. For the same-size chip, it is possible to have more bits of ROM than of RAM, because the internal binary cells in ROM occupy less space than in RAM. For this reason, the diagram specifies a 512-byte ROM, while the RAM has only 128 bytes.

The nine address lines in the ROM chip specify any one of the 512 bytes stored in it. The two chip select inputs must be CS1 = 1 and $\overline{CS2} = 0$ for the unit to operate. Otherwise, the data bus is in a high-impedance state. There is no need for a read or write control because the unit can only read. Thus when the chip is enabled by the two select inputs, the byte selected by the address lines appears on the data bus.

Memory Address Map

The designer of a computer system must calculate the amount of memory required for the particular application and assign it to either RAM or ROM. The interconnection between memory and processor is then established from knowledge of the size of memory needed and the type of RAM and ROM chips available. The addressing of memory can be established by means of a table that specifies the memory address assigned to each chip. The table, called a *memory address map*, is a pictorial representation of assigned address space for each chip in the system.

To demonstrate with a particular example, assume that a computer system needs 512 bytes of RAM and 512 bytes of ROM. The RAM and ROM chips

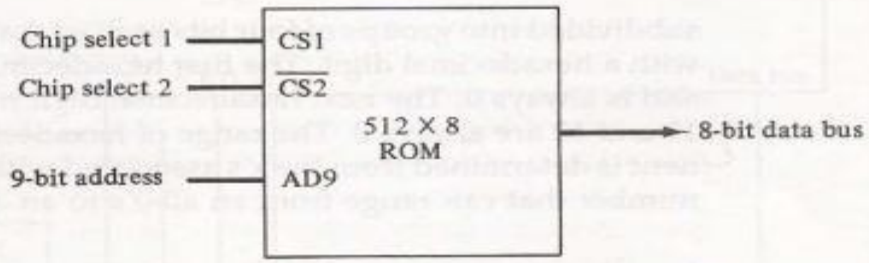


Figure 12-3 Typical ROM chip.

to be used are specified in Figs. 12-2 and 12-3. The memory address map for this configuration is shown in Table 12-1. The component column specifies whether a RAM or a ROM chip is used. The hexadecimal address column assigns a range of hexadecimal equivalent addresses for each chip. The address bus lines are listed in the third column. Although there are 16 lines in the address bus, the table shows only 10 lines because the other 6 are not used in this example and are assumed to be zero. The small x's under the address bus lines designate those lines that must be connected to the address inputs in each chip. The RAM chips have 128 bytes and need seven address lines. The ROM chip has 512 bytes and needs 9 address lines. The x's are always assigned to the low-order bus lines: lines 1 through 7 for the RAM and lines 1 through 9 for the ROM. It is now necessary to distinguish between four RAM chips by assigning to each a different address. For this particular example we choose bus lines 8 and 9 to represent four distinct binary combinations. Note that any other pair of unused bus lines can be chosen for this purpose. The table clearly shows that the nine low-order bus lines constitute a memory space for RAM equal to $2^9 = 512$ bytes. The distinction between a RAM and ROM address is done with another bus line. Here we choose line 10 for this purpose. When line 10 is 0, the CPU selects a RAM, and when this line is equal to 1, it selects the ROM.

The equivalent hexadecimal address for each chip is obtained from the information under the address bus assignment. The address bus lines are

TABLE 12-1 Memory Address Map for Microprocomputer

Component	Hexadecimal address	Address bus									
		10	9	8	7	6	5	4	3	2	1
RAM 1	0000–007F	0	0	0	x	x	x	x	x	x	x
RAM 2	0080–00FF	0	0	1	x	x	x	x	x	x	x
RAM 3	0100–017F	0	1	0	x	x	x	x	x	x	x
RAM 4	0180–01FF	0	1	1	x	x	x	x	x	x	x
ROM	0200–03FF	1	x	x	x	x	x	x	x	x	x

subdivided into groups of four bits each so that each group can be represented with a hexadecimal digit. The first hexadecimal digit represents lines 13 to 16 and is always 0. The next hexadecimal digit represents lines 9 to 12, but lines 11 and 12 are always 0. The range of hexadecimal addresses for each component is determined from the x's associated with it. These x's represent a binary number that can range from an all-0's to an all-1's value.

Memory Connection to CPU

RAM and ROM chips are connected to a CPU through the data and address buses. The low-order lines in the address bus select the byte within the chips and other lines in the address bus select a particular chip through its chip select inputs. The connection of memory chips to the CPU is shown in Fig. 12-4. This configuration gives a memory capacity of 512 bytes of RAM and 512 bytes of ROM. It implements the memory map of Table 12-1. Each RAM receives the seven low-order bits of the address bus to select one of 128 possible bytes. The particular RAM chip selected is determined from lines 8 and 9 in the address bus. This is done through a 2×4 decoder whose outputs go to the CS1 inputs in each RAM chip. Thus, when address lines 8 and 9 are equal to 00, the first RAM chip is selected. When 01, the second RAM chip is selected, and so on. The RD and WR outputs from the microprocessor are applied to the inputs of each RAM chip.

The selection between RAM and ROM is achieved through bus line 10. The RAMs are selected when the bit in this line is 0, and the ROM when the bit is 1. The other chip select input in the ROM is connected to the RD control line for the ROM chip to be enabled only during a read operation. Address bus lines 1 to 9 are applied to the input address of ROM without going through the decoder. This assigns addresses 0 to 511 to RAM and 512 to 1023 to ROM. The data bus of the ROM has only an output capability, whereas the data bus connected to the RAMs can transfer information in both directions.

The example just shown gives an indication of the interconnection complexity that can exist between memory chips and the CPU. The more chips that are connected, the more external decoders are required for selection among the chips. The designer must establish a memory map that assigns addresses to the various chips from which the required connections are determined.

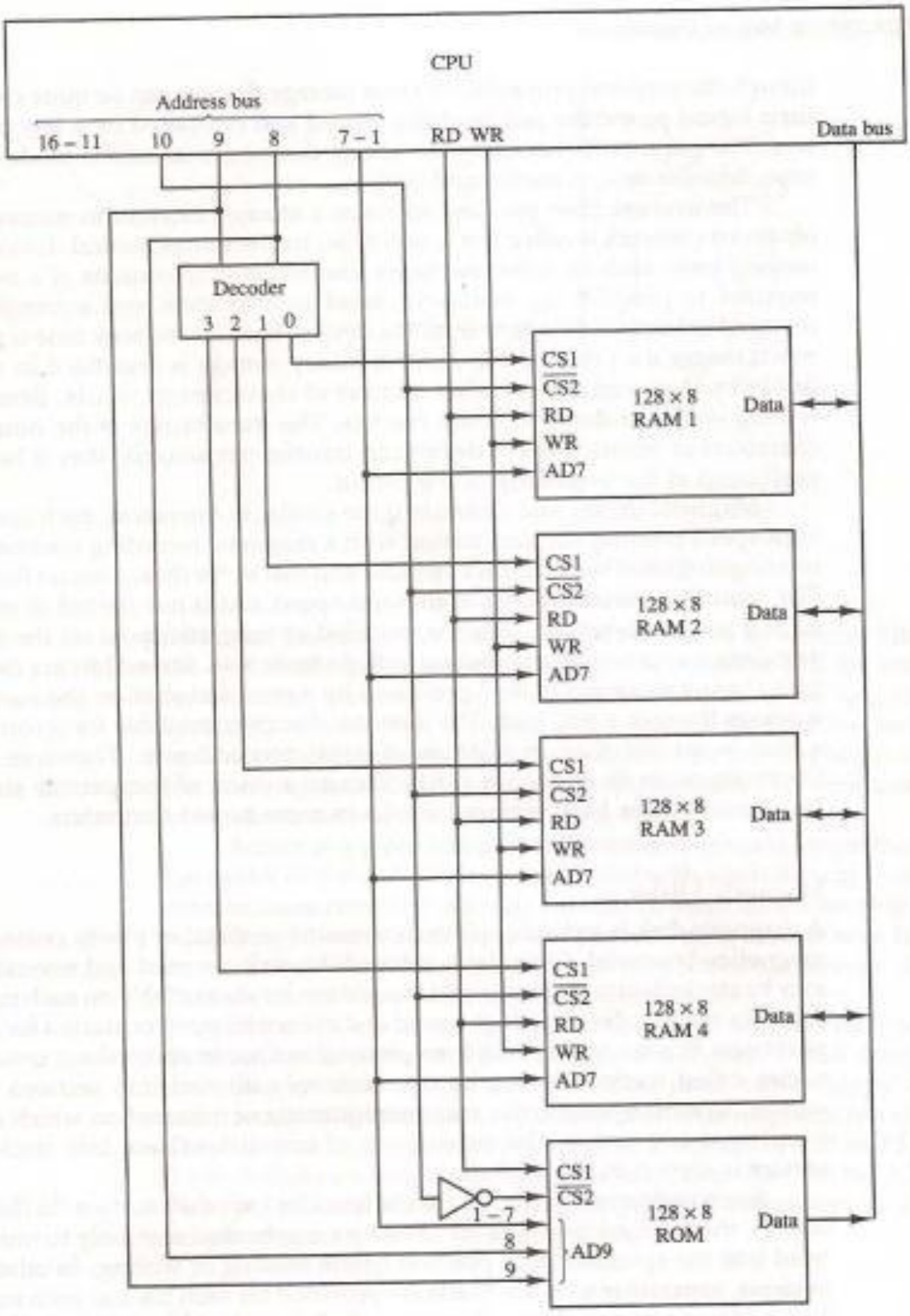


Figure 12-4 Memory connection to the CPU.

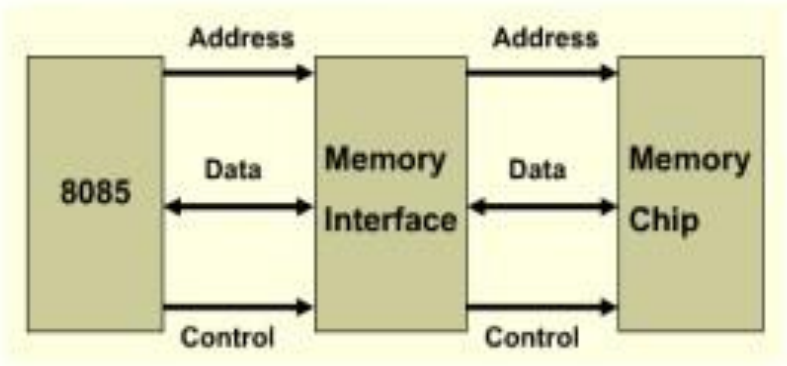


Fig: 8085 interfacing with memory Chips

The interface process involves designing a circuit that will match the memory requirements with the microprocessor signal.

Memory has certain signal requirements to read from and write into memory. Similarly Microprocessor initiates the set of signals when it wants to read from and write into memory.

- 8085 has 16 address lines (A0 - A15), hence a maximum of 64 KB (= 216 bytes) of memory locations can be interfaced with it.
- The memory address space of the 8085 takes values from 0000H to FFFFH.
- The 8085 initiates set of signals such as IO/M, RD' and WR' when it wants to read from and write into memory.
- Similarly, each memory chip has signals such as CE or CS (chip enable or chip select), OE or RD' (output enable or read) and WE or WR' (write enable or write) associated with it.

Generation of Control Signals for Memory: When the 8085 wants to read from and write into memory, it activates IO/M, RD and WR signals as shown. Status of IO/M, RD' and WR' signals during memory read and write operations

IO/M'	RD'	WR'	Operation
0	0	1	8085 reads data from memory
0	1	0	8085 writes data into memory

Using IO/M , RD and WR signals, two control signals MEMR (memory read) and MEMW (memory write) are generated. Fig. 16 shows the circuit used to generate these signals.

2 Example for Memory Interfacing

Consider a system in which the full memory space 64kb is utilized for EPROM memory. Interface the EPROM with 8085 processor. The memory capacity is 64 Kbytes. i.e 2^n = 64 x 1000 bytes where n = address lines. So, n = 16.

In this system the entire 16 address lines of the processor are connected to address input pins of memory IC in order to address the internal locations of memory. The chip select (CS) pin of EPROM is permanently tied to logic low (i.e., tied to ground).

Since the processor is connected to EPROM, the active low RD pin is connected to active low output enable pin of EPROM. The range of address for EPROM is 0000H to FFFFH.

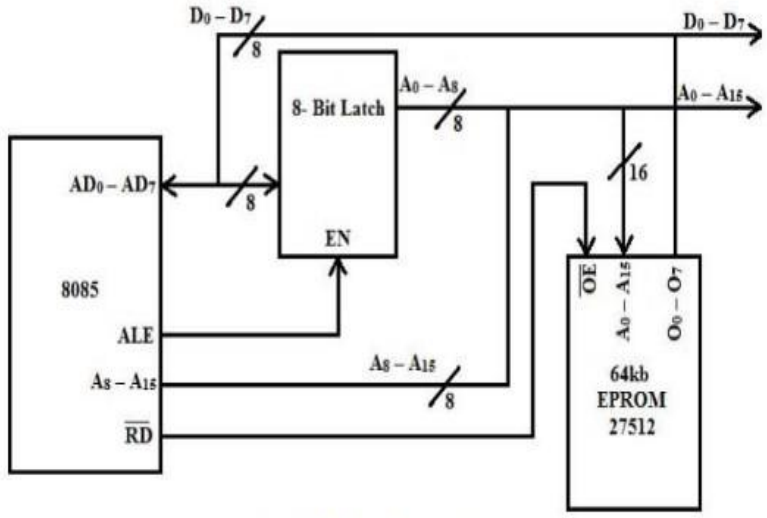


Fig 1.6 Memory Interfacing

CHAPTER 4: ASSEMBLY LANGUAGE PROGRAMMING

Programming with Intel 8085 Microprocessor

INSTRUCTION DESCRIPTION AND FORMAT:

The computer can be used to perform a specific task, only by specifying the necessary steps to complete the task. The collection of such ordered steps forms a 'program' of a computer. These ordered steps are the instructions. Computer instructions are stored in central memory locations and are executed sequentially one at a time.

Each instruction of a program is pointed by the program counter (PC). It is first moved to the instruction register (IR), then decoded into binary format and stored as an instruction in the memory. The computer takes a certain

period to complete this task i.e., instruction fetching, decoding and executing on the basis of clock speed. Such a time period is called 'Instruction cycle' and consists two cycles namely fetch & decode and Execute cycle.

Generally each instruction has two parts: one is the task to be performed, called the operation code (Op-Code) field, and the second is the data to be operated on, called the operand or address field. The operand (or data) can be specified in various ways. It may include 8-bit (or 16-bit) data, an internal register, a memory location, or an 8-bit (or 16-bit) address. The Op-Code field specifies how data is to be manipulated and address field indicates the address of a data item. Though computer only understands binary language, it is complex and tedious for us to enter all the instructions in binary format. Besides, identification and correction of errors in the program is even harder. So, the instructions are written in meaningful words (relating to task they perform) as mnemonics so that it is easier for programmers to learn, understand and use easily. For example:

ADD R1, R0

Op-code address: Here R0 is the source register and R1 is the destination register.

The instruction adds the contents of R0 with the content of R1 and stores result in R1.

8085 A can handle at the maximum of 256 instructions (28) (246 instructions are used). Depending on the number of address specified in instruction sheet, the instruction format can be classified into three categories:

- **One Byte Instruction:** Here 1 byte will be Op-code and operand will be default. E.g. ADD B, MOV A,B
- **Two-Byte Instruction:** Here first byte will be Op-code and second byte will be the operand/data. E.g. IN 40H, MVI A 8-bit Data
- **Three-Byte Instruction:** Here first byte will be Op-code, second and third byte will be operands/data. That is 2nd byte- lower order data.
  3rd byte – higher order data.
  E.g. LXI B, 4050 H

THE 8085 INSTRUCTION SETS

An instruction is a binary pattern designed inside a microprocessor to perform a specific function (task). The entire group of instructions called the instruction set. The 8085 instruction set can be classified into 5- different groups:

- Data Transfer Instructions
- Arithmetic Instructions
- Branching Instructions
- Logical Instructions
- Control Instructions

DATA TRANSFER INSTRUCTIONS

It is the longest group of instructions in 8085. This group of instruction copy data from a source location to destination location without modifying the contents of the source. The transfer of data may be between the registers or between register and memory or between an I/O device and accumulator.
None of these instructions changes the flag. The instructions of this group are:

1. MOV Rd, Rs Rd←Rs (move register instruction)

- 1 byte instruction
- Copies data from source register to destination register.
- Rd & Rs may be A, B, C, D, E, H & L
- E.g. MOV A, B

2. MVI R, 8 bit data (move immediate instruction)

- 2 byte instruction
- Loads the second byte (8 bit immediate data) into the register specified.
- R may be A, B, C, D, E, H & L
- E.g. MVI C, 53H C←53H

3. MOV M, R (Move to memory from register)

- Copy the contents of the specified register to memory. Here memory is the location specified by contents of the HL register pair.
- E.g. MOV M, B

4. MOV R, M (move to register from memory)

- Copy the contents of memory location specified by HL pair to specified register.

- E. g. MOV B, M

5. LXI R_P, 2 bytes data (load register pair)

- 3-byte instruction
- Load immediate data to register pair
- Register pair may be BC, DE, HL & SP(Stack pointer)
- 1^{st} byte - Op-code
- 2^{nd} byte - lower order data
- 3^{rd} byte- higher order data

-

  E.g. L X I B, 4532H; B←45, C←32H

6. MVI M, immediate data (load memory immediate)

- 2 byte instruction.
- Loads the 8-bit data to the memory location whose address is specified by the contents of HL pair.
- E.g. MVI M , 35H; [HL]←35H

#Write a program to load memory locations 7090 H and 7080 H with data 40H and 50H and then swap these data.

Soln:

```
MVI H, 70H
MVI L, 90H
MVI A, 40H
MOV M, A
MOV C, M
MVI L, 80H
MVI B, 50H
MOV M, B
MOV D, M
MOV M, C
MVI L, 90H
MOV M, D
HLT
```

7. LDA 16-bit address (Load accumulator direct)

- 3-byte instruction
- Loads the accumulator with the contents of memory location whose address is specified by 16 bit address.
- E.g. LDA 4035H (implies A ← [4035H])

8. LDAX R_P (Load accumulator indirect)

- 1 byte instruction
- Loads the contents of memory location pointed by the contents of register pair to accumulator.
- E.g. LDAX B [A] ← [[BC]]
- LXI B, 9000H (implies B= 90, C= 00)
- LDAX B (implies A ← [9000])

9. STA 16-bit address (store accumulator contents direct)

- 3-byte instruction.
- Stores the contents of accumulator to specified address
- E.g. STA FA00H (implies [FA00]←[A])

10. STAX RP (implies [RP]←A)

- Stores the contents of accumulator to memory location specified by the contents of register pair.
- 1 byte instruction
- E.g. STAX B

Input	Output
LXI B, 9500H	B←95, C←00
LXI D, 9501H	D←95, E←01
MVI A, 32H	A←32H
STAX B	[BC]←A
MVI A, 7AH	A←7A
STAX D	[DE]←A

11. IN 8-bit address

- 2-byte instruction
- Read data from the input port address specified in the second byte and loads data into the accumulator i.e. input port content to accumulator: A←P
- E.g. IN 40H (implies A←[40H])

12. OUT 8-bit address

- 2-byte instruction
- Copies the contents of the accumulator to the output port address in the 2nd byte. That means accumulator to output port: P ←A
- E.g. OUT 40H (implies [40H]←A)

13. LHLD 16-bit address (Load HL directly)

- 3-byte instruction.
- Loads the contents of specified memory location to L-register and contents of next higher location to H-register.
- E.g.

-

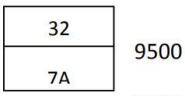
E.g. LXI H, 9500H

MVI M, 32H

MVI L, 01H

MVI m, 7AH

LHLD 9500H H=7A, L=32



14. SHLD 16-bit address (store HL directly)

- Opposite to LHLD.
- Stores the contents of L-register to specified memory location and contents of H-register to next higher memory location.
- E.g. LXI H, 9500H
- SHLD 8500H (implies [8501]←95 and [8500]←00)

15. XCHG (Exchange)

- Exchanges DE pair with HL pair.
  E. g. LXI H, 7500H (H= 75, L=00)
- LXI D, 9532H (D=95, E=32)
- XCHG (H=95, L=32 & D=75 E=00)

16. SPHL

- The instruction loads the contents of the H and L registers into the stack pointer register, the contents of the H register provide the high-order address and the contents of the L register provide the low-order address. The contents of the H and L registers are not altered.
- Example: SPHL

17. XTHL

- The contents of the L register are exchanged with the stack location pointed out by the contents of the stack pointer register. The contents of the H register are exchanged with the next stack location (SP+1); however, the contents of the stack pointer register are not altered.
- Example: XTHL

18. PUSH Rp

- The contents of the register pair designated in the operand are copied onto the stack in the following sequence. The stack pointer register is decremented and the contents of the high-order register (B, D, H, A) are copied into that location. The stack pointer register is decremented again

and the contents of the low-order register (C, E, L, flags) are copied to that location.

- Example: PUSH B or PUSH A

19. POP Rp

- The contents of the memory location pointed out by the stack pointer register are copied to the low-order register (C, E, L, status flags) of the operand. The stack pointer is incremented by 1 and the contents of that memory location are copied to the high-order register (B, D, H, A) of the operand. The stack pointer register is again incremented by 1. Example: POP H or POP A
-

ARITHMETIC INSTRUCTIONS:

The 8085 microprocessor performs various arithmetic operations such as addition, subtraction, increment and decrement. These arithmetic operations have the following mnemonics.

1. ADD R/M

- 1 byte add instruction.
- Adds the contents of register/memory to the contents of the accumulator and stores the result in accumulator. E. g. Add B (implies A←[A] + [B])

2. ADI 8 bit data

- 2 byte add immediate instruction.

- Adds the 8 bit data with the contents of accumulator and stores result in accumulator. E.g. ADI 9BH ;A←A+9BH

3. SUB R/M

- 1 byte subtract instruction.
- Subtracts the contents of specified register/M with the contents of accumulator and stores the result in accumulator.
- E. g. SUB D ; A ←(A-D)

4. SUI 8 bit data

- 2 byte subtract immediate instruction.
- Subtracts the 8 bit data from the contents of accumulator stores result in accumulator. E. g. SUI D3H ;A ←A-D3H

5. INR R/M, DCR R/M

- 1 byte increment and decrement instructions.
- Increase and decrease the contents of R(register) or M(memory) by 1 respectively
- E. g. DCR B ; B=B-1
  DCR M ; [HL] = [HL]-1
  INR A ; A=A+1
  INR M ; [HL] +1

For these, all flags are affected except carry.

6. INX Rp, DCX RP

- Increase and decrease the register pair by 1.
- Acts as 16 bit counter made from the contents of 2 registers (1 byte instruction)
- E.g. INX B ;BC=BC+1 DCX D ;DE=DE+1
- No flags affected

7. ADC R/M and ACI 8-bit data (addition with carry (1 byte))

- ACI 8-bit data= immediate (2 byte).
- Adds the contents of register or 8 bit data whatever used suitably with the previous carry.
- E.g. ADC B ;A← (A+B+CY) ACI 70H ;A ←(A + 70+CY)

8. SBB B/M

- 1 byte instruction.

- Subtracts the contents of register or memory from the contents of accumulator and stores the result in accumulator.
- e. g. SBB D ; A ←(A-D-Borrow)

9. SBI 8 bit data

- 2 byte instruction.
- Subtracts the 8-bit immediate data from the content of the accumulator and stores the result in accumulator.
  E.g. SBI 70H ;A ← (A-70-Borrow)

10. DAD Rp (double addition)

- 1 byte instruction.
- Adds register pair with HL pair and store the 16 bit result in HL pair.
- E. g. DAD B ; HL=HL+BC

11. DAA (Decimal adjustment accumulator)

- Used only after addition.
- 1 byte instruction.
- The content of accumulator is changed from binary to two 4-bit BCD digits.
- E. g MVI A, 78H ;A=78 MVI B, 42H ;B=42
  ADD B ;A=A+B = BA
  DAA ;A=20, CY=1

The arithmetic operation add and subtract are performed in relation to the contents of accumulator. The features of these instructions are

- They assume implicitly that the accumulator is one of the operands.
- They modify all the flags according to the data conditions of the result. They place the result in the accumulator.
- They do not affect the contents of operand register or memory. But the INR and DCR operations can be performed in any register or memory. These instructions
  - ✓ Affect the contents of specified register or memory.
  - ✓ Affect the flag except carry flag.

BRANCHING INSTRUCTIONS

▪

The microprocessor is a sequential machine; it executes machine codes from one memory location to the next. The branching instructions instruct the microprocessor to go to a different memory location and the microprocessor continues executing machine codes from that new location.

1. Jump unconditionally:

JMP 16-bit address

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. JMP 16-bit address Example: JMP 2034H or JMP XYZ

2. Jump conditionally

The conditional jump instructions allow the microprocessor to make decisions based on certain conditions indicated by the flags. After logic and arithmetic operations, flags are set or reset to reflect the condition of data. These instructions check the flag conditions and make decisions to change or not to change the sequence of program. The four flags namely carry, zero, sign and parity are used by the jump instruction.

The program sequence is transferred to the memory location specified by the 16-bit address given in the **16-bit operand** based on the specified flag of the PSW as described below. Example: JZ 2034H or JZ XYZ

Opcode	Description	Flag
JC	Jump on Carry	Status CY = 1
JNC	Jump on no Carry	CY = 0
JP	Jump on positive	S = 0
JM	Jump on minus	S = 1
JZ	Jump on zero	Z = 1
JNZ	Jump on no zero	Z = 0
JPE	Jump on parity even	P = 1
JPO	Jump on parity odd	P = 0

3. Unconditional subroutine call

CALL 16 bit address

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand. Before the transfer, the address of the next instruction after CALL (the contents of the program counter) is pushed onto the stack. Example: CALL 2034H or CALL XYZ

4. Call conditionally

The program sequence is transferred to the memory location specified by the 16-bit address given in the operand based on the specified flag of the PSW as described below. Before the transfer, the address of the next instruction after the call (the contents of the program counter) is pushed onto the stack.

Example: CZ 2034H or CZ XYZ

Opcode	Description	Flag Status
CC	Call on Carry	CY = 1
CNC	Call on no Carry	CY = 0
CP	Call on positive	S = 0

CM	Call on minus	S = 1
CZ	Call on zero	Z = 1
CNZ	Call on no zero	Z = 0
CPE	Call on parity even	P = 1
CPO	Call on parity odd	P = 0

5. Return from subroutine unconditionally

The program sequence is transferred from the subroutine to the calling program. The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address. Example: RET

6. Return from subroutine conditionally

The program sequence is transferred from the subroutine to the calling program based on the specified flag of the PSW as described below. The two bytes from the top of the stack are copied into the program counter, and program execution begins at the new address. Example: RZ

Opcode	Description	Flag Status
RC	Return on Carry	CY = 1
RNC	Return on no Carry	CY = 0
RP	Return on positive	S = 0
RM	Return on minus	S = 1
RZ	Return on zero	Z = 1
RNZ	Return on no zero	Z = 0
RPE	Return on parity even	P = 1
RPO	Return on parity odd	P = 0

7. PCHL (Load program counter with HL contents)

The contents of registers H & L are copied into the program counter. The contents of H are placed as the high-order byte and the contents of L as the low-order byte. Example: PCHL

8. RST (Restart)

The RST instruction is used as software instructions in a program to transfer the program execution to one of the following eight locations.

Instruction	Restart Address
RST 0	0000H
RST 1	0008H
RST 2	0010H
RST 3	0018H
RST 4	0020H
RST 5	0028H
RST 6	0030H
RST 7	0038H

The 8085 has additionally 4 interrupts, which can generate RST instructions internally and doesn't require any external hardware. Following are those instructions and their Restart addresses:

Interrupt	Restart Address
TRAP	0024H

RST 5.5	002CH
RST 6.5	0034H
RST 7.5	003CH

LOGICAL INSTRUCTIONS

A microprocessor is basically a programmable logic chip. It can perform all the logic functions of the hardwired logic through its instruction set. The 8085 instruction set includes such logic functions as AND, OR, XOR and NOT (Complement).

1. ANA R/M (the contents of register/memory)

- Logically AND the contents of register/memory with the contents of accumulator.
- 1 byte instruction.
- CY flag is reset and AC is set.

2. ANI 8 bit data

- Logically AND 8 bit immediate data with the contents of accumulator.
- 2 byte instruction.
- CY flag is reset and AC is set. Others as per result.

3. ORA R/M

- Logically OR the contents of register/memory with the contents of accumulator.
- 1 byte instruction.
- CY and AC is reset and other as per result.

4. ORI 8 bit data

- Logically OR 8 bit immediate data with the contents of the accumulator.
- 2 byte instruction.
- CY and AC is reset and other as per result.

5. XRA R/M

- Logically exclusive OR the contents of register memory with the contents of accumulator.
- 1 byte instruction.
- CY and AC is reset and other as per result.

6. XRI 8 bit data

- Logically Exclusive OR 8 bit data immediate with the content of accumulator.
- 2 byte instruction.
- CY and AC is reset and other as per result.

7. CMA (Complement accumulator)

- One-byte instruction.
- Complements the contents of the accumulator.
- No flags are affected. Example:

Instruction	CY	AC
ANA/ANI	0	1
ORA/ORI	0	0
XRA/XRI	0	0

8. CMC (Complement Carry)

- The Carry flag is complemented. No other flags are affected. Example: CMC

9. CPI 8-bit data

- The second byte (8-bit data) is compared with the contents of the accumulator. The values being compared remain unchanged. The result of the comparison is shown by setting the flags of the PSW as follows:
- if (A) < data: carry flag is set if (A) = data ➔ zero flag is set
- if (A) > data: carry and zero flags are reset
- Example: CPI 89H

10. RLC (Rotate accumulator left)

- Each binary bit of the accumulator is rotated left by one position. Bit D_7 is placed in the position of D_0 as well as in the Carry flag. CY is modified according to bit D7. S, Z, P, AC are not affected. Example: RLC

11. RRC (Rotate accumulator right)

- Each binary bit of the accumulator is rotated right by one position. Bit D_0 is placed in the position of D_7 as well as in the Carry flag. CY is modified according to bit D_0. S, Z, P, AC are not affected.
- Example: RRC

12. RAL (Rotate accumulator left through carry)

- Each binary bit of the accumulator is rotated left by one position through the Carry flag. Bit D_7 is placed in the Carry flag, and the Carry flag is placed in the

least significant position D_0. CY is modified according to bit D_0. S, Z, P, AC are not affected. Example: RAL

13. RAR (Rotate accumulator right through carry)

- Each binary bit of the accumulator is rotated right by one position through the Carry flag. Bit D_0 is placed in the Carry flag, and the Carry flag is placed in the most significant position D_0. CY is modified according to bit D_0. S, Z, P, AC are not affected. Example: RAR

14. STC (Set Carry)

- The Carry flag is set to 1. No other flags are affected. Example: STC

CONTROL INSTRUCTIONS

Opcode	Operand	Meaning	Explanation
NOP	None	No operation	No operation is performed, i.e., the instruction is fetched and decoded.
HLT	None	Halt and enter wait state	The CPU finishes executing the current instruction and stops further execution. An interrupt or reset is necessary to exit from the halt state.
DI	None	Disable interrupts	The interrupt enable flip-flop is reset and all the interrupts are disabled except TRAP.
EI	None	Enable interrupts	The interrupt enable flip-flop is set and all the interrupts are enabled.
RIM	None	Read interrupt mask	This instruction is used to read the status of interrupts 7.5, 6.5, 5.5 and read serial data input bit.
SIM	None	Set interrupt mask	This instruction is used to implement the interrupts 7.5, 6.5, 5.5, and serial data output.

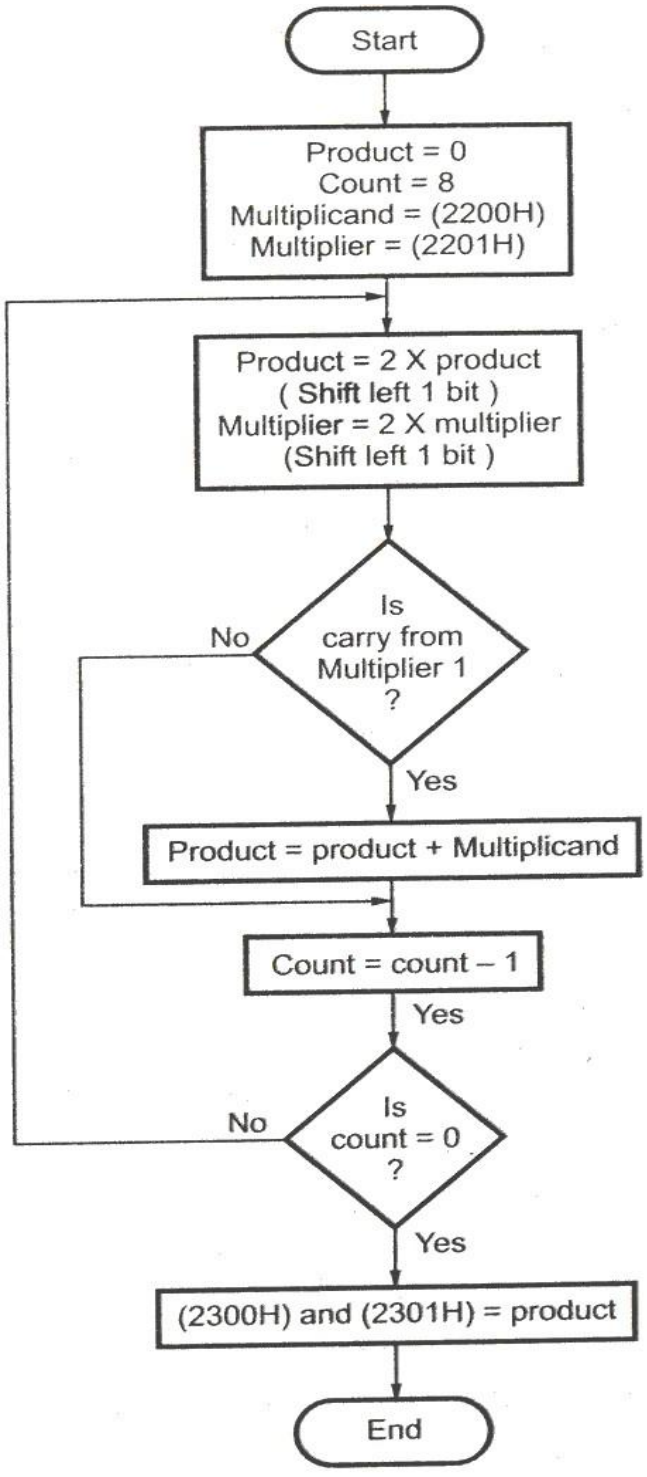
MULTIPLICATION OF TWO 8 BIT NUMBERS

Statement: Multiply the 8-bit unsigned number in memory location 2200H by the 8-bit unsigned number in memory location 2201H. Store the 8 least

significant bits of the result in memory location 2300H and the 8 most significant bits in memory location 2301H.

PROGRAM:

LXI H, 2200 : Initialize the memory pointer
MOV E, M : Get multiplicand
MVI D, 00H : Extend to 16-bits
INX H : Increment memory pointer
MOV A, M : Get multiplier
LXI H, 0000 : Product = 0
MVI B, 08H : Initialize counter with count 8
MULT: DAD H : Product = product x 2
RAL
JNC SKIP : Is carry from multiplier 1?
DAD D : Yes, Product =Product + Multiplicand
SKIP: DCR B : Is counter = zero
JNZ MULT : no, repeat
SHLD 2300H :Storethe result
HLT : End of program



DIVISION OF TWO 8 BIT NUMBERS

AIM: To perform the division of two 8 bit numbers using 8085.

ALGORITHM:

1. Start the program by

loading HL register pair with address of memory location.

2. Move the data to a register (B register).
3. Get the second data and load into Accumulator.
4. Compare the two numbers to check for carry.
5. Subtract the two numbers.
6. Increment the value of carry.
7. Check whether repeated subtraction is over and store the value of product and carry in memory location.
8. Terminate the program.

PROGRAM:

```
        LXI  H, 4150
        MOV  B,M     Get the dividend in B – reg.
        MVI  C,00        Clear C – reg for qoutient
INX  H
        MOV A,M      Get the divisor in A – reg. NEXT:
CMP B          Compare A - reg with register B.
JC  LOOP       Jump on carry to LOOP            SUB  B
Subtract A – reg from B- reg.
        INR C          Increment content of register C.
        JMP NEXT     Jump to NEXT
LOOP:   STA 4152       Store the remainder in Memory
        MOV A,C
        STA 4153           Store the quotient in memory
HLT             Terminate the program.
```

OBSERVATION:

Input:

oF (4150)

FF (4251)

Output:

01 (4152) ---- Remainder

FE (4153) ---- Quotient

8085 program to add numbers in an array

STATEMENT: Suppose the size of the array is stored at memory location 2050 and the base address of the array is 2051. The sum will be stored at memory location 3050 and carry will be stored at location 3051.

ALGORITHM:

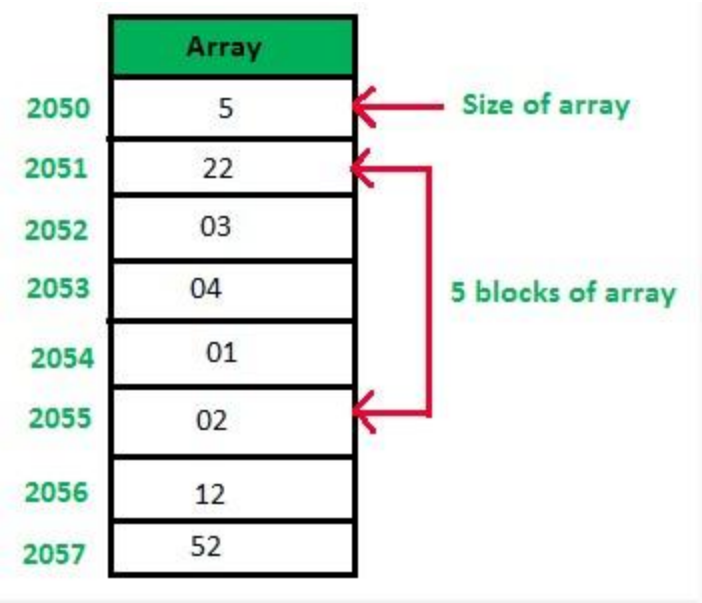
1. Load the base address of the array in HL register pair.
2. Use the size of the array as a counter.
3. Initialize accumulator to 00.
4. Add content of accumulator with the content stored at memory location given in HL pair.
5. Decrease counter on each addition.

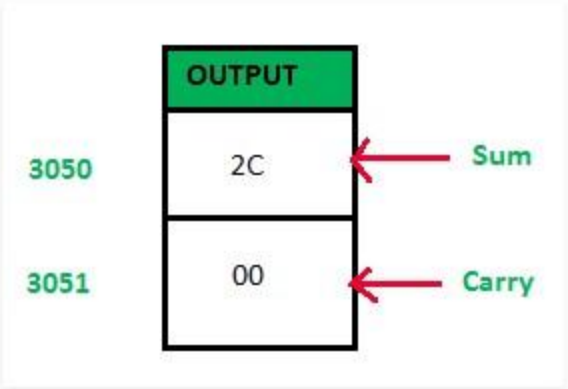
PROGRAM:

MNEMONICS	COMMENTS
LDA 2050	[A] ←[2050]
MOV B, A	[B] ← [A]
LXI H, 2051	[H] ← 20 and [L] ← 51
MVI A, 00	[A] ← 00
MVI C, 00	[C] ← 00
MNEMONICS	**COMMENTS**
ADD M	[A] ← [A]+[M]
INR M	[M] ← [M]+1
JNC 2011	
INR C	[C] ← [C]+1
DCR B	[B] ← [B]-1
JNZ 200B	
STA 3050	[3050] ← [A]
MOV A, C	[A] ←[C]

STA 3051	[3051] ← [A]
HLT	Terminates the program

OBSERVATION:





8085 program for bubble sort

STATEMENT: Write an assembly language program in 8085 microprocessor to sort a given list of n numbers using Bubble Sort. Size of list is stored at 2040H and list of numbers from 2041H onwards.

ALGORITHM:

1. Load size of list in C register and set D register to be 0
2. Decrement C as for n elements n-1 comparisons occur
3. Load the starting element of the list in Accumulator
4. Compare Accumulator and next element
5. If accumulator is less than next element jump to step 8
6. Swap the two elements
7. Set D register to 1
8. Decrement C
9. If C>0 take next element in Accumulator and go to point 4

10. If D=0, this means in the iteration, no exchange takes place consequently we know that it won't take place in further iterations so the loop in exited and program is stopped
11. Jump to step 1 for further iteration PROGRAM:

LABEL	INSTRUCTION	COMMENT
START:	LXI H, 2040H	Load size of array
	MVI D, 00H	Clear D register to set up a flag
	MOV C, M	Set C register with number of elements in list
	DCR C	Decrement C
	INX H	Increment memory to access list
CHECK:	MOV A, M	Retrieve list element in Accumulator
	INX H	Increment memory to access next element
	CMP M	Compare Accumulator with next element
	JC NEXTBYTE	If accumulator is less then jump to NEXTBYTE
	MOV B, M	Swap the two elements
	MOV M, A	
	DCX H	
	MOV M, B	
	INX H	
	MVI D, 01H	If exchange occurs save 01 in D register
NEXTBYTE:	DCR C	Decrement C for next iteration
	JNZ CHECK	Jump to CHECK if C>0
	MOV A, D	Transfer contents of D to Accumulator

	CPI 01H	Compare accumulator contents with 01H
	JZ START	Jump to START if D=01H
	HLT	HALT

OBSERVATION:

8085 program to convert a BCD number to binary

STATEMENT: Write an assembly language program for converting a 2 digit BCD number to its binary equivalent using 8085 microprocessor.

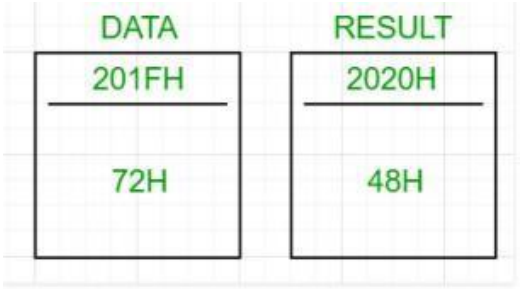
ALGORITHM:

1. Load the BCD number in the accumulator
2. Unpack the 2 digit BCD number into two separate digits. Let the left digit be BCD1 and the right one BCD2
3. Multiply BCD1 by 10 and add BCD2 to it
4. If the 2 digit BCD number is 72, then its binary equivalent will be 7 x OAH + 2 = 46H + 2 = 48H PROGRAM:

LABEL	MNEMONIC
	LDA 201FH
	MOV B, A
	ANI 0FH
	MOV C, A
	MOV A, B
	ANI F0H

	JZ SKIPMULTIPLY
	RRC
	RRC
	RRC
	RRC
	MOV D, A
	XRA A
	MVI E, 0AH
SUM	ADD D
	DCR E
	JNZ SUM
SKIPMULTIPLY	ADD C
	STA 2020H
	HLT

OBSERVATION:



Programming with 8086 Microprocessor

ASSEMBLY LANGUAGE PROGRAM FEATURES

Comments: A Comment begins with Semicolon(;).

#RESERVED WORDS
Instructions: MOV, ADD
Directives : END,SEGMENT
Operators : FAR,OFFSET
Predefined Symbols : @DATA

#IDENTIFIERS: An Identifier (or symbol) is a name that you apply to an item in your program that you expect to reference. The two types of identifiers are NAME and LABEL.

NAME: Refers to the Address of a data item COUNTER DB 0

LABEL: Refer to the Address of an instruction, procedure, or segment.

MAIN PROC A20: MOV AL, BL

#STATEMENTS: An Assembly Program consists of a set of statements. The two types of statements are:

INSTRUCTIO: Instructions such as MOV & ADD which the Assembler translates to Object Code.

DIRECTIVES: Directives tell the Assembler to perform a specific action, such as define a data item etc.

ASSEMBLY LANGUAGE PROGRAMMING USING MASM GENERAL PATTERN FOR WRITING ALP IN MASM

[PAGE DIRECTIVE] [TITLE DIRECTIVE]
[MEMORY MODEL DEFINITION]

[SEGMENT DIRECTIVES]
[PROC DIRECTIVES]
……

……

……

……
[END DIRECTIVES]

BASIC FORMAT OF ALP BASED UPON THE GENERAL PATTERN

```
PAGE 60, 80
TITLE "ALP TO PRINT FACTORIAL NO"

.MODEL [MODEL NAME]
.STACK
.DATA
……; INITIALIZE DATA VARIALBLES
.CODE

MAIN PROC ……
……..
……..
……; INSTRUCTION SETS
……..  ……
……..
……

MAIN ENDP END MAIN
```

DIRECTIVES

Assembly Language supports a number of statements that enable to control the way in which a source program assembles and lists. These Statements are called Directives.

They act only during the assembly of a program and generate no machine executable code.

The most common Directives are PAGE, TITLE, PROC, and END.

 # PAGE DIRECTIVE

- The PAGE Directive helps to control the format of a listing of an assembled program.

- It is optional Directive.

- At the start of program, the PAGE Directive designates the maximum number of lines to list on a page and the maximum number of characters on a line.

- Its format is
- PAGE [LENGTH],[WIDTH]

- Omission of a PAGE Directive causes the assembler to set the default value to PAGE 50,80

 # TITLE DIRECTIVE

- The TITLE Directive is used to define the title of a program to print on line 2 of each page of the program listing.

- It is also optional Directive.

- Its format is TITLE [TEXT]

- TITLE "PROGRAM TO PRINT FACTORIAL NO"

 # SEGMENT DIRECTIVE

- The SEGMENT Directive defines the start of a segment.
- A Stack Segment defines stack storage, a data segment defines data items and a code segment provides executable code.
- MASM provides simplified Segment Directive.

The format (including the leading dot) for the directives that defines the stack, data and code segment are

.STACK [SIZE]
.DATA
…….. Initialize Data Variables
.CODE

The Default Stack size is 1024 bytes.

To use them as above, Memory Model initialization should be carried out.

 # MEMORY MODEL DEFINTION

- The different models tell the assembler how to use segments to provide space and ensure optimum execution speed.
- The format of Memory Model Definition is
- .MODEL [MODEL NAME]
- The Memory Model may be TINY, SMALL, MEDIUM, COMPACT, LARGE AND HUGE.

MODEL TYPE	**DESCRIPTION**
TINY	All DATA, CODE & STACK Segment must fit in one Segment of Size <=64K.

SMALL	One Code Segment of Size <=64K. One Data Segment of Size <=64 K.
MEDIUM	One Data Segment of Size <=64K. Any Number of Code Segments.
COMPACT	One Code Segment of Size < =64K. Any Number of Data Segments.
LARGE	Any Number of Code and Data Segments.
HUGE	Any Number of Code and Data Segments.

#THE PROC DIRECTIVE

The Code Segment contains the executable code for a program, which consists of one or more procedures, defined initially with the PROC Directive and ended with the ENDP Directive.
Its Format is given as: PROCEDURE NAME PROC
……..
……..
…….. PROCEDURE NAME ENDP

#END DIRECTIVE
As already mentioned, the ENDP Directive indicates the end of a procedure. An END Directive ends the entire Program and appears as the last statement.

Its Format is

```
END [PROCEDURE NAME]
```

```
#PROCESSOR DIRECTIVE
```

Most Assemblers assume that the source program is to run on a basic 8086 level.

As a result, when you use instructions or features introduced by later processors, you have to notify the assemblers be means of a processor directive as .286,.386,.486 or
.586

This directive may appear before the Code Segment.

#THE EQU DIRECTIVE

It is used for redefining symbolic names
EXAMPLE
DATAX DB 25 DATA EQU DATAX

#THE .STARTUP AND .EXIT DIRECTIVE

MASM 6.0 introduced the .STARTUP and .EXIT Directive to simplify program initialization and Termination.

.STARTUP generates the instruction to initialize the Segment Registers.

.EXIT generates the INT 21H function 4ch instruction for exiting the Program.

DEFINING TYPES OF DATA

The Format of Data Definition is given as [NAME] DN [EXPRESSION]

EXAMPLES

STRING DB 'HELLO WORLD' NUM1 DB 10
NUM2 DB 90

DEFINITION	**DIRECTIVE**
BYTE	DB
WORD	DW
DOUBLE WORD	DD
FAR WORD	DF
QUAD WORD	DQ
TEN BYTES	DT

Duplication of Constants in a Statement is also possible and is given by
[NAME] DN [REPEAT-COUNT DUP (EXPRESSION)]
EXAMPLES

```
            DATAX DB 5 DUP(12) ; 5 Bytes containing hex
    0c0c0c0c0c
    DATA DB 10 DUP(?) ; 10 Words Uninitialized

    DATAZ DB 3 DUP(5 DUP(4)) ; 44444 44444 44444
```

CHARACTER STRINGS

Character Strings are used for descriptive data. Consequently DB is the conventional format for defining character data of any length

An Example is
DB 'Computer City' DB "Hello World"
DB "St.Xavier's College"

2. NUMERIC CONSTANTS

#BINARY : VAL1 DB 10101010B

#DECIMAL : VAL1 DB 230

#HEXADECIMAL : VAL1 DB 23H

ALP SAMPLES USING DOS AND VIDEO BIOS FUNCTIONS

SAMPLE 1

;Program to Print Hello World in ALP

```
;-------------------------------------------------------------
.MODEL SMALL
.STACK
.DATA
STRING DB 'HELLO WORLD $'
```

```
.CODE
;-------------------------------------------------------
MAIN PROC
MOV AX,@DATA
MOV DS,AX ; Initialize the DATA Segment

MOV DX,OFFSET STRING ; Load the Offset Address into DX MOV AH,09H
; AH=09H For String Display until $ INT 21H ;   DOS
Interrupt Function

MOV AX,4C00H ; End Request with AH=4CH
; or AX=4C00H
INT 21H
MAIN ENDP ; End Procedure END MAIN ; End Program
```

 # SAMPLE 2

;Program to Print the Sum of Two 8 Bit Numbers

```
;-------------------------------------------------------------
.MODEL SMALL
.STACK
.DATA
       VAL1 DB 89
VAL2 DB 10
MSG DB 'SUM OF 2 NUMBERS: $'
     .CODE

;-------------------------------------------------------------

MAIN PROC
MOV AX,@DATA MOV DS,AX

MOV AX,0 MOV AL,VAL1 ADD AL,VAL2

AAM ;AAM Converts Binary Value to Unpacked BCD.


ADD AX,3030H ;Ax is Added with 3030H to Obtain ASCII Value
```

```
PUSH AX
;;;;;;;;;;;;;DISPLAY MESSAGE LEA DX,MSG
MOV AH,09H INT 21H
;;;;;;;;;;;;;END DISPLAY MESSAGE
POP AX MOV DL,AH
MOV DH,AL MOV AH,02H INT 21H

MOV DL,DH MOV AH,02H INT 21H

MOV AX,4C00H INT 21H
MAIN ENDP
END MAIN ;-------------------------------------------------------
```

------ **# SAMPLE 3**

;Program To Clear the Screen With Bios Interrupt Function.

```
;-------------------------------------------------------------
.MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------------------


MAIN PROC
MOV AX,@DATA MOV DS,AX


;;;;;;;;;;;;;;;;;;;;;;;;;
;INT 10H Function 06H : Scroll Up Screen
;AH=Function 06H
;AL=Number of lines to scroll,or 00H for full screen
;BH=Attribute value(color,blinking)
;CX=Strating row:column
;DX=Ending row :column
;;;;;;;;;;;;;;;;;;;;;;;;;

MOV AX,0600H ;AH=06,AL=00 for Full Screen
MOV BH,71H        ;White background(7),Blue foreground(1) MOV
CX,0000H ;Upper left row:column
```

```
MOV DX,184FH ;lower right row:column INT 10H ;Bios
Interrupt Function

MOV AX,4C00H INT 21H

MAIN ENDP END MAIN
;-------------------------------------------------------
```

 # SAMPLE 4
;Progam to Display Character with Attributes

```
;-------------------------------------------------------
-------
MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------------
-------

MAIN PROC
MOV AX,@DATA MOV DS,AX

;;;;;;;;;;;;;;;;;;;;;;
;INT 10H Function 09H: Display Character
;AH=09H
;AL=ASCII Character
;BH=Page Number
;BL=Attribute or Pixel value.
;CX=Count
;;;;;;;;;;;;;;;;;;;;;;

MOV AH,09H ;Request Display
MOV AL,01H ;Happy Face for display MOV BH,00H ;Page
Number 0
MOV BL,0C1H ;Red background,Blue foreground MOV CX,79
```

```
     ;No of repeated characters
INT 10H ;Bios Interrupt Function

MOV AX,4C00H INT 21H

MAIN ENDP END MAIN
;-------------------------------------------------------
-------
```

;Program to Print Hello World in ALP

```
;-------------------------------------------------------------
------
.MODEL SMALL
.STACK
.DATA
     STRING DB 'HELLO WORLD $'
.CODE
 ;-------------------------------------------------------------
------

MAIN PROC
     MOV AX,@DATA
     MOV DS,AX            ; Initialize the DATA Segment

     MOV DX,OFFSET STRING ; Load the Offset Address into DX
     MOV AH,09H           ; AH=09H For String Display until $
     INT 21H              ; DOS Interrupt Function
```

```
      MOV AX,4C00H       ; End Request with AH=4CH or AX=4C00H
     INT 21H
     MAIN ENDP            ; End Procedure
END MAIN                  ; End Program


;-----------------------------------------------------------------
------
```

;16 Bit Binary Content of Ax is Converted to 4 Digit ASCII

```
;-----------------------------------------------------------------
------
.MODEL SMALL
.STACK
.DATA
        VAL DW 256
.CODE
;-----------------------------------------------------------------
------
MAIN PROC
        MOV AX,@DATA
        MOV DS,AX ;DATA SEGMENT INITIALIZATION

        MOV AX,VAL
        XOR DX,DX   ;DX IS CLEARED
        MOV CX,100  ;DIVISOR IS PASSED INTO CX REGISTER

        DIV CX      ;DX:AX PAIR DIVIDED BY CX
    ;QUOTIENT  IN AX     ;REMAINDER IN DX
```

```
        AAM           ;QUOTIENT IS ADJUSTED TO UNPACKED BCD
        ADD AX,3030H  ;QUOTIENT IS CONVERTED TO ASCII
        XCHG AX,DX    ;DX AND AX ARE SWAPPED
        AAM           ;REMAINDER IS ADJUSTED TO UNPACKED BCD
        ADD AX,3030H  ;REMAINDER IS CONVERTED TO ASCII

        ;;;;;;;;;;;;;;;;;;DISPLAY OPERATION
        ;;DISPLAY QUOTIENT
        ;;DISPLAY REMAINDER


        ;;;;;;;;;;;;;;;;;;END DISPLAY


        MOV AX,4C00H ;END REQUEST
        INT 21H      ;DOS INTERRUPT FUNCTION

MAIN ENDP
END MAIN
 ;-------------------------------------------------------------
------
```

;Program to Print the Difference of Two 8 Bit Numbers

```
;-------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        VAL1 DB 89
        VAL2 DB 10
        MSG DB 'DIFFERENCE OF 2 NUMBERS: $'
.CODE

;-------------------------------------------------
---------------
```

```
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX

    MOV AX,0
    MOV AL,VAL1
    SUB AL,VAL2


    ;;;;; AAM Converts Binary Value to Unpacked BCD.
    ;;;;; AAM Only Works with AX Register
    AAM

    ;;;;; AX is Added with 3030H to Obtain ASCII Value.
    ADD AX,3030H

    PUSH AX

    ;;;;; DISPLAY MESSAGE With AH=09H
    LEA DX,MSG
    MOV AH,09H
    INT 21H
    ;;;;; END DISPLAY MESSAGE

    POP AX

    MOV DL,AH
    MOV DH,AL
    MOV AH,02H
    INT 21H

    MOV DL,DH
    MOV AH,02H
    INT 21H

    MOV AX,4C00H
    INT 21H
MAIN ENDP
END MAIN

;-------------------------------------------------------
---------------
```

;Program to Print the Sum of Two 16 Bit Numbers

```
;-------------------------------------------------------
--------------
.MODEL SMALL
.STACK
.DATA
        VAL1 DW 2010
        VAL2 DW 2050
 .CODE
;-------------------------------------------------------
--------------
MAIN PROC
    MOV AX,@DATA
    MOV DS,AX    ;INITIALIZE THE DATA SEGMENT

    MOV AX,VAL1
    ADD AX,VAL2  ;AX Holds 16 bit Value
    ;; 16 Bit Answer Splitting Strategy.
    ;; 16 Bit Division is Carried out
 ;; 32 Bit Divident (DX:AX) and 16 Bit Divisor is
Required.

    XOR DX,DX    ;Register DX is Cleared
    MOV CX,100
    DIV CX       ;DX:Ax Divided by CX.
                  ;Remainder in DX and Quotient in AX
    AAM          ;Quotient is Converted to Unpacked BCD
     ADD AX,3030H ;Ready  For Display i.e Converted to
ASCII
    MOV BX,AX    ;Store the Quotient to BX

    XCHG AX,DX   ;Exchange the Contents of AX and DX
      AAM         ;Remainder is Converted to Unpacked
BCD
    ADD AX,3030H ;Remainder Converted to ASCII
    PUSH AX      ;Remainder Pushed to Stack
 ;;;;;;;;;;;;;;Quotient Ready in BX and Remainder Ready
in AX
     ;;;;;;;;;;;;;;Display Quotient First and then the
Remainder

    ;;;;;;;;;;;;;;Display Operation Started
    MOV DL,BH
    MOV AH,02H
    INT 21H
```

```
    MOV DL,BL
    MOV AH,02H
    INT 21H
    POP AX
    MOV DL,AH
    MOV DH,AL
    MOV AH,02H
    INT 21H
    MOV DL,DH
    MOV AH,02H
    INT 21H
    ;;;;;;;;;;;;; Display Operation End

    MOV AX,4C00H ; End Request AH=4CH
     INT 21H    ; Dos Interrupt Function
MAIN ENDP
END MAIN
;-------------------------------------------------------
--------------
```

;Program to Display Numbers From 0 To 9.[0 1 2 3 4 5 6 7 8 9]

```
;-------------------------------------------------------
--------------
.MODEL SMALL
.STACK
.DATA
        VAL DB '0'
.CODE

;-------------------------------------------------------
--------------
SPACE MACRO
        MOV DL,' '
        MOV AH,02H
        INT 21H
ENDM
;-------------------------------------------------------
--------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX
```

```
        MOV CX,26
MOV DL,VAL
        TOP:
                MOV AH,02H
                INT 21H ;DISPLAY THE NUMBER
                INC DL  ;DL IS INCREMENTED BY 1
                PUSH DX ;PUSH THE CONTENT OF DX TO
                          STACK

                ;;;;;;;;; INVOKED SPACE MACRO
              SPACE
                ;;;;;;;;; END OF SPACE MACRO


               POP DX  ;POP THE CONTENT FROM STACK TO DX
DEC CX  ;DECREMENT THE CONTENT OF COUNT BY 1.
                JZ LAST ;JUMP TO LABEL LAST IF CX=0
                JMP TOP ;UNCONDITIONAL JUMP TO LABEL
TOP

        LAST:
                MOV AH,4CH
                INT 21H
MAIN ENDP
END MAIN

;-------------------------------------------------------



;Program to Display Numbers From 0 To 9 With Line Feed
;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        VAL DB 0
.CODE

;-------------------------------------------------------
---------------
LFEED MACRO
        MOV AH,06H
        MOV DL,0AH
```

```
        INT 21H
        MOV DL,0DH
        INT 21H
ENDM
;-------------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        MOV CX,10
        MOV AH,00H
        MOV AL,VAL
TOP:

                MOV BL,AL
                AAM
                ADD AX,3030H
                MOV DL,AL
                MOV AH,02H
                INT 21H

                ;;;;;;;; INVOKED LFEED MACRO FOR LINE
FEED
                LFEED
              ;;;;;;;; END OF LFEED MACRO

                INC BL
                MOV AL,BL
                DEC CX
                JZ LAST
                JMP TOP
        LAST:
                MOV AH,4CH
                INT 21H
MAIN ENDP
END MAIN

;-------------------------------------------------------
---------------
```

```
;Program to Display Alphabets From A To Z. [A B C D E F
…….. Z]

;-----------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        VAL DB 'A'
.CODE
;-----------------------------------------------------
---------------

SPACE MACRO
        MOV DL,' '
        MOV AH,02H
        INT 21H
ENDM

;-----------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        MOV CX,26
MOV DL,VAL
        TOP:
                MOV AH,02H
                INT 21H
                INC DL
                PUSH DX
              ;;;;;;;;; INVOKED SPACE MACRO
                SPACE
              ;;;;;;;;; END OF SPACE MACRO

                POP DX
                DEC CX
                JZ LAST
                JMP TOP
        LAST:
                MOV AH,4CH
                INT 21H
MAIN ENDP
```

```
END MAIN

;-----------------------------------------------------
--------------
;Program to Print the Sum of Natural Nos From 1 To
10.[1+2+3….+10]

;-----------------------------------------------------
--------------
.MODEL SMALL
.STACK
.DATA
        VAL DB 1
.CODE
;-----------------------------------------------------
--------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        MOV CX,10
        MOV AL,VAL
        MOV DL,0

        TOP:
                ADD DL,AL ;DL=DL+AL
                INC AL    ;INCREMENT THE CONTENT OF AL
                DEC CX    ;DECREMENT THE CONTENT OF CX
                JZ LAST   ;JUMP TO LAST IF CX=0
                JMP TOP   ;UNCONDITIONAL JUMP TO LABEL
TOP


        LAST:

             MOV AL,DL  ;FINAL SUM IN DL IS PASSED TO AL
                          ;AAM ALWAYS WORKS WITH AX
         REGISTER

             ;;;;;;;;;;;;DISPLAY OPERATION STARTED
               AAM
               ADD AX,3030H
               MOV DL,AH
               MOV DH,AL
```

```
                MOV AH,02H
                INT  21H
                MOV DL,DH
                MOV AH,02H
                INT 21H

              ;;;;;;;;;;;;END DISPLAY
                MOV AH,4CH
                INT 21H
MAIN ENDP
END MAIN
;-------------------------------------------------------
---------------


;Program to Demonstrate Multiplication Table of a Given
Number

;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
       VAL DB 7

.CODE

;-------------------------------------------------------
---------------
GODOWN MACRO
     MOV DL,0DH
     INT 21H
     MOV DL,0AH
     INT 21H
ENDM

;-------------------------------------------------------
---------------

MAIN PROC
     MOV AX,@DATA
     MOV DS,AX ; Initialize Data Segment

     MOV AX,0  ; Make AX as 0
     MOV CX,10 ;Initialize the Counter
```

```
     MOV BL,1

     TOP:
          MOV AL,BL
           MUL VAL ; Multiply the Content of Val and AL :
Answer in AL
          AAM
          ADD AX,3030H
          MOV DL,AH
          MOV DH,AL
          MOV AH,02H
          INT 21H
          MOV DL,DH
          MOV AH,02H
          INT 21H
          GODOWN ; Invoked GoDOWN Macro
          MOV AX,0
          INC BL
          DEC CX
          JZ LAST
        JMP TOP
   LAST:
     MOV AX,4C00H
     INT 21H
MAIN ENDP
END MAIN

;-------------------------------------------------------
---------------

;Program to Calculate Factorial of Given Number
;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
         VAL DW 7

.CODE
;-------------------------------------------------------
---------------
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX  ; Initialize Data Segment
```

```
    MOV AX,VAL ; AL is Passed with the Content of DI
    MOV CX,AX  ; Counter to  no given in val
  DEC CX     ; Decrement the Content of CL to make n1

    TOP:
          MUL CX      ; Multiply the content of AL and
CL answer in AL
         DEC CX      ; Decrement the content of cl
         JZ LAST     ; Jump if Zero to Last
       JMP TOP     ; JMP to Top Unconditionally
    LAST:
         MOV CX,100
       DIV CX      ; Quotient in AX and Remainder in DX
         AAM         ; Adjust Quotient to Unpacked BCD
         ADD AX,3030H
         MOV BX,AX   ; Prserve the Content of AX to BX
         XCHG AX,DX
         AAM
         ADD AX,3030H
          PUSH AX     ; Preserve the Content of AX into
Stack

           ;;;;;;;;;;;;; Display Operation Started
         MOV DL,BH
         MOV AH,02H
         INT 21H
         MOV DL,BL
         MOV AH,02H
         INT 21H     ; Quotient Printing is Finished
         POP AX
         MOV DL,AH
         MOV DH,AL
         MOV AH,02H
         INT 21H
         MOV DL,DH
         MOV AH,02H
         INT 21H     ; Remainder Printing is Finished
         ;;;;;;;;;;;;; Display Operation Finished

         MOV AX,4C00H
         INT 21H     ; Normal End Request
MAIN ENDP
END MAIN

;-------------------------------------------------
```

```
---------------
;Program to Print the Fibonaci Series.[1 2 3 5 8 13 21
..]

;-----------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        VAL1 DB 0
        VAL2 DB 1
.CODE
;-----------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        MOV AL,VAL1 ; A = VAL1
        MOV BL,VAL2 ; B = VAL2
        MOV CX,10
        MOV AH,00H

        TOP:
                ADD AL,BL ; P= A+B
                MOV BH,BL ; TEMP1=B
                PUSH AX   ; CONTENT OF A IS PUSHED TO
STACK

              ;;;;;;;;;;; DISPLAY OPERATION STARTED

                AAM
                ADD AX,3030H
                MOV DL,AH
                MOV DH,AL
                MOV AH,02H
                INT 21H

                MOV DL,DH
                MOV AH,02H
                INT 21H

                MOV DL,' '
                MOV AH,02H
```

```
                INT 21H
                ;;;;;;;;;;; DISPLY OPERATION END

                POP AX    ; CONTENT OF STACK IS POPED
TO AX
                MOV BL,AL ; B=P
                MOV AL,BH ; A=B

                LOOP TOP  ; DECREMENT THE CONTENT OF CX
BY1
                      ; JUMP TO LABEL TOP UNTIL CX > 0
                MOV AX,4C00H
                INT 21H

MAIN ENDP
END MAIN

;-----------------------------------------------------
---------------




;Program To Demonstrate File Handles For Input and
Output
;-----------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        KEYDISP DB 'Enter Character/Number[10]: $'

        KEYINP  DB 20 DUP(' ') ;Additional 2 Characters
For 0DH & 0CH
                               ;0DH : Enter
                               ;0CH : Line Feed
        KEYOUT  DB 'Output : $'
.CODE
;-----------------------------------------------------
---------------
DISPLAY MACRO A
        LEA DX,A
        MOV AH,09H
        INT 21H
```

```
ENDM
;-------------------------------------------------------
---------------
MAIN PROC
        MOV AX,@DATA
        MOV DS,AX
                              ; KEYDISP IS A PARAMETER TO
MACRO
        DISPLAY KEYDISP     ; Invoked DISPLAY MACRO
        CALL READ
                         ; KEYOUT IS A PARAMETER TO MACRO
       DISPLAY KEYOUT   ; Invoked DISPLAY MACRO
MOV CX,20
        TOP :
                MOV DL,[BX]
                MOV AH,02H
                INT 21H
                INC BX
                DEC CX
                JZ LAST
                JMP TOP
         LAST:
                MOV AX,4C00H
                INT 21H
MAIN ENDP
;-------------------------------------------------------
---------------

;READ PROCEDURE FOR KEYBOARD INPUT USING FILE HANDLES
;AH=FUNCTION 3FH
;BX=00H INDICATES INPUT
;CX=MAXIMUM NO OF CHARACTERS TO ACCEPT
;DX=ADDRESS OF AREA FOR ENTERING CHARACTERS
READ PROC
        MOV AH,3FH
        MOV BX,00H
        MOV CX,20
        LEA DX,KEYINP
        INT 21H
        MOV BX,DX
        RET
READ ENDP
;-------------------------------------------------------
---------------
END MAIN
```

```
;Program to Print the Input String In Reverse Order.

;-------------------------------------------------
.MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------

MAIN PROC
    ;;;;;;;;;;;;;;;Data Segment Initialization Started

    MOV AX,@DATA
    MOV DS,AX

    ;;;;;;;;;;;;;;;;Data Segment Initialization Ended


    MOV CX,0              ;Initialize the Counter as 0

    READ_CHAR:
        MOV AH,01H     ;Put AH=01H to Read Character
                           From Keyboard
        INT 21H        ;Reads Character and Puts it in
                           AL
        CMP AL,0DH     ;0DH is the ASCII Code For Enter
                           Key
                       ;Check if Enter Key is Pressed

         JE END_OF_LINE  ;If Enter Pressed Go to
                           END_OF_LINE
         PUSH AX        ;Push Character into Stack
         INC CX         ;Increment Counter CX
        JMP READ_CHAR    ;Unconditional Jump to
                           READ_CHAR

    END_OF_LINE:
         POP DX          ;Pop Character From Stack into
                              DX
        MOV AH,02H         ;Code for Displaying
                              character on VDU
         INT 21H          ;DOS Interrupt Function
        LOOP END_OF_LINE  ;Loop Until CX=0

        MOV AX,4C00H    ;End Request
```

```
        INT 21H         ;Dos Interrrupt Function

MAIN ENDP
END MAIN
;-------------------------------------------------------
;Program  To  Clear  the  Screen  With  Bios  Interrupt
Function.
;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

       ;;;;;;;;;;;;;;;;;;;;;;;;;;
        ;INT 10H Function 06H : Scroll Up Screen
        ;AH=Function 06H
        ;AL=Number of lines to scroll,or 00H for full
            screen
        ;BH=Attribute value(color,blinking)
        ;CX=Strating row:column
        ;DX=Ending row :column
        ;;;;;;;;;;;;;;;;;;;;;;;;;;

        MOV AX,0600H  ;AH=06,AL=00 for Full Screen
MOV BH,71H    ;White background(7),Blue
                        foreground(1)
        MOV CX,0000H  ;Upper left row:column
        MOV DX,184FH  ;lower right row:column
        INT 10H       ;Bios Interrupt Function

        MOV AX,4C00H
        INT 21H

MAIN ENDP
END MAIN
;-------------------------------------------------------
---------------
```

```
;Program to Set the Cursor in Desired Location

;-------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
.CODE ;-------------------------------------------------
-----
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        ;;;;;;;;;;;;;;;;;;;
        ;INT 10H Function 02H: Set Cursor Position
        ;AH=Request Cursor with AH=02H
        ;BH=Page Number [Default 00H]
        ;DH=Row
        ;DL=Column
        ;;;;;;;;;;;;;;;;;;;


        MOV AH,02H ;Request Set Cursor
        MOV BH,00  ;Page Number to 0
        MOV DH,08  ;Row 8
        MOV DL,50  ;Column 50
        INT 10H    ;Bios Interrupt Function


        MOV DL,'C' ;Character to be Displayed in DL
        MOV AH,02H ;Character Mode
        INT 21H    ;Dos Interrupt Function


        MOV AX,4C00H
        INT 21H
MAIN ENDP
END MAIN
;-------------------------------------------------
-------------
;Program to Display Character with Attributes

;-------------------------------------------------
```

```
---------------
.MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        ;;;;;;;;;;;;;;;;;;;;;;
        ;INT 10H Function 09H: Display Character
        ;AH=09H
        ;AL=ASCII Character
        ;BH=Page Number
        ;BL=Attribute or Pixel value.
        ;CX=Count
        ;;;;;;;;;;;;;;;;;;;;;;

        MOV AH,09H  ;Request Display
        MOV AL,01H  ;Happy Face for display
        MOV BH,00H  ;Page Number 0
        MOV BL,0C1H ;Red background,Blue foreground
        MOV CX,79   ;No of repeated characters
        INT 10H     ;Bios Interrupt Function

        MOV AX,4C00H
        INT 21H

MAIN ENDP
END MAIN
;-------------------------------------------------------
---------------


;Program to Demonstrate Screen Scroll Down

;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
.CODE
```

```
;-------------------------------------------------------
---------------

MAIN PROC
        MOV AX,@DATA
        MOV DS,AX

        ;;;;;;;;;;;;;;;;;;;;;;;;;
        ;INT 10H Function 07H : Scroll Down Screen
        ;AH=Function 07H
        ;AL=Number of lines to scroll,or 00H for full
screen
        ;BH=Attribute value(color,blinking)
        ;CX=Strating row:column
        ;DX=Ending row :column
        ;;;;;;;;;;;;;;;;;;;;;;;;;

        MOV AX,0702H  ;AH=07,AL=00 for Full Screen
        MOV BH,71H    ;White background(7),Blue
foreground(1)
        MOV CX,0C19H  ;Upper left row:column
        MOV DX,1236H  ;lower right row:column
        INT 10H       ;Bios Interrupt Function

       MOV AX,4C00H
       INT 21H
MAIN ENDP
END MAIN

;-------------------------------------------------------
---------------
;Program to Plot Pixels in Graphics Mode

;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
.CODE
;-------------------------------------------------------
---------------
MAIN PROC
        MOV AX,@DATA
        MOV DS,AX
```

```
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
;INT 10H: Get Current Video Mode
;AH=0FH
;AL=Returns Current Video Mode
;AH=Number of Screen Columns
;BH=Active Video Page

MOV AH,0FH
INT 10H
PUSH AX ;PUSH the Current Mode to Stack

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
;INT 10H : Set Video Mode
;AH=00H
;AL=Required Mode
MOV AH,00H
MOV AL,13H
INT 10H
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;
;INT 10H : Write Pixel Dot
;AH=0CH
;AL=Color of the Pixel
;BH=Page Number
;CX=Column
;DX=ROW
MOV BH,00
MOV DX,10
MOV CX,10
MOV BL,00
TOP:
        MOV AH,0CH
        MOV AL,BL
        INT 10H
        INC DX
        INC BL
        CMP DX,100
        JNE TOP
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
```

```
        ;
        ;KEYBOARD INPUT
        ;AH=01H
        ;AL=Returns Key IN ASCII

        MOV AH,01H
        INT 21H
        ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;


        ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
        ;
        ;Return to Previous Graphics Mode

        POP AX
        MOV AH,00H
        INT 10H
        ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;


        MOV AX,4C00H
        INT 21H

MAIN ENDP
END MAIN
;-------------------------------------------------------
---------------

;Program to Print the String in Reverse Order

;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
         STRING DB '!EMOC-LEW'
         REVERSE DB 9 DUP(' ')
.CODE
;-------------------------------------------------------
---------------

MAIN PROC
         MOV AX,@DATA
         MOV DS,AX
```

```
        LEA SI,STRING  ;Load Effective Address of STRING
                          into SI
        LEA DI,REVERSE ;Load Effective Address of
                          REVERSE into DI
        ADD DI,9       ;Add the Address of DI With 9
        MOV CX,9    ;Initialize Counter as 9

        TOP:
                MOV AL,[SI] ;Content of SI to AL
MOV [DI],AL ;Content of AL to Content
                              of DI
                INC SI    ;Increment SI
                DEC DI    ;Decrement DI
                LOOP TOP    ;Loop Until  CX!=0


         ADD DI,10   ;Add DI With 10 to Locate to End
         MOV AL,'$'  ;Add String Termination Character
         MOV [DI],AL

         MOV AH,09H     ;AH=09 Specifies String
         LEA DX,REVERSE ;Load Effective Address of
REVERSE into DX
         INT 21H     ;DOS Interrupt Function

         MOV AH,4CH
         INT 21H

MAIN ENDP
END MAIN ;-----------------------------------------------
--------
---------------




;Program To Change The String Into Toggle Case
;-------------------------------------------------------
---------------
.MODEL SMALL
.STACK
.DATA
        STRING DB 'Welcome'
        CASE DB 7 DUP(' ')
```

```
.CODE
;-------------------------------------------------
---------------
MAIN PROC
        MOV AX,@DATA
        MOV DS,AX
        LEA SI,STRING ;Load Effective Address of STRING
                        into SI
        LEA DI,CASE   ;Load Effective Address of CASE
                        with DI
        MOV CX,7   ;Load Counter with 7
        TOP:
                CMP CX,0000H ;Compare CX with 0
                JE EXIT     ;If CX=0 then Goto Exit
                MOV AH,[SI]  ;Load Content of SI into
                                AH
              CMP AH,60H   ;Compare AH with 60H i.6 96
JA ISSMALL
                CMP AH,5AH   ;Compare AH with 5AH i.e
                                90
                JB ISCAP

             ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
           ;TO UPPERCASE
         ISSMALL:
                        AND AH,11011111B ;Mask With
                                          11011111B
                        MOV [DI],AH
                        INC SI
                        INC DI
                        DEC CX
                        JMP TOP
            ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;; ;TO LOWERCASE
ISCAP:
                        OR AH,00100000B ;Mask With
                                          00100000B
                        MOV [DI],AH
                        INC SI
                        INC DI
                        DEC CX
                        JMP TOP
              ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
          EXIT:
```

```
                    MOV AL,'$'   ;Add String Terminator.
MOV [DI],AL   ;Pass the Content of AL
                  to DI.
                    MOV DX,OFFSET CASE
                    MOV AH,09H
                    INT 21H
                    MOV AH,4CH
                    INT 21H
MAIN ENDP
END MAIN
;-------------------------------------------------
---------------
```

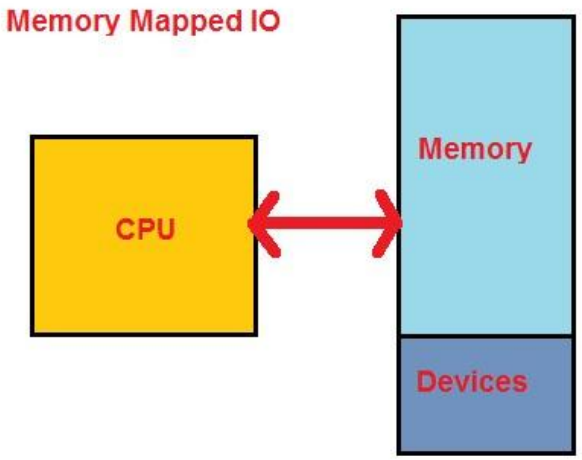
CHAPTER 5: BASIC I/O, MEMORY R/W AND INTERRUPT OPERATIONS

Memory Mapped I/O, I/O Mapped I/O and Hybrid I/O

Memory Mapped I/O

In case of memory mapped IO, external devices are mapped to the system memory in the same way as ROM and RAM is mapped. It means devices can be accessed in the same way as we access memory in general scenario. Every instruction which can excess memory can be used to access any IO. No need of having any special set of instruction to access these kinds of IO.

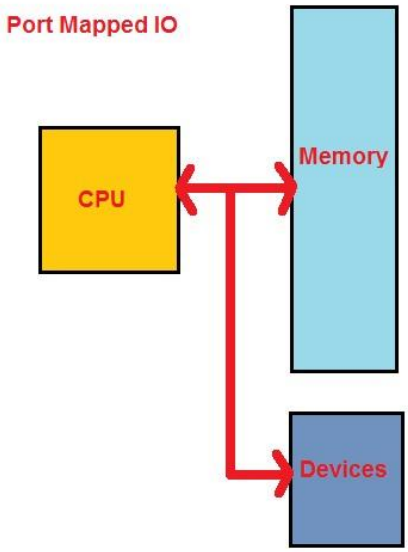
In case of memory mapped IO, IO devices use same address bus as memory.



The disadvantage of this approach is that entire address bus needs to be decoded for every device. For accessing these IO address bus will require logic gates to decode the address of device being accessed. It increase the cost of hardware.

I/O Mapped I/O (Port Mapped I/O)

I/O mapped I/O use different dedicated address space. This address space can be accessed by special instructions which are completely dedicated to access I/O mapped I/O. For example Intel x86 architecture use IN and OUT commands to Read and Write data respectively. I/O mapped I/O also use the same address bus but might be needed only few lines to access all the available devices, which means less hardware will be required to decode the address of devices and hence less system cost.



If System has option of I/O based I/O, then its recommended to use I/O mapped I/O for accessing the devices. This will avoid underutilization of system resources. For example: if a system has capacity to access 64K memory and 256 I/Os. If we use memory mapped I/Os then in that case we need to reserve some memory for I/Os and only the left over part can be accessed for general purpose memory. But if you will use I/O based I/O, then you have access to whole 64K memory as well as 256 I/Os.

Direct Access Memory(DMA)

- The data transfer technique in which peripherals manage the memory buses for direct interaction with main memory without involving the CPU is called direct memory access

(DMA). Using DMA technique large amounts of data can be transferred between memory and the peripheral without severely impacting CPU performance.

- During the DMA transfer, the CPU is idle and has no control of the memory buses. A DMA controller takes over the buses to manage the transfer directly between the I/O device(s) and main memory. The structure of DMA controller is described below.

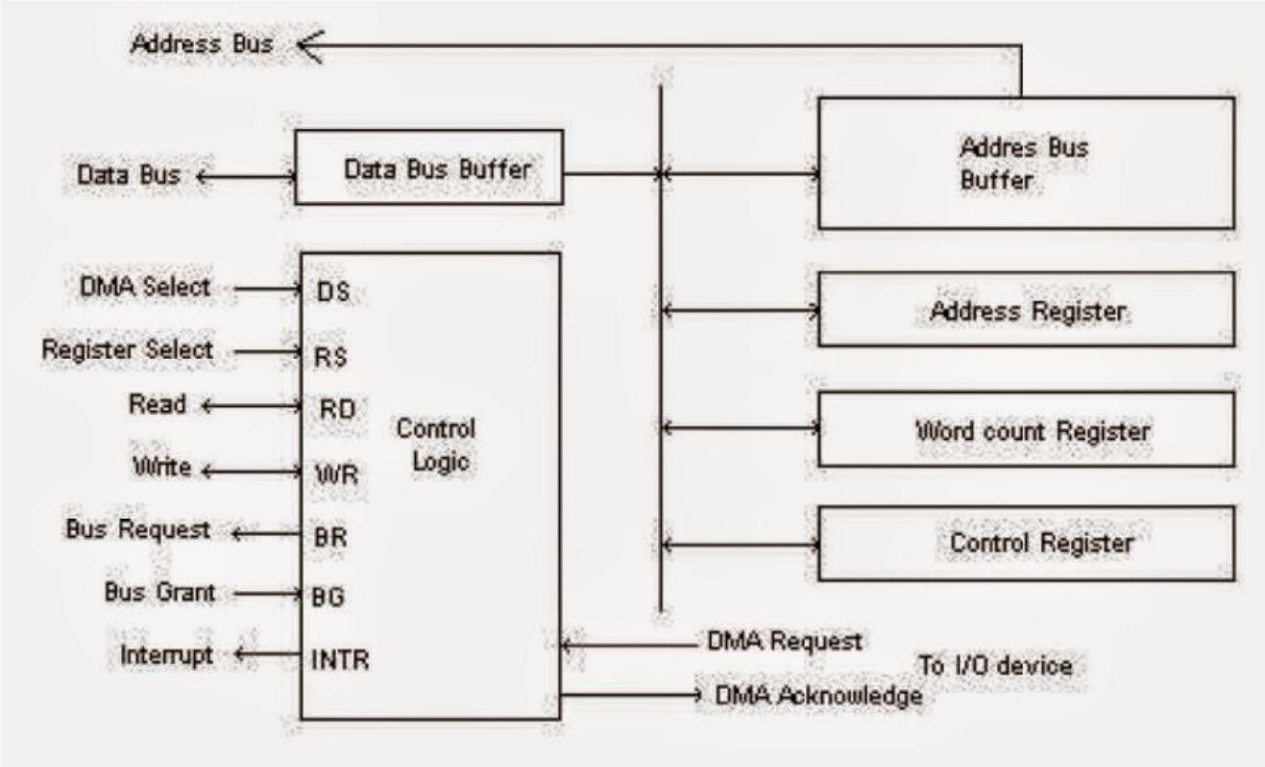


Figure: Block Diagram of DMA Controller

The control unit communicates the CPU via data bus and control lines.

- The DMA controls/relinquishes the system bus using BR (Bus Request) and BG (Bus Grant) signals. DMA operates read and write operations via RD (Read) and WR (Write) signals. DMA sends request and acknowledge to I/O devices via DMA request and DMA acknowledge signals.
- The registers in DMA are selected by CPU through the address bus by enabling DS (DMA Select) and RS (Register Select) inputs.
- All registers in the DMA appear to the CPU as I/O interface registers. The address register contains an address to specify the desired location in memory. It is incremented after each word that is transferred to the memory. The word count register holds the number of words to be transferred. It is decremented by one after each word transfer and internally tested for zero. The control register specifies the mode of transfer.
- The DMA transfer operation is described below:

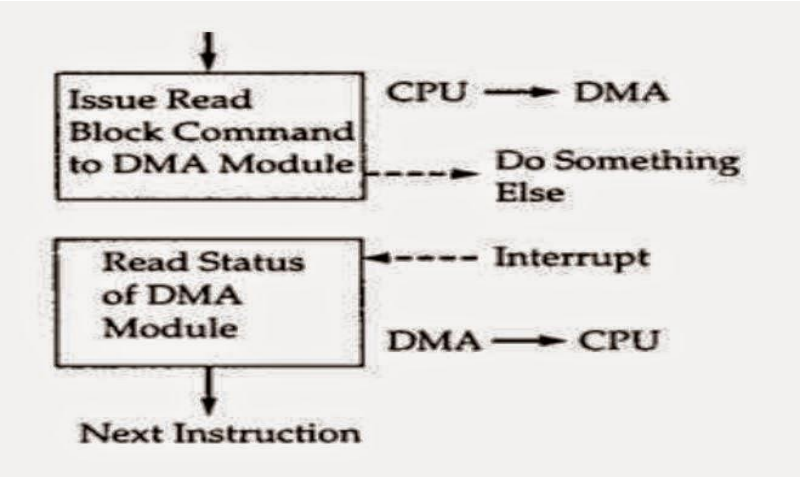


Figure: Direct Memory Access Technique

The DMA request CPU to handle control of buses to the DMA using bus request (BR) signal. The CPU grants the control of buses to DMA using bus grant (BG) signal after placing the address bus, data bus and read and write lines into high impedance state (which behave like open circuit).

CPU initializes the DMA by sending following information through the data bus.

1. Starting address of memory block for read or write operation.
2. The word count which is the no. of words in the memory block.
3. Control to specify the mode of transfer such as read or write.
4. A control to start the DMA transfer.

The DMA takes control over the buses directly interacts with memory and I/O units and transfers the data without CPU intervention. When the transfer completes, DMA disables the BR line. Thus CPU disable BG line, takes control over the buses and return to its normal operation.

The DMA transfer operation is illustrated below:

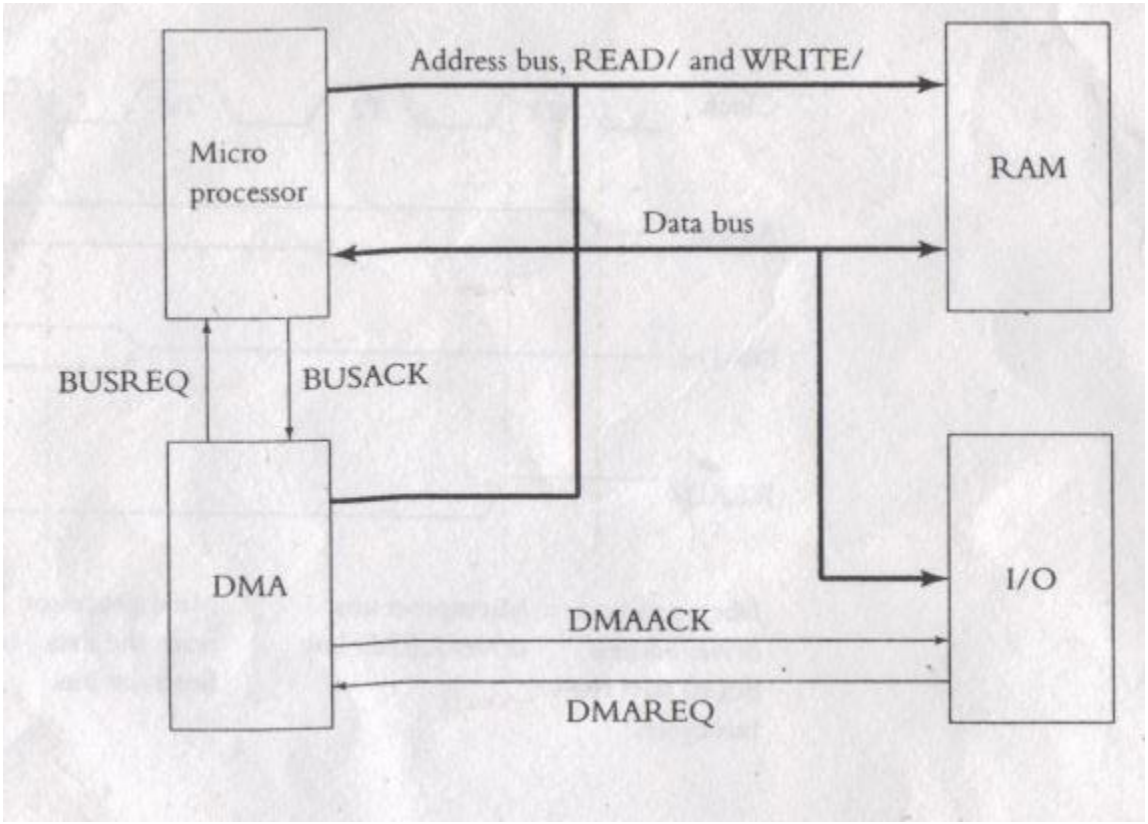


Fig: Illustration for DMA transfer in a computer system

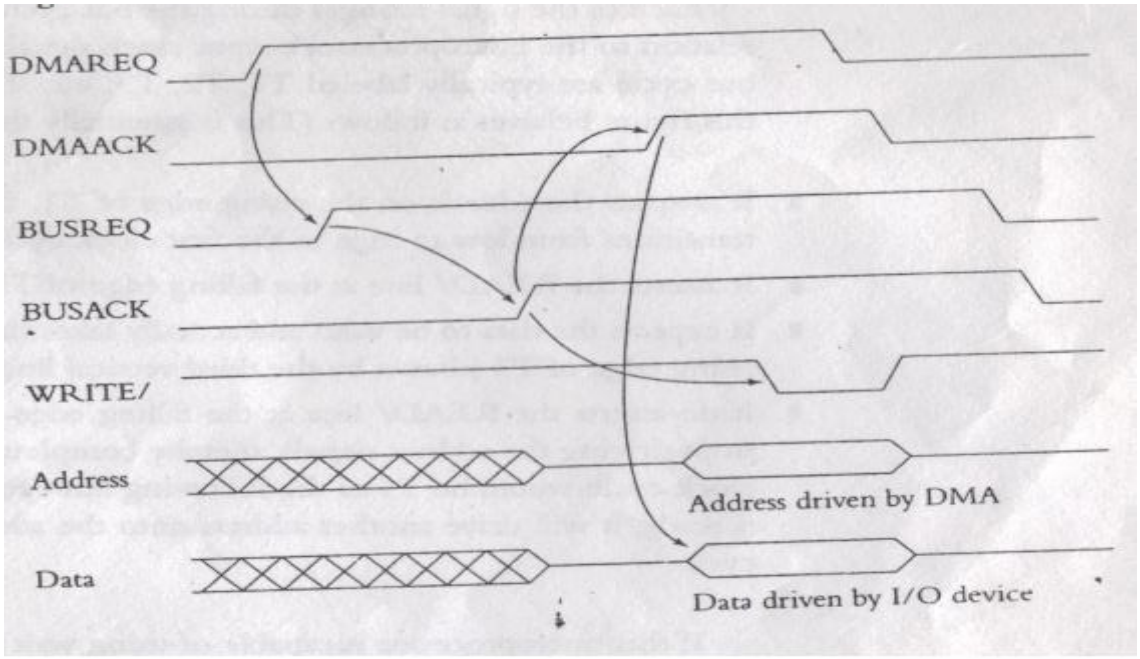


Figure : DMA Timing Diagram

DMA Transfer Modes

There are 3 different modes of DMA data transfer. They vary by how DMA controller determines when to transfer data (but actual data transfer process remains the same in all three cases).

(a) Burst Mode

- Entire block of data is transferred in one continuous sequence.
- Once the DMA controller is granted access to the system bus by CPU, it transfer all bytes of data in the data block before relinquishing control of system buses back to the CPU.
- This mode is useful for loading programs or data files into memory, but it keeps CPU idle for relatively long period of time.

(b) Cycle Stealing Mode

- DMA controller obtains access to system bus as in burst mode; transfers one byte of data and returns the control of the system bus to CPU. It continually issues requests using Bus Request (BR) signals, transferring one byte of data per request, until it has transferred its entire block of data. (steals one CPU cycle).
- The data block is not transferred as quickly as in burst mode, but the CPU is not idled for long period of time as in burst mode.

(c) Transparent Mode

- DMA controller only transfers data when CPU is performing operations that do not use system buses.
- The main advantage of this mode is that CPU never stops executing its program.
- The main disadvantages of this mode are
- Hardware needed to determine when the CPU is not using the system buses can be quite complex and relatively expensive.
- Requires highest time to transfer a block of data as compared to above two modes.

❖ To serve as a data transfer processor, DMA controller should have:

1. A data bus
2. An address bus
3. Read/Write control signals
4. Control signal to disable its role as a peripheral and to enable it as a processor

❖ **ADVANTAGES:**

1. Quick data transfer because a dedicated piece of hardware transfers data from one computer location to another and only one or two bus read/write cycles are required per piece of data transferred.
2. Minimizes latency in servicing a data acquisition device because the dedicated hardware responds more quickly than interrupts and transfer time is short.
3. Minimizes latency reduces the amount of temporary storage (memory) required on an I/O device.
4. Processor is not used for holding the data transfer activity and is available for other processing activity.
5. Also in systems where the processor primarily operates out of its cache, data transfer actually occurring in parallel, thus increasing overall system utilization.

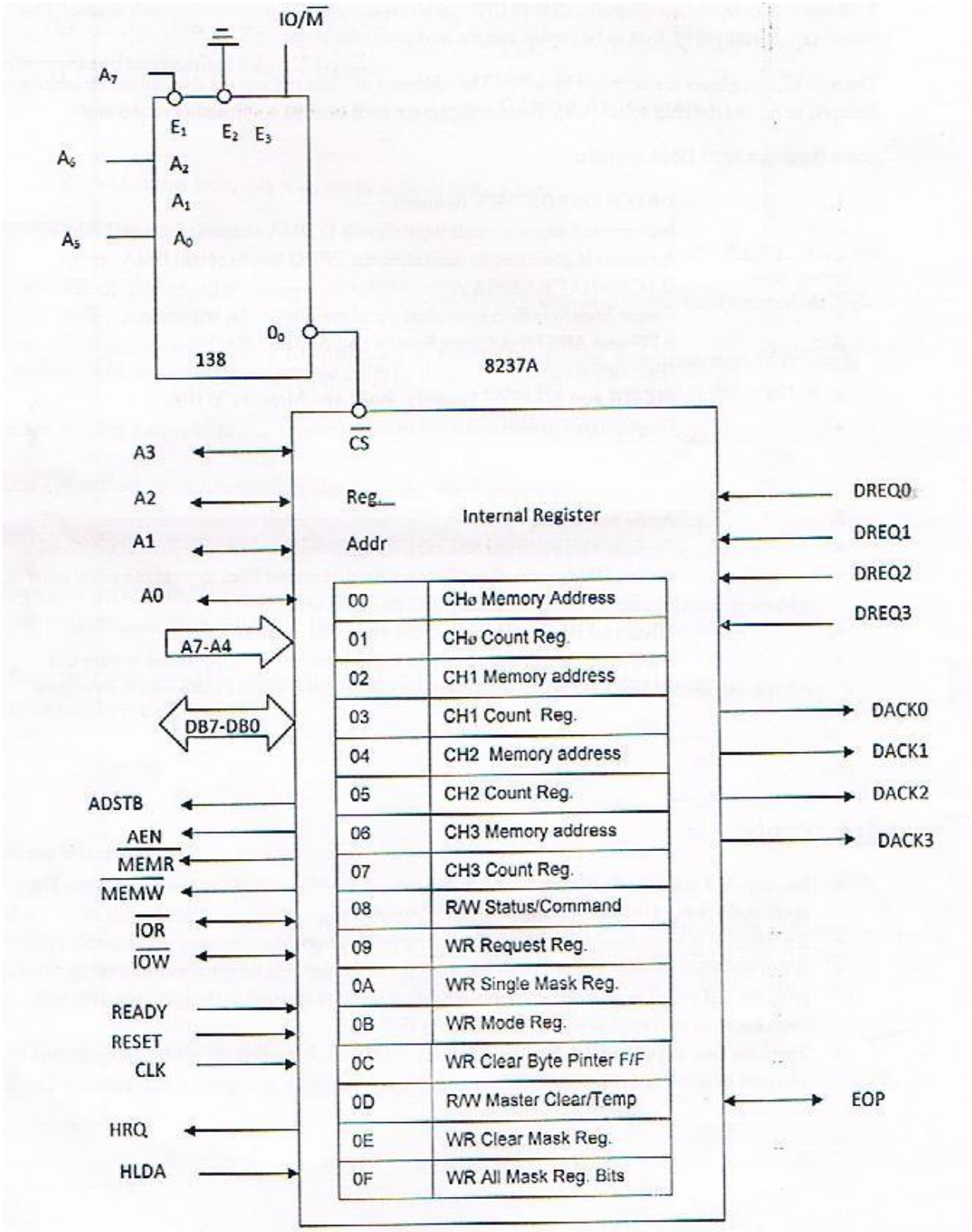
❖ **APPLICATION:**

1. Extensively used for computer-based data acquisition applications including streaming data to disk, real-time screen data display and continuous data acquisition applications.

2. DMA was used for floppy disk I/O in the original PC and for hard disk I/O in later versions.
3. PC-based DMA technology, along with high-speed bus technology, is driven by data storage, communications and graphics needs-all of which require the highest rates of data transfer between system memory and I/O devices.
4. Data acquisition applications have the same needs and therefore can take advantage of the technology developed for larger markets.

❖ 8237 DMA CONTROLLER AND INTERFACING:

1. Is a programmable DMA controller.
2. It is a 40 pin package.
3. It must interface with two types of devices-MPU and peripherals (e.g.: floppy disk, pen drive, etc.)
4. Following is the logical pin diagram for 8237 DMA controller:



8237 has four independent channels, CH_0 to CH_3. 16 bit registers are associated with each channel: One stores starting address of byte to be copied and the next store the count.

The next eight registers are accessed by MPU. The addresses of these register are determined by address lines, A_3 to A_0 and the chip select (CS). These

registers are used to write command or read status. Below are some important DMA Signals:

1. DREQ0-DREQ3-DMA Request:
   Independent, asynchronous input signals to DMA channels from peripherals
   A request is generated by activating the DREQ line to obtain DMA service
2. DACK0-DACK3-DMA Acknowledge:
   Output lines to inform individual peripheral that DMA is granted.
3. AEN and DSTB-Address Enable and Address Scribe:
   High output signals to latch a high order address bytes to generate 16-bit address
4. MEMR and MEMW Memory Read and Memory Write: Output signal is read and write from memory
5. A3-A0 and A7-A4-Address
   A3-A0 are bidirectional address lines used as input to access control registers. During DMA cycle, these lines are used as output lines to generate a low order address that combined with the remaining address lines A7-A4
6. HRQ and HLDA (Hold Request and Hold Acknowledge):
   HRQ is the output signal used to request the MPU control of the system bus. After receiving the HRQ, the MPU completes bus cycle in process and issue the HLDA signal.

Interrupt

Interrupt is a mechanism by which an I/O or an instruction can suspend the normal execution of processor and get itself serviced. In response to an interrupt, the processor stops what it is currently doing and executes a service routine. When the execution of the service routine is terminated, the original process may resume its previous operation. The interrupt is initiated by an external device and is asynchronous, meaning that it can be initiated at any time without reference to system clock. However, the response to an interrupt request is directed or controlled by the microprocessor. Interrupts are primarily issued on:

1. Initiation of I/O operation.
2. Completion of I/O operation.
3. Occurrence of hardware or software errors.

Types of interrupts:

There are three major types of interrupts that cause a break in the normal execution of a program. They can be classified as:

- External interrupts ➢ Internal interrupts
- Software interrupts.

1. External interrupts:

External interrupts are initiated via the microprocessor's interrupt pins by external devices I/O devices, timing device, circuit monitoring the power supply etc. Causes of these interrupts may be; I/O device requesting transfer of data, I/O device finished transfer of data, elapsed time of an event, or power failure. Timeout interrupt may result from a program that is an endless loop and thus exceeded its time allocation. Power failure interrupt may have as its service routine a program that transfers the complete state of the CPU into a non-destructive memory in few milliseconds before power ceases. External interrupts can be further divided into two types:

- Maskable interrupt.
- Non-maskable interrupt.

Maskable interrupt:

A maskable interrupt is one which can be enabled or disabled by executing instructions such as EI (enable interrupts) and DI (Disable interrupt). If the microprocessor's 'interrupt enable flip flop' is disabled, it ignores a maskable interrupt. In 8085, the 1 byte instruction EI sets the interrupt enable flip flop and enables the interrupt process. Similarly the 1 byte instruction DI resets the interrupt enable flip flop and disables the interrupt process. No maskable interrupts are recognized by the processor when the interrupt is disabled.

Non maskable interrupt:

This type of interrupt cannot be enabled or disabled by instructions. This types has higher priority over the maskable interrupt. This means that if both the maskable and non maskable interrupts are activated at the same time, then the processor will service the non-maskable interrupt first. In 8085 TRAP is an example of non maskable interrupt.

2. Internal interrupt:

Internal interrupt arise from illegal or erroneous use of an instruction or data. Cause of this interrupt may be: register overflow, attempt to divide by zero, an invalid operation code, stack overflow etc. These error conditions usually occur as a result of premature termination of the instruction execution. These are even termed as exceptions.

The difference between internal and external interrupt is that the internal interrupt is initiated by some exceptional conditions caused by the program itself rather than by an external events. Internal interrupts are synchronous with the program, while external interrupts are asynchronous. If the program is return, the internal interrupt will occur in the same place each time.

External interrupts depends on external conditions that are independent of the program being executed at the time.

3. **Software interrupt:**

External and internal interrupts are initiated from signal that occur in the hardware of the CPU. A software interrupt is initiated by executing an instruction. Software interrupt is a special call instruction that behaves like an interrupt rather than a subroutine call. It can be used by the programmer to initiate an interrupt procedure at any desired point in the program. The most common use of software interrupt is associated with a supervisor call instruction. This instruction provides means for switching from a CPU user mode to the supervisor mode. Certain operations in the computer may be assigned to the supervisor mode only, as for example, a complex input or output transfer procedure. In 8085 the instruction like RST0,RST1, RST2, RST3…..etc. causes a software interrupt.

Interrupt priority:

Data transfer between the CPU and an I/O device is initiated by the CPU. However, the CPU cannot start the transfer unless the device is ready to communicate with the CPU. The readiness of the device can be determined from an interrupt signal. The CPU responds to the interrupt request by storing the return address form PC into a memory stack and then the program branches to a service routine that process the required transfer. In microcomputer a number of I/O device are attached to the processor, with each device being able to originate an interrupt request. The first task of the interrupt system is to indemnify the source of the interrupt. There is also the possibility that several sources will request service simultaneously. In this case the system must also decide which device to service first. An interrupt priority is a system that established a priority over the various sources to determine which condition is to be serviced first when two or more request arrive simultaneously. The system may also determine which conditions are permitted to interrupt the computer while another interrupt is being serviced. Higher-priority interrupt levels are assigned to request which, if delayed or interrupted, could have serious consequences. Device with higher speed transfers such as magnetic disks are given high priority, and slow devices such

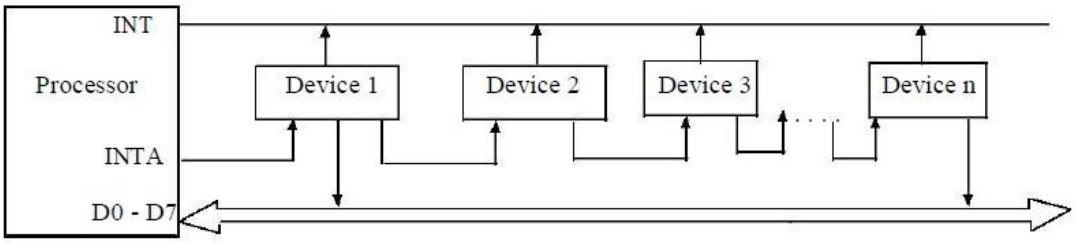
as keyboards receive low priority. When two devices interrupt the processor at the same time, the processor services the device, with the higher priority first.
There are mainly two ways of servicing multiple interrupts. These are:

- Polled interrupt.
- Chained (Vectored) interrupt.

Polled interrupt:

Polled interrupt are handled using software and are therefore slower compared to vectored (hardware) interrupts. In this method there is one common branch address for all interrupts. The program that takes care of interrupts begins at the branch address and polls the interrupts sources in sequence. The order in which they are tested determines the priority of each interrupt. The highest priority source is tested first, and if its interrupt signal is on, control branches to a service routine for this source. Otherwise the next lower priority source is tested, and so on. Thus, the initial service routine for all interrupts consists of a program that test the interrupt sources in sequence and branches to one of many possible service routines.
Polled interrupts are very simple. But or large number of devices, the time required to poll each device may exceed the service to the device. In such case, the faster mechanism called chained interrupt is used.

Chained interrupt:
This is hardware concept of handling the multiple interrupts. In this technique, the devices are connected in a chain fashion as shown in figure below for setting up the priority system.



Here the device with the highest priority placed in the first position, followed by lower priority devices. Suppose that one or more devices interrupt the processor at a time. In response, the processor saves its current status and then generates an interrupt acknowledge (INTA) signal to the highest priority device, which is device 1 in our case. If this device has generated the interrupt

it will accept the INTA signal from the processor; otherwise, it will pass INTA on to the next device until the INTA is accepted by the interrupting device. Once accepted, the device provides a means to the processor for finding the interrupt address vector using external hardware. Usually the requesting device responds by placing a word on the data lines. With the help of hardware it generates interrupts vector address. This word is referred to as vector, which the processor used as a pointer to the appropriate device service routine.

This avoids the need to execute a general interrupt service routine first. So this technique is also referred to as vectored interrupts.

Interrupts of 8085

The 8085 has five hardware interrupts:

- TRAP
- RST 7.5
- RST 6.5
- RST 5.5
- INTR

The four interrupts TRAP, RST 7.5, 6.5, 5.5 are automatically vectored (transferred) to specific locations on without any external hardware. They do not require INTA signal or an input port; the necessary hardware is already implemented inside the 8085. These interrupts and their call locations are:

INTERRUPT	CALL LOCATIONS
TRAP	0024 H
RST 7.5	003 CH
RST 6.5	0034 H
RST 5.5	002C H.

TRAP:

It is a non-maskable interrupt, having the highest priority among all interrupts. By default, it is enabled until it gets acknowledged. In case of failure, it executes as ISR and sends the data to backup memory. This interrupt transfers the control to the location 0024H.

RST7.5:

It is a maskable interrupt, having the second highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 003CH address.

RST 6.5:

It is a maskable interrupt, having the third highest priority among all interrupts. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 0034H address.

RST 5.5:

It is a maskable interrupt. When this interrupt is executed, the processor saves the content of the PC register into the stack and branches to 002CH address.

INTR:

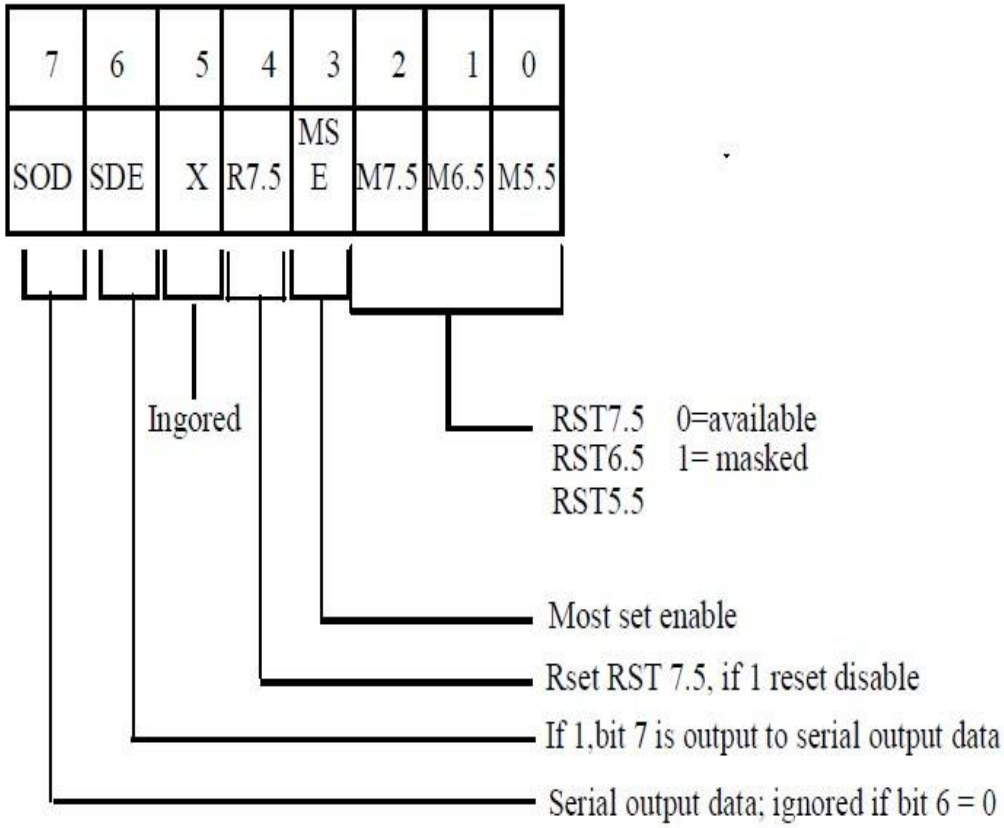
It is a maskable interrupt, having the lowest priority among all interrupts. It can be disabled by resetting the microprocessor.

RST Instructions

The signal INTA is used to insert a Restart (RST) instruction, (it saves the memory address of the next instruction to the stack. The program is transferred to the call location.). The RST instruction and their call locations are:

- Bit D_3 is a control bit and should be 1 for bits D_0 , D_1, and D_2 to be effective.
- Logic 0 on D_0, D_1, and D_2 will enable the corresponding interrupts and logic 1 will disable the interrupts.
- The second function is to reset RST 7.5 flip flop. Bit D_4 is additional control for RST 7.5 ❖ If D_4 =1, RST 7.5 is reset. This is used to ignore RST 7.5 without servicing it.
- The third function is to implement serial I/O. Bit D_7 and D_6 are used for serial I/O and do not effect the interrupts.

Instruction	Hex-code	Call location
RST 0	C7	0000
RST 1	CF	0008
RST 2	D7	0010
RST 3	DF	0018
RST 4	E7	0020
RST 5	EF	0028
RST 6	F7	0030
RST 7	FF	0038



The 8259 Programmable Interrupt Controller

The 8259A programmable interrupt controller designed to work with Intel microprocessors 8085, 8086 and 8088. The 8259A interrupt controller can

1. Manage eight interrupts according to the instructions written into its control registers. This is equivalent to proving eight interrupt pins on the processor in place of one INTR (8085) pin.

2. Vector can interrupt request anywhere in the memory map. However, all eight interrupts are spaced at the interval of either four or eight locations. This eliminates all the major drawback of the 8085 interrupts in which all interrupts are vectored to memory locations on page 00H.

3. Resolve eight levels of interrupt priorities in a variety of modes, such as fully nested mode, automatic rotation mode, and specific rotation mode.

4. Mask each interrupt request individually.

5. Read the status of pending interrupts, in-service interrupts, and masked interrupts.

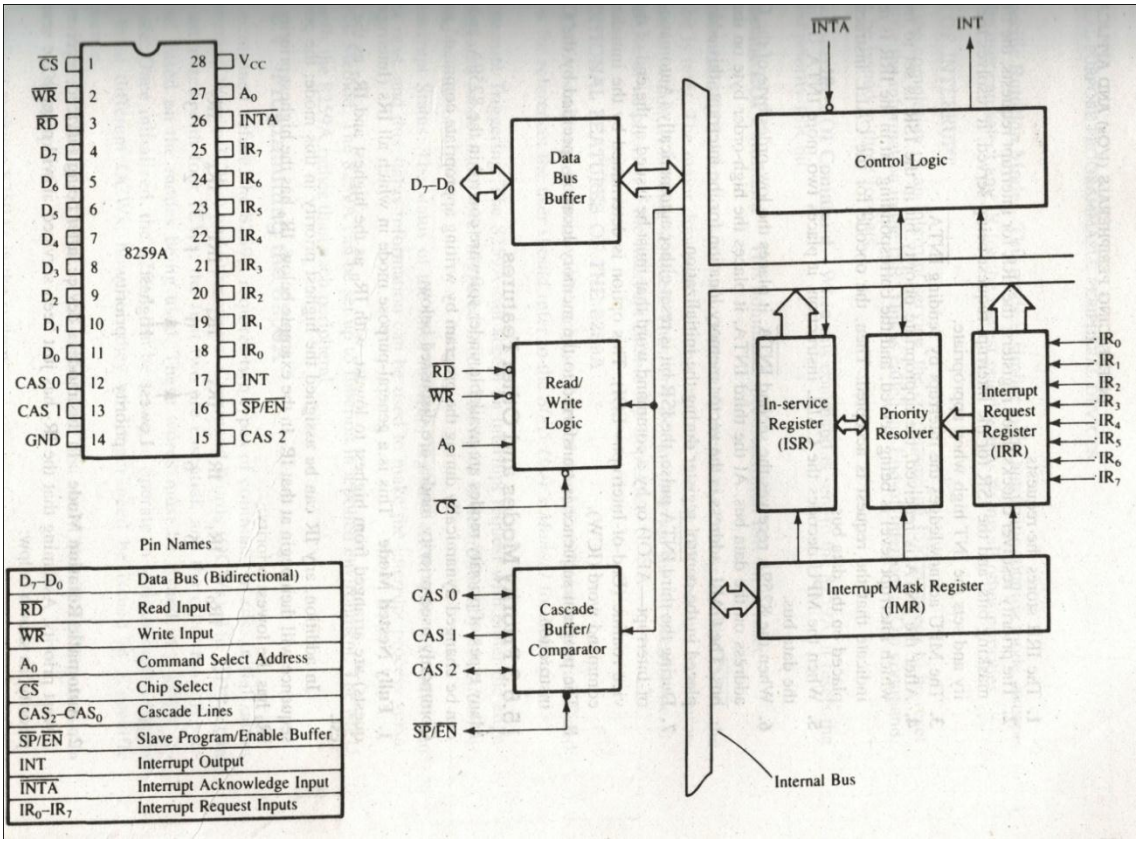
6. Be set up to accept either the level-triggered or the edge-triggered interrupt request

7. Be expanded to 64 priority levels by cascading additional 8259As.

8. Be set up to work with either the 8085 microprocessor mode or the 886/8088 microprocessor mode.

The 8259A is upward-compatible with its predecessor, the 8259. The main difference between the two is that the 8259A can be used with Intel's 8086/88 16-bit microprocessor. It also includes additional features such as the level-triggered mode, buffered mode, and automatic-end-of interrupt mode. To simplify the explanation of the 8259A, illustrative examples will not include the cascade mode or the 8086/88 mode and will be limited to modes continuously used with the 8085.

BLOCK DIAGRAM OF THE 8259A



The figure shows the internal block diagram of the 8259A. It includes eight blocks: control logic, Read/Write logic, data bus buffer, three registers (IRR, ISR and IMR), priority resolver, and cascade buffer. This diagram shows all the elements of a programmable device, plus additional blocks. The functions of some of these blocks need explanation, which is given below:

READ/WRITE LOGIC

This is a typical Read/Write control logic. When the address line A_0 is at logic 0, the controller is selected to write a command or read a status. The Chip Select logic and A_0 determine the port address of the controller.

CONTROL LOGIC

This block has two pints: INT (Interrupt) as an output, and INTA (Interrupt Acknowledge) as an input. The INT is connected to the interrupt pin of the MPU. Whenever a valid interrupt is asserted, this signal goes high. The INTA in the Interrupt Acknowledge signal from the MPU.

INTERRUPT REGISTERS AND PRIORITY RESOLVER

The interrupt Request Register (IRR) has eight input lines (IR_0-IR_7) for the interrupts. When these lines go high, the requests are stored in the register. The In-Service Register (ISR) stores all the levels that are currently being serviced, and the Interrupt Mask Register (IMR) stored the masking bits of the interrupt lines to be masked. The Priority Resolver (PR) examines these three registers and determines whether INT should be sent to the MPU.

CASCADE BUFFER/COMPARATOR

This block is used to expand the number of interrupt levels by cascading two or more 8259As. To simplify this discussion, this block will not be mentioned again.

Interrupt Operation

To implement interrupts, the Interrupt Enable flip-flop in the microprocessor should be enabled by writing the EI instruction, and the

Pin Configuration

Block Diagram

8259A should be initialized by writing control words in the control register. The 8259A requires two types of control words: Initialization Command Words (ICWs) and Operational Command Words (OCWs). The ICWs are used to set up the proper conditions and specify RST vector addresses. The OCWs are used to perform functions such as masking interrupts, setting up status-read operations, etc. After the 8259A is initialized, the following sequence of events occurs when one or more interrupt request lines go high:

1. The IRR stores the requests.
2. The priority resolver checks three registers: the IRR for interrupt requests, the IMR for masking bits, and the ISR for the interrupt request being served. It resolves the priority and sets the INT high when appropriate.
3. The MPU acknowledges the interrupt by sending $\overline{\text{INTA}}$.
4. After the $\overline{\text{INTA}}$ is received, the appropriate priority bit in the ISR is set to indicate which interrupt level is being served, and the corresponding bit in the IRR is reset to indicate that the request is accepted. Then, the opcode for the CALL instruction is placed on the data bus.
5. When the MPU decodes the CALL instruction, it places two or more $\overline{\text{INTA}}$ signals on the data bus.
6. When 8259A receives the second $\overline{\text{INTA}}$, it places the low-order byte of the CALL address on the data bus. At the third $\overline{\text{INTA}}$, it pla~~ces t~~he

highorder byte on the data bus. The CALL address is the vector memory location for the interrupt: this address is placed in the control register during the initialization.

7. During the third INTA pulse, the ISR bit is reset either automatically (Automatic-End-of-Interrupt—AEOI) or by a command word that must be issued at the end of the service routine (End-of-Interrupt—EOI). This option is determined by the initialization command word (ICW).

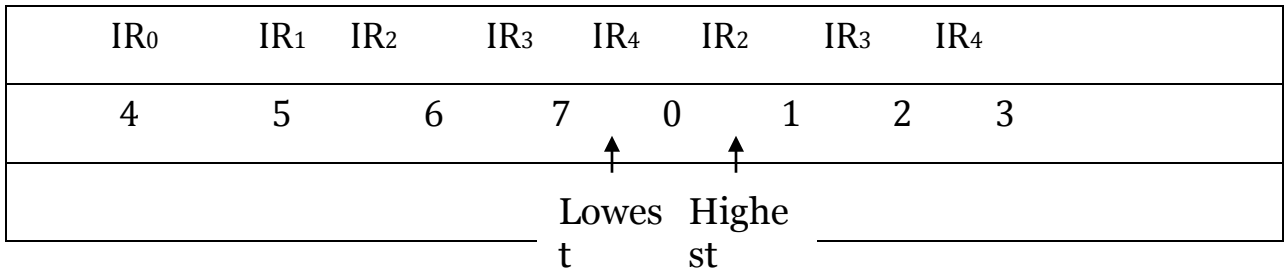
8. The program sequence is transferred to the memory location specified by the CALL instruction.

Priority Modes and Other Features

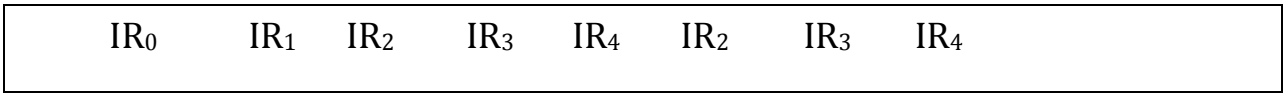
Many types of priority modes are available under software control in the 8259A, and they can be changed dynamically during the program by writing appropriate command words. Commonly used priority modes are discussed below:

1. Fully Nested Mode: This is a general-purpose mode in which all IRS (interrupt Requests) are arranged from highest to lowest, with IR_0 as the highest and IR_7 as the lowest.

In addition, any IR can be assigned the highest priority in this mode; the priority sequence will then begin at that IR. In the example below, IR_4 has the highest priority, and IR_3 has the lowest priority:



2. Automatic Rotation Mode: In this mode, a device, after being serviced, receives the lowest priority. Assuming that the IR_2 has just been serviced, it will receive the seventh priority, as shown below:



1	6	7	0	1	2	3	4

3. Specific Rotation Mode: This mode is similar to the automatic rotation mode, except that the user can select any IR for the lowest priority, thus fixing all other priorities.

END OF INTERRUPT

After the completion of an interrupt service, the corresponding ISR bit needs to be reset to update the information in the ISR. This is called the End-ofInterrupt (EOI) command. It can be issued in three formats.

1. Nonspecific EOI Command When this command is sent to 8259A, it resets the highest priority ISR bit.

2. Specific EOI Command This command specifies which ISR bit to reset.

3. Automatic EOI In this mode, no command is necessary. During the third INTA, the ISR bit is reset. The major drawback with this mode is that the ISR does not have information on which IR is being serviced. Thus, any IR can interrupt the service routine irrespective of its priority, if the Interrupt Enable flip-flop is set.

ADDITIONAL FEATURES OF THE 8259A

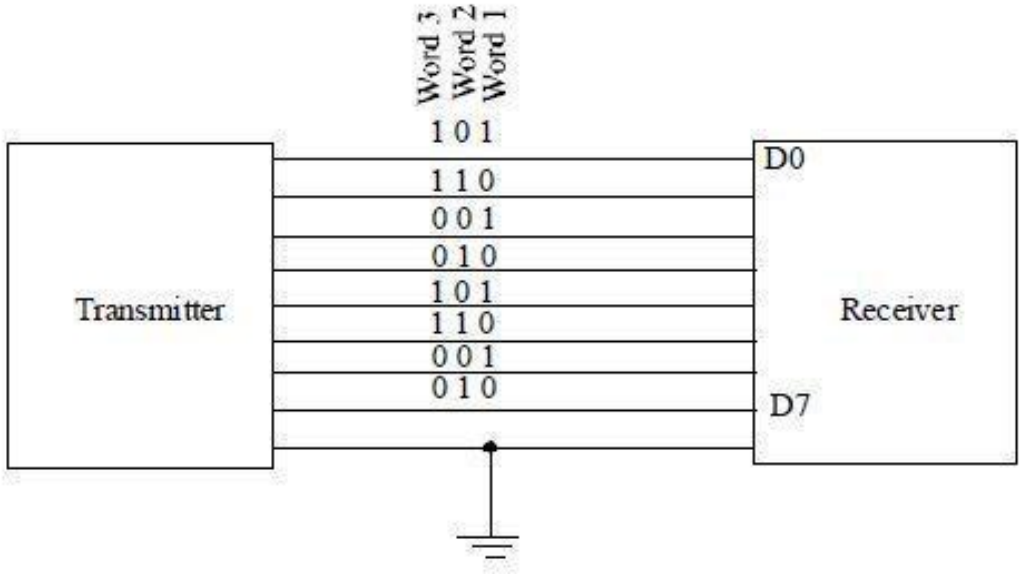
The 8259A is a complex device with various modes of operation. These modes are listed below for reference; the user should refer to the 8085 User's manual for details.

- Interrupt Triggering: The 8259A can accept an interrupt request with either the edge triggered mode or the level-triggered mode. This mode is determined by the initialization instructions.
- Interrupt Status: The status of the three instruction registers (IRR, ISR and IMR) can be read, and this status information can be used to make the interrupt process versatile.
- Poll Method: The 8259A can be set up to function in a polled environment. The MPU polls the 8259A rather than each peripheral.

CHAPTER 6: I/O INTERFACE

Parallel Communication

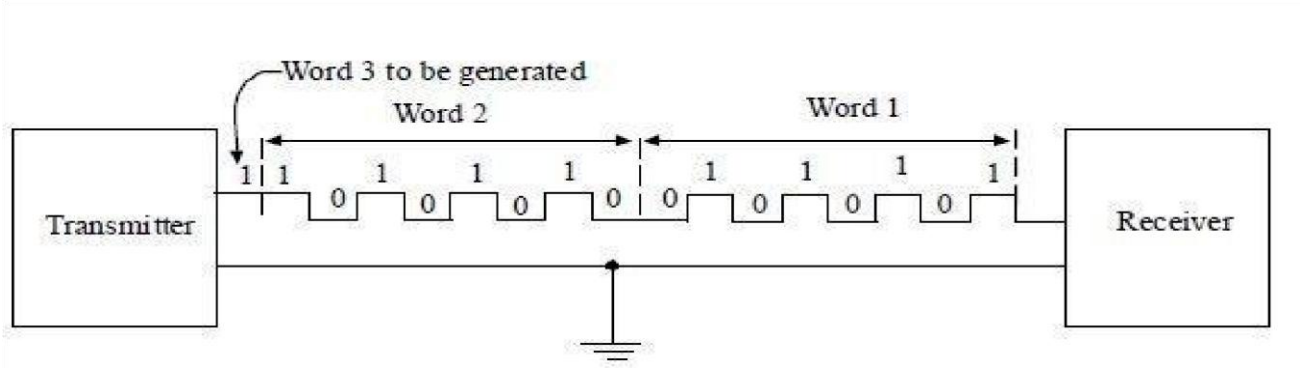
Parallel Data Transfer: When a word of n bits is to be transmitted in parallel each bit is transmitted on a separate line along with a common ground line with respect to which the status of each line is measured. Thus, a channel comprises of (n+1) lines.



Here, the time required to transfer one word is equal to the time taken to transmit a bit. Parallel data transmission is impractical over long distances because of prohibitive cost of installing a large number of lines.

Serial Communication

In serial data transfer, each bit of the word is sent in succession, one at a time over a single pair of wires. A parallel to serial converters is used to convert the incoming parallel data to serial form and then the data is sent out with the least significant bit Do first and most significant bit D7 coming last of all. If the bit rate is retained after the parallel to serial conversion, the time taken to transmit a word in serial data transmission will be n times more than the time taken in parallel data transmission. If the word in the above example were to sent serially, the data on the channel will appear as in figure below.



8251 UNIVERSAL SYNCHRONOUS ASYNCHRONOUS RECEIVER TRANSMITTER:

The 8251 is a USART (Universal Synchronous Asynchronous Receiver Transmitter) for serial data communication. As a peripheral device of a microcomputer system, the 8251 receives parallel data from the CPU and transmits serial data after conversion. This device also receives serial data from the outside and transmits parallel data to the CPU after conversion.

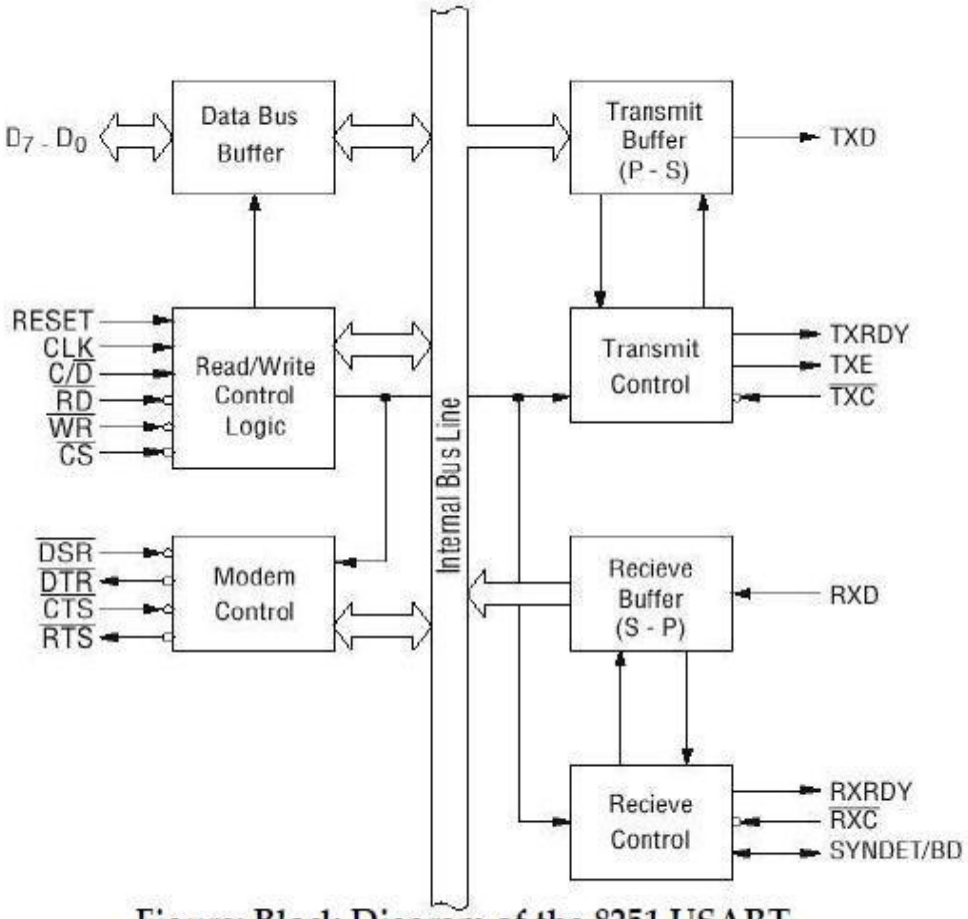


Figure: Block Diagram of the 8251 USART

The 8251 functional configuration is programmed by software. Operation between the 8251 and a CPU is executed by program control. Table 1 shows the operation between a CPU and the device.

$\overline{CS}$	C/$\overline{D}$	$\overline{RD}$	$\overline{WR}$	
1	×	×	×	Data Bus 3-State
0	×	1	1	Data Bus 3-State
0	1	0	1	Status → CPU
0	1	1	0	Control Word ← CPU
0	0	0	1	Data → CPU
0	0	1	0	Data ← CPU

CONTROL WORDS

1. MODE INSTRUCTION

Mode instruction is used for setting the function of the 8251. Mode instruction will be in "wait for write" at either internal reset or external reset. That is, the writing of a control word after resetting will be recognized as a "mode instruction." Items set by m ode instruction are as follows: • Synchronous/ asynchronous mode • Stop bit length (asynchronous m ode)

• Character length

• Parity bit

• Baud rate factor (asynchronous mode)

• Internal/ external synchronization (synchronous mode)

• Number of synchronous characters (Synchronous mode)

The bit configuration of m ode instruction is shown in Figures 2 and 3. I n the case of synchronous m ode, it is necessary to write one- or two byte sync characters. If sync characters were written, a function will be set because the writing of sync characters constitutes part of m ode instruction.

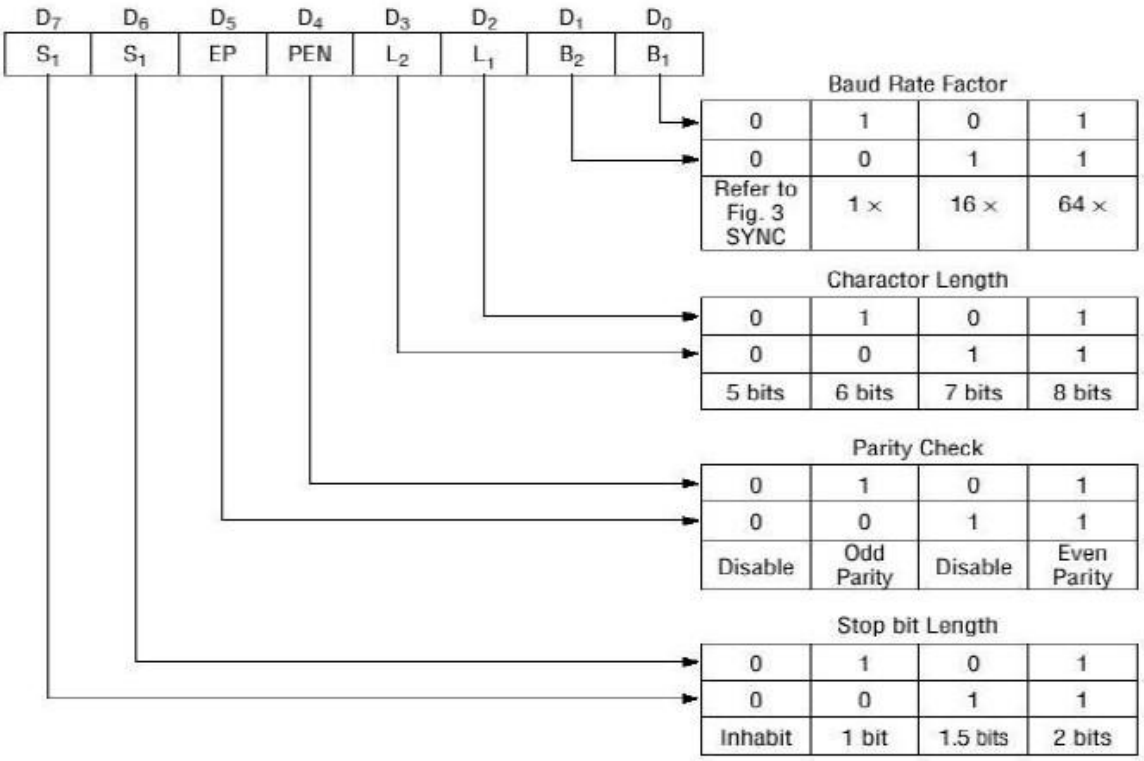


Fig. 2 Bit Configuration of Mode Instruction (Asynchronous)

2. COMMAND WORD:

Command is used for setting the operation of the 8251. It is possible to write a command whenever necessary after writing a mode instruction and sync characters. I tem s to be set by com m and are as follows:

- Transmit Enable/ Disable
- Receive Enable/ Disable
- DTR, RTS Output of data
- Resetting of error flag
- Sending to break characters
- Internal resetting
- Hunt mode (synchronous mode)

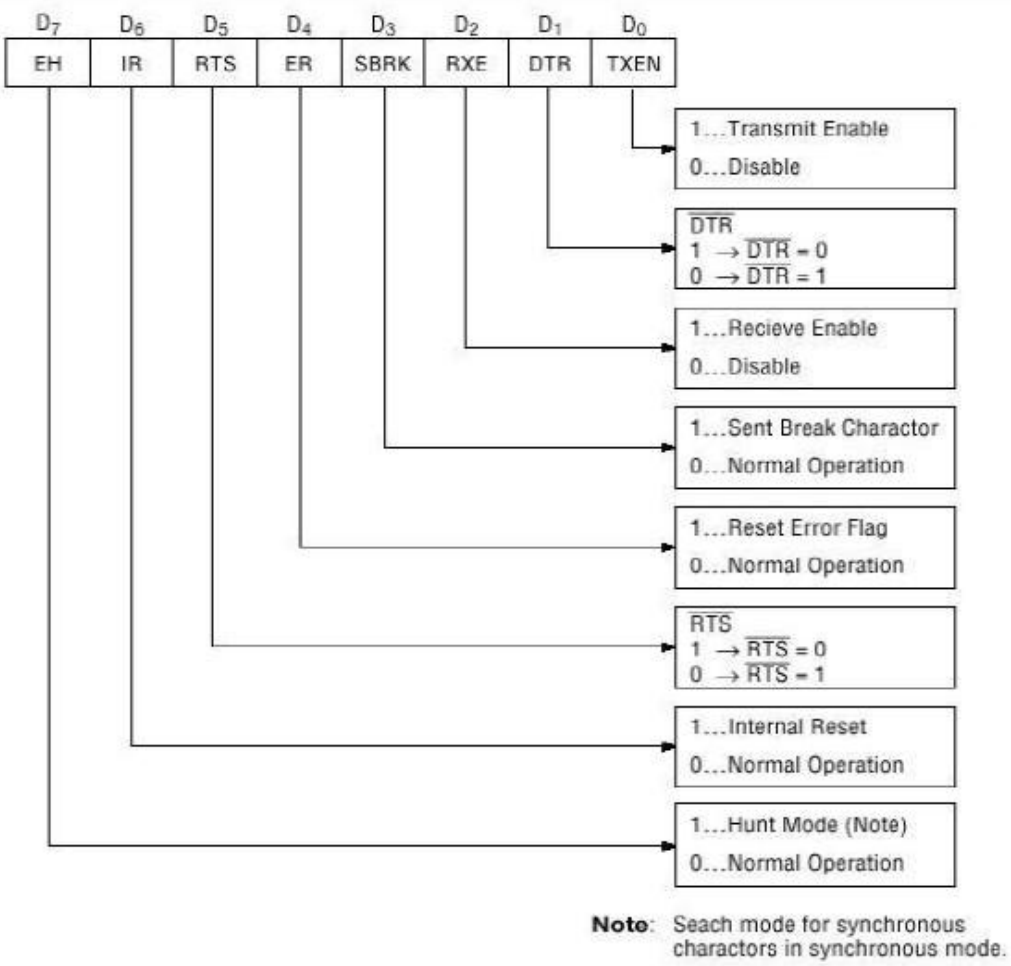


Fig. 4 Bit Configuration of Command

3. STATUS W ORD:

It is possible to see the internal status of the 8251 by reading a status word. The bit configuration of status word is shown in Fig 5.

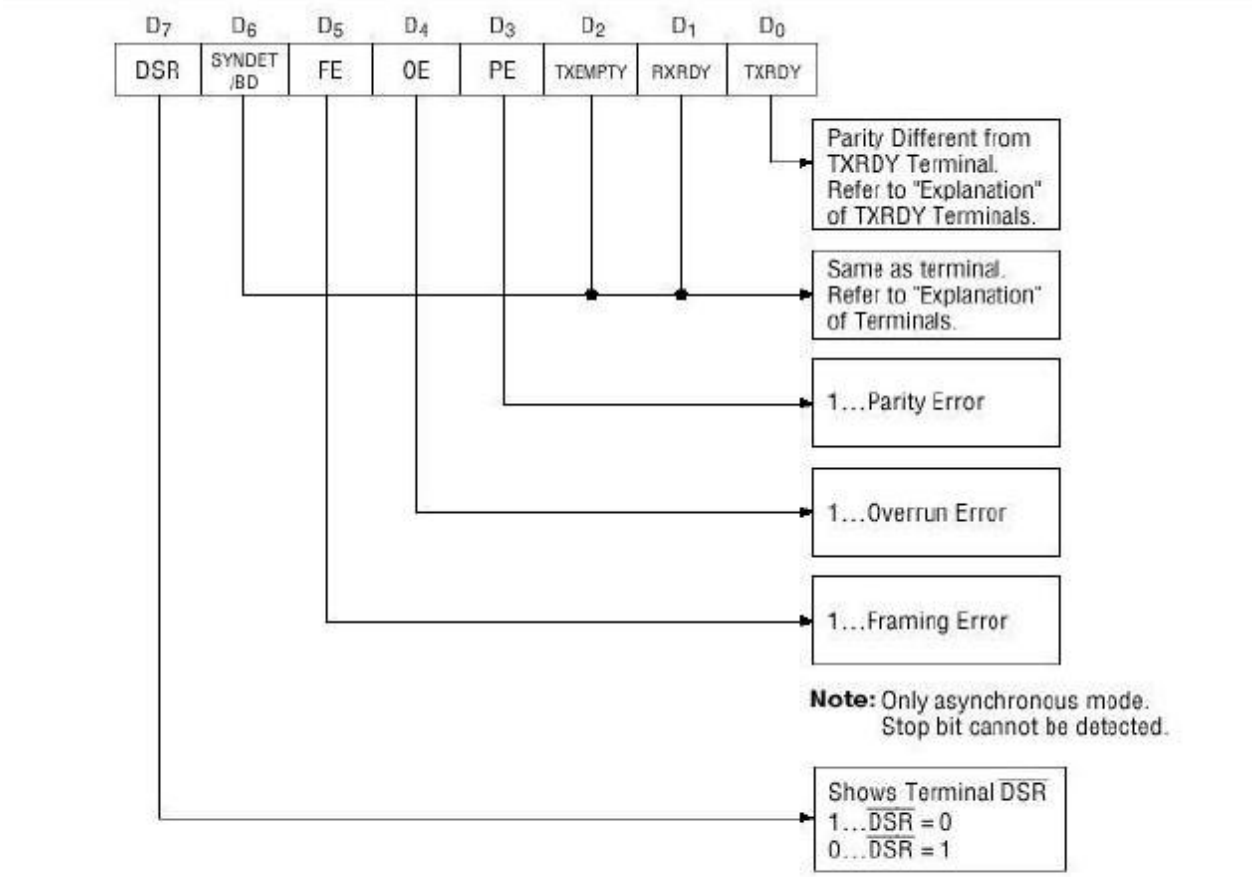


Fig. 5 Bit Configuration of Status Word

Basic Concept of Synchronous and Asynchronous Modes:

1. SYNCHRONOUS SERIAL DATA COMMUNICATION :

- A more efficient method of transferring serial data is to synchronize the transmitter and the receiver and then send a large block of data characters one after the other with no time between characters.

- No start or stop bits are needed with individual data characters because the receiver automatically knows that every 8 bits received after synchronization represents a data character.

- To indicate start of transmission, transmitter sends out one or more unique characters called sync. Characters.

- The receiver uses the sync characters or the flag to synchronize its internal clock with that of the receiver.

- ISDN, High Speed Modems and Digital Communication Channel use synchronous transmission.

2. ASYNCHRONOUS SERAIL DATA COMMUNICATION

- For asynchronous transmission, each data character has a bit which identifies its start and 1 or 2 bits which identify end.

- Since each character is individually identified, character can be sent at any time asynchronously.

- The beginning of a data character is identified by the line going low for 1 bit time. This bit is called start bit.

- The data bits are then sent out on the line one after the other.

- Note that LSB is transmitted first.

    - After parity bit, the signal line is returned high for at least 1 bit time to identify the end of the character. This high bit is always referred as stop bit.

UART: Universal Asynchronous Receiver Transmitter [IN 8250]
USART: Universal Synchronous Asynchronous Receiver Transmitter [INTEL 8251A]

Methods of Parallel Data Transfer

1. SIMPLE I/O

 When you need to get digital data from a simple switch, all you have to do is connect the switch to an I/P port line and read the value.

 Likewise, when you need to O/P data to simple L ED, all you have to do is connect the LED to an O/P port and send the value.

 The LED is always ready, so you can send data at any time.

2. STROBE I/O

 In many applications, valid data is present on an external device only at a certain time, so it must be read in at that time.

 When a key is pressed, circuitry on the keyboard sends out the ASCII code for the pressed key on eight parallel data line, and then sends out a strobe signal on another line to indicate that valid data is present on the eight data lines.

 For higher speed data transfer this method does not work.

 The sending system might send data bytes faster than the receiving system could read them. To prevent this handshake data transfer is required.

3. SINGLE HANDSHAKE I/O DATA TRANSFER

- The peripheral outputs some parallel data and sends STB signal to MPU.
- The MPU detects STB signal on a polled or interrupt basis and reads data byte.
- Then the MP sends on ACK signal to the peripheral to indicate that the data has been read and the peripheral can send to next byte of data.

4. DOUBLE HANDSHAKE I/O DATA TRANSFER

 For data transfers where coordination is required between sending syste m and the receiving system, a double handshake is used.

 The sending device asserts its STB low to ask , **Are you ready?"**

 The receiving system raises its ACK line high to say , **I'm ready".**

 The peripheral device then sends the data byte and raises i ts STB signal high to say **"Here is valid data for you".**

 When the receiving system finishes to read the data, receiving system drop its ACK line low to say **"I have data than you and I await your request to send the next byte of data".**

8255A Programmable Peripheral Interface

INTRODUCTION

The 8255A PPI provides three 8 bit input/output ports in one 40 pin package. The chip can be interfaced directly to the data bus of the processor allowing its function to be programmed. In one application, a port may appear as an output, but in another by reprogramming it an be as input.

DESCRIPTION

 The 8255A is a general purpose parallel I/O interfacing device.

 It provides 24 I/O lines organized as three 8 bit input/output ports labeled A,B and C.

 Each of the ports A or B can be programmed as an 8 bit input or output port.

 Port C can be divided in half, with the topmost or bottommost four bits programmed as inputs or outputs.

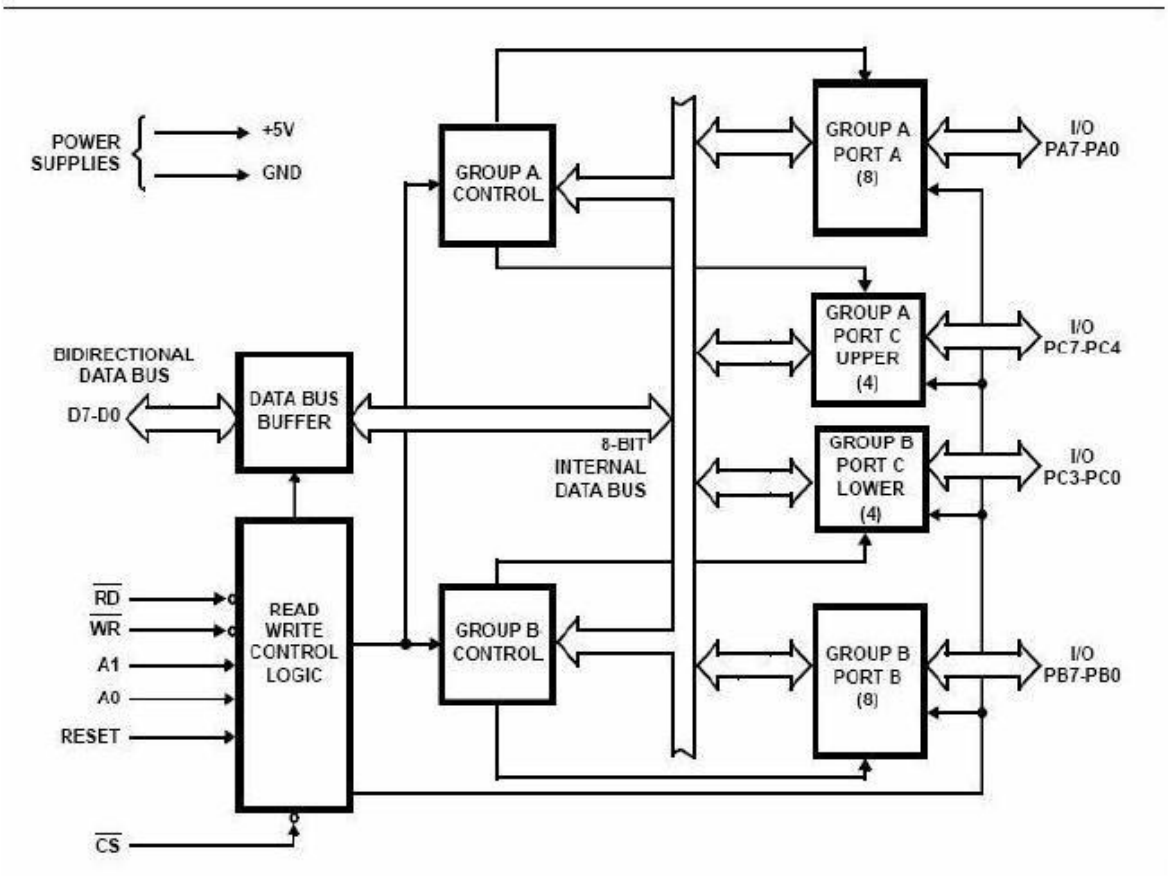
 Port C can be used as an 8 bit input or output port as two 4 bit ports or to produce handshake signals for Port A and B.

 The 8255A is a very versatile device.

 It can be programmed to look like

- Three simple I/O ports (called MODE 0)
- Two handshaking I/O ports (called MODE 1)
- Port A as a bidirectional I/O port with 5 handshaking signals (called MODE 2) The modes can also be intermixed.
- For example, Port A can be programmed to operate in MODE 2 while Port B in MODE 0.

INTERNAL BLOCK DIAGRAM OF 8255A



- The address inputs A0, A1 allow you to selectively access one of the three ports or the control register.

- The internal addresses for the devices are tabulated below

DESCRIPTION	A1	A0
PORT A	0	0
PORT B	0	1
PORT C	1	0
CONTROL	1	1

- The CS input of 8255A enables it for reading or writing

- This input is driven by an address decoder.

- The RD and WR input pins determine the direction of data flow over the chips 8 bit bidirectional data bus.

- The RESET input of 8255A is connected to the system reset line so that, when the system is reset all the port lines are initialized as input lines. This is done to prevent destruction of circuitry connected to port lines.

OPERATIONAL MODES

1. MODE 0

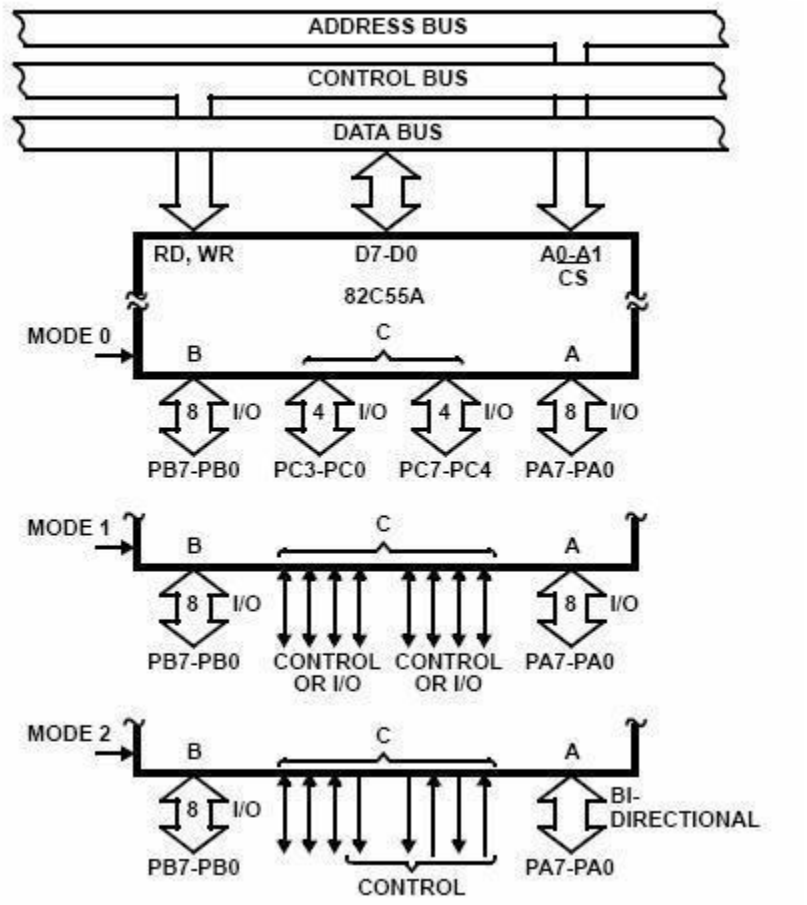
- When programmed for MODE 0, the PPI offers three simple I/O ports with no handshaking signals.
- This mode is appropriate for I/O devices that do not need special synchronizing signals to exchange data with the processor.
- A common example is a keyboard used for data entry.
- When used as O/Ps, the PORT c lines can be individually set or reset by sending a special control word to control register address.
- Two halves of PORT C are independent so one half can be initialized as input and the other half as output.

2. MODE 1

- When programmed for MODE 1, the PPI offers PORT A or PORT B for a handshake input/output operation.
- Pins PC0,PC1 and PC2 function as handshake lines for PORT B
- Pins PC3,PC4 and PC5 function as handshake signal for PORT A (input).
- Pins PC6 and PC7 are available for use as input/output lines for PORT A
- If PORT A is initialized as handshake O/P port, then PORT C pins.
- PC3, PC6 and PC7 function as handshake signals.
- PORT C pins PC4 and PC5 are used as input/output lines for PORT A

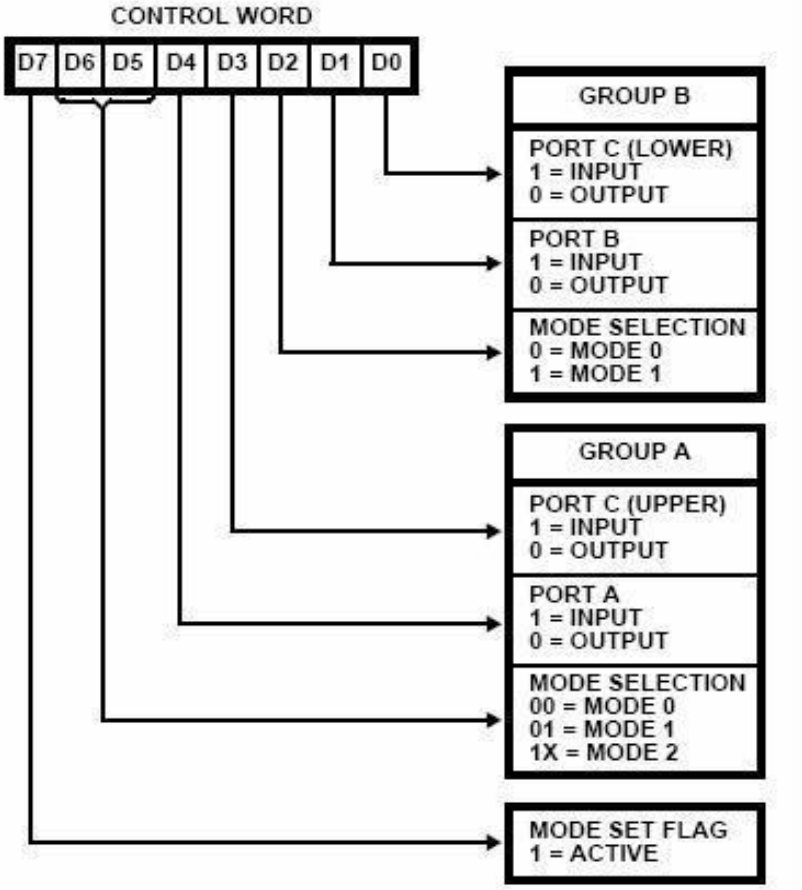
3. MODE 2

- Only PORT A can be initialized in MODE 2.
- In MODE 2, PORT A can be used as bidirectional handshake data transfer.
- This means that data can be outputted/inputted from same eight lines.
- If PORT A is initialized in MODE 2, then pins PC3 through PC7 are used as handshake lines for PORT A.
- The other three pins PC0 through PC2 can be used for I/O port if PORT B is in MODE 0.
- The same 3 pins will be used for PORT B handshake signals if PORT B is initialized in MODE 1.

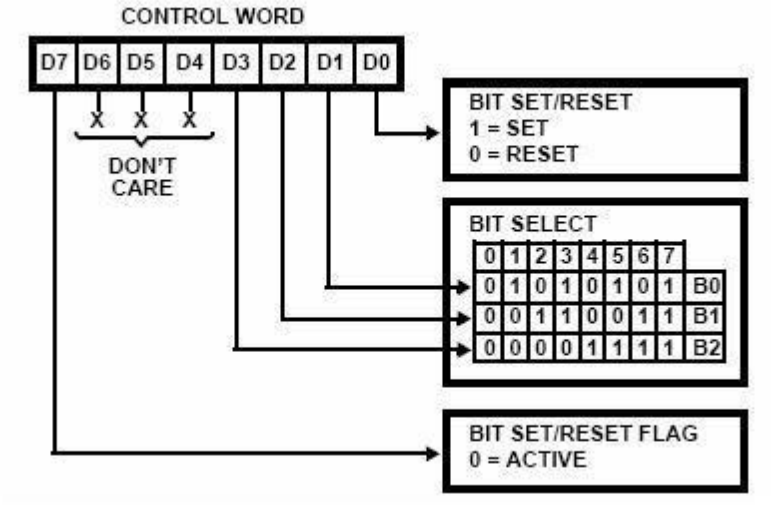


CONTROL WORDS:

MODE SET CONTROL WORD



PORT C BIT SET/RESET CONTROL WORD



RS-232:

This interface standard is most widely used standard for serial communication between microcomputers and peripheral devices. The interface, defined by EIA, relates essentially to two types of equipment. The first is known as data terminal equipment. While the second is referred as data communication equipment (DTE). The data terminal equipment (e.g. a microcomputer) is capable of sending and/or receiving data via the serial interface. The data communication equipment on the other hand is generally through of as a device which can facilitate serial data communications (e.g. modems). ○ RS 232C works in a negative logic. The standard specifies that 'the logic one' level is a voltage between -3 and -15 v and 'the logic zero' is a voltage between +3 and +15 V. The commonly used voltages are +12v and -12V.

- The transmission line normally used a twisted pair of shielded wire with a line capacitance of more than 1200PF and no less than 300Pf. The standard specifies the line length to 50 feets only.
- The standard describes the function of 25 signal and handshake pins for serial data transfer. It also specifies that the DTE connector should be male and the DCE connector should be female. Usually 9 pin and 25 pin connectors are available.

Fig :9 pin R-232 & Fig: 25 pin R-232

Among the 25 pins of RS232C the independent pin numbers are:

Pin no. Signals Functions

2 →Transmit data, TXD Output, transmit data from DTE to DCE

3→ Receive data RXD Input, DTE receive data from DCE

4 →Request to send, RTS General Purpose output from DTE

5 →Clear to send, CTS Input to DTE, used as handshake

6 →Data set ready, DSR Input to DTE, indicate that DCE is ready

7 →Signal ground, GND Common reference bet. DTE and DCE 8 →Data carrier detect, DCD Used by DTE to disable data reception 20 →Data terminal ready, DTR Output means DTE is ready.

The main problem with –RS-232C is that it can only transfer data reliably about 50ft (16.4m) at its maximum rate of 20k band. If longer lines are used the transmission rate has to be drastically reduced. For higher rate of transfers and for longer distance we have another standard defined.

RS 232C interface with DTE and DCE:

The figure below shows a interfacing with minimum lines.

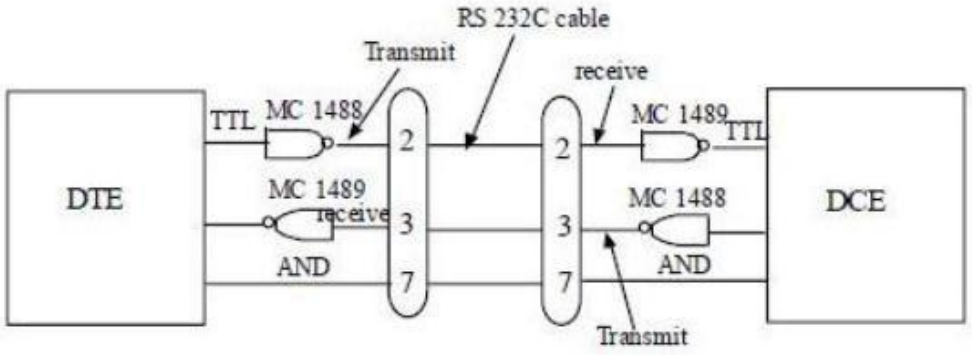


Fig. RS 232C interface

The signaling in RS-232C is not compatible with the TTL logic level. For TTL 0 v to 0.2V is considered a logic 0 and 3.4 v to 5v as logic 1. But RS-232C works in a negative logic -3 to -15v considered as logic 1 and +3 to +15v as logic 0. Because of this incompatibility of the data lines with the TTL logic, voltage translators called line drivers and line receivers are required to interface TTL logic with RS-232C signals. The line driver MC 1488 converts logic 1 into approx. -9V and logic 0 into +9v. Before it is received by the DCE it is again converted by the line receiver MC 1489 into TTL-Compatible logic.

The minimum interface required both a computer and a peripheral device requires three lines; pin 2, 3 and 7. These lines are defined in relation to the DTE; the terminal transmits on pin 2 and receives as pin 3. On the other hand the DCE transmits on pin 3 and receives on pin 1. Pin 7 is ground pin.

CHAPTER 7- ADVANCED MICRPROCESSORS

7.1 80286 Microprocessor

Salient Features of 80286

- The 80286 is the first member of the family of advanced microprocessors with memory management and protection abilities. The 80286 CPU, with its 24-bit address bus is able to address 16 Mbytes of physical memory. Various versions of 80286 are available that runs on 12.5 MHz, 10 MHz and 8 MHz clock frequencies. 80286 is upwardly compatible with 8086 in terms of instruction set.
- 80286 has two operating modes namely real address mode and virtual address mode. In real address mode, the 80286 can address up to 1Mb of physical memory address like 8086. In virtual address mode, it can address up to 16 Mb of physical memory address space and 1 GB of virtual memory address space.
- The instruction set of 80286 includes the instructions of 8086 and 80186. 80286 has some extra instructions to support operating system and memory management. In real address mode, the 80286 is object code compatible with 8086. In protected virtual address mode, it is source code compatible with 8086. The performance of 80286 is five times faster than the standard 8086.
- **Need for Memory Management:** The part of main memory in which the operating system and other system programs are stored is not accessible to the users. It is required to ensure the smooth execution of the running process and also to ensure their protection. The memory management which is an important task of the operating system is supported by a hardware unit called memory management unit.

- **Swapping in of the Program:** Fetching of the application program from the secondary memory and placing it in the physical memory for execution by the CPU.

- **Swapping out of the executable Program:** Saving a portion of the program or important results required for further execution back to the secondary memory to make the program memory free for further execution of another required portion of the program.

- **Concept of Virtual Memory:** Large application programs requiring memory much more than the physically available 16 Mbytes of memory, may be executed by diving it into smaller segments. Thus for the user, there exists a very large logical memory space which is not actually available. Thus there exists a virtual memory which does not exist physically in a system. This complete process of virtual memory management is taken care of by the 80286 CPU and the supporting operating system.

Register Organization of 80286

The 80286 CPU contains almost the same set of registers, as in 8086, namely

1. Four 16-bit general purpose registers
2. Four 16-bit segment registers
3. Status and control registers
4. Instruction Pointer

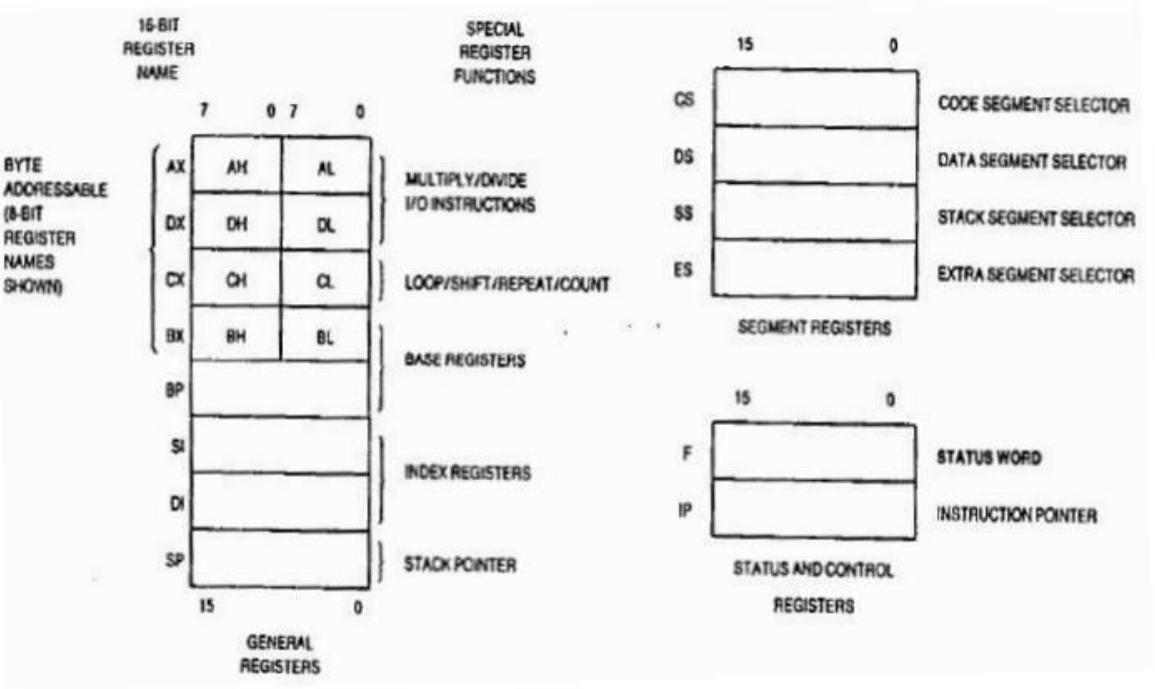


Fig.2.17 Register set of 80286

The flag register reflects the results of logical and arithmetic instructions.

-	NT	IOPL	OF	DF	IF	TF	SF	ZF	-	AF	-	PF	-	CF

Fig 2.18 Flag registers

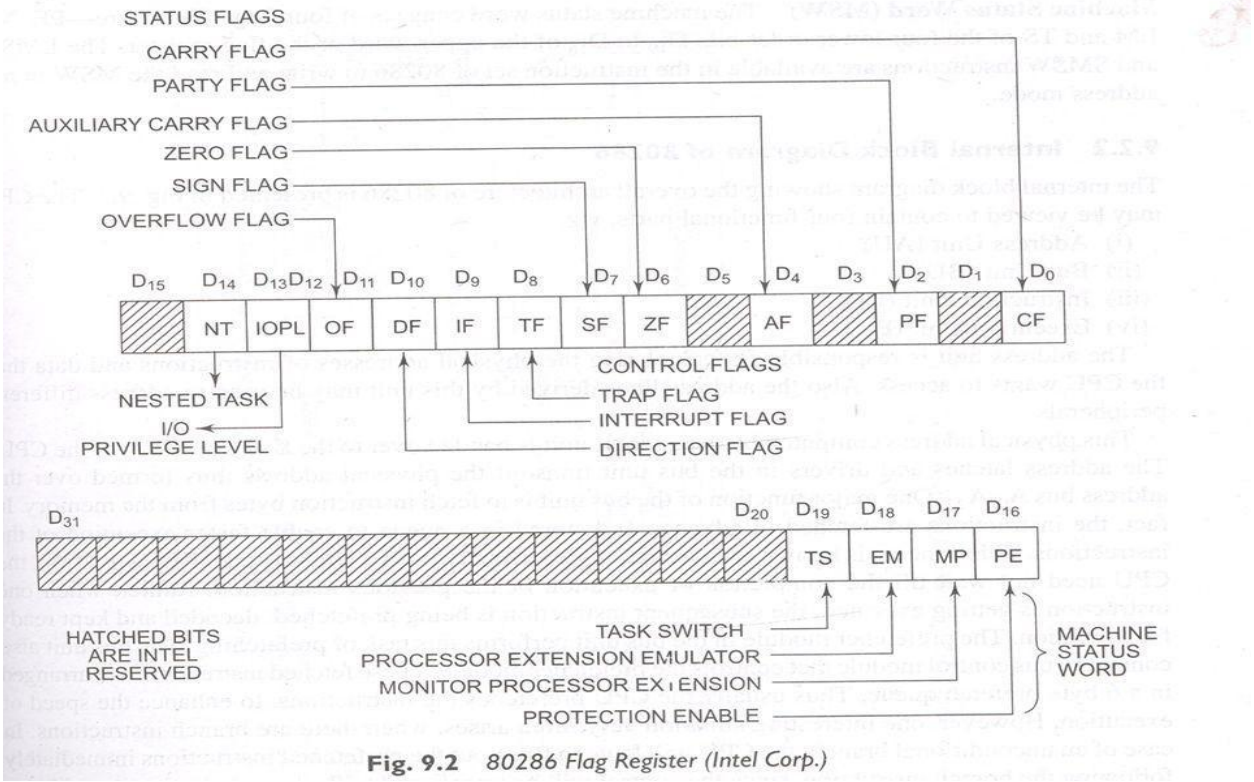


Fig. 9.2 *80286 Flag Register (Intel Corp.)*

D2, D4, D6, D7 and D11 are called as status flag bits. The bits D8 (TF) and D9 (IF) are used for controlling machine operation and thus they are called control flags. The additional fields available in 80286 flag registers are:

1. IOPL - I/O Privilege Field (bits D12 and D13)
2. NT - Nested Task flag (bit D14)
3. PE - Protection Enable (bit D16)
4. MP - Monitor Processor Extension (bit D17)
5. EM - Processor Extension Emulator (bit D18)
6. TS – Task Switch (bit D19)

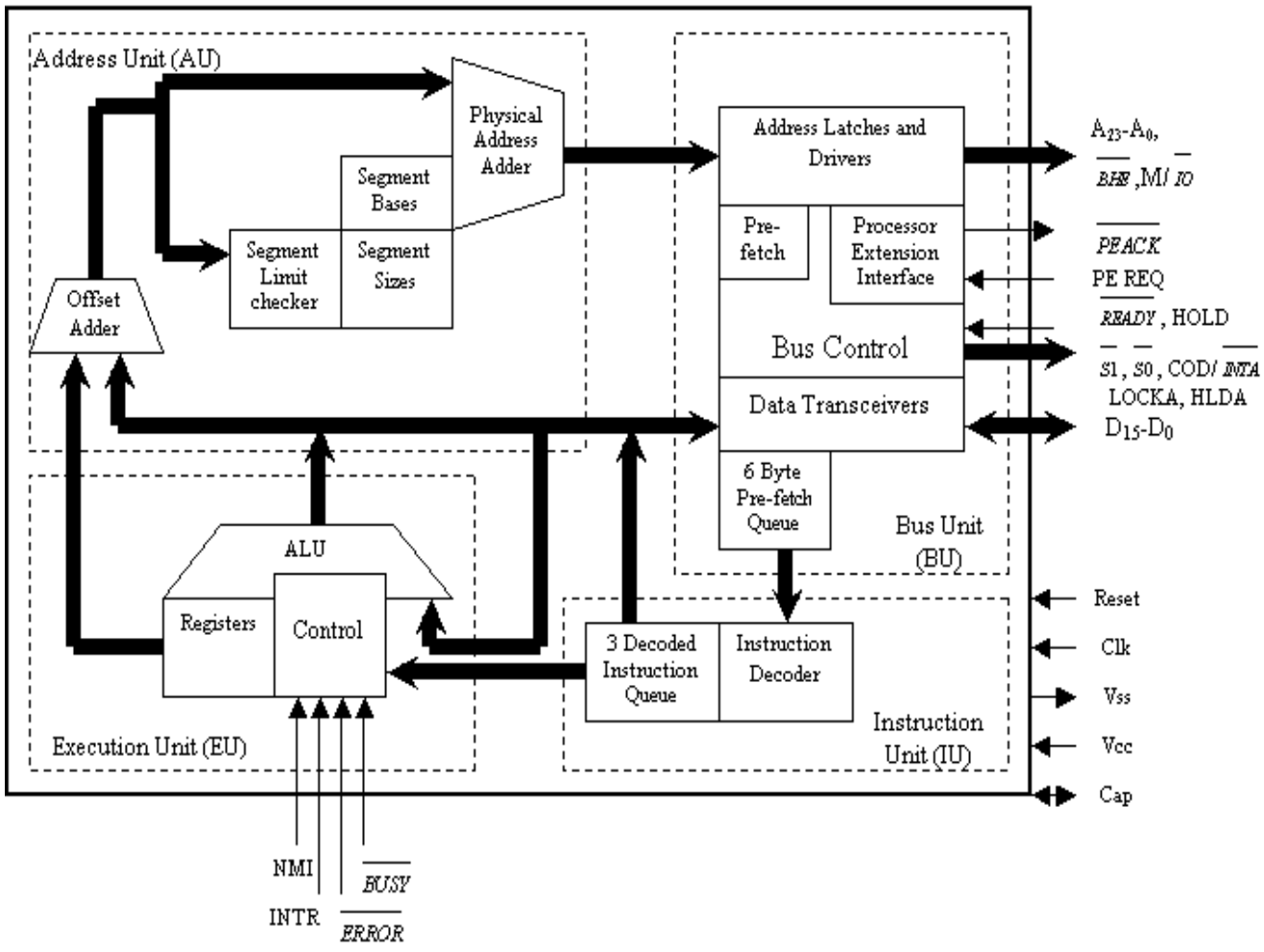
Protection Enable flag places the 80286 in protected mode, if set. This can only be cleared by resetting the CPU. If the Monitor Processor Extension flag is set, allows WAIT instruction to generate a processor extension not present exception.

Processor Extension Emulator flag if set, causes a processor extension absent exception and permits the emulation of processor extension by the CPU. Task Switch flag if set, indicates the next instruction using extension will generate exception 7, permitting the CPU to test whether the current processor extension is for the current task.

Machine Status Word (MSW)

The machine status word consists of four flags – PE, MO, EM and TS of the four lower order bits D19 to D16 of the upper word of the flag register. The LMSW and SMSW instructions are available in the instruction set of 80286 to write and read the MSW in real address mode.

Internal Block Diagram of 80286



The CPU contain four functional blocks

1. Address Unit (AU) 3. Instruction Unit (IU)

2. Bus Unit (BU) 4. ExecutionUnit(EU)

The address unit is responsible for calculating the physical address of instructions and data that the CPU wants to access. Also the address lines derived by this unit may be used to address different peripherals. The physical address computed by the address unit is handed over to the bus unit (BU) of the CPU. Major function of the bus unit is to fetch instruction bytes from the memory. Instructions are fetched in advance and stored in a queue to enable faster execution of the instructions. The bus unit also contains a bus control

module that controls the prefetcher module. These prefetched instructions are arranged in a 6-byte instructions queue. The 6-byte prefetch queue forwards the instructions arranged in it to the instruction unit (IU). The instruction unit accepts instructions from the prefetch queue and an instruction decoder decodes them one by one. The decoded instructions are latched onto a decoded instruction queue. The output of the decoding circuit drives a control circuit in the execution unit, which is responsible for executing the instructions received from decoded instruction queue. The decoded instruction queue sends the data part of the instruction over the data bus. The EU contains the register bank used for storing the data as scratch pad, or used as special purpose registers. The ALU, the heart of the EU, carries out all the arithmetic and logical operations and sends the results over the data bus or back to the register bank.

Signal Description of 80286

CLK: This is the system clock input pin. The clock frequency applied at this pin is divided by two internally and is used for deriving fundamental timings for basic operations of the circuit. The clock is generated using 8284 clock generator.

D15-D0: These are sixteen bidirectional data bus lines.

A23-A0: These are the physical address output lines used to address memory or I/O devices. The address lines A23 - A16 are zero during I/O transfers BHE: This output signal, as in 8086, indicates that there is a transfer on the higher byte of the data bus (D15 – D8)

S1, S0: These are the active-low status output signals which indicate initiation of a bus cycle and with M/IO and COD/INTA, they define the type of the bus cycle.

M/ IO: This output line differentiates memory operations from I/O operations. If this signal is it “0” indicates that an I/O cycle or INTA cycle is in process and if it is “1” it indicates that a memory or a HALT cycle is in progress.

COD/ INTA: This output signal, in combination with M/ IO signal and S1 , S0 distinguishes different memory, I/O and INTA cycles.

LOCK: This active-low output pin is used to prevent the other masters from gaining the control of the bus for the current and the following bus cycles. This pin is activated by a "LOCK" instruction prefix, or automatically by hardware during XCHG, interrupt acknowledge or descriptor table access

READY: This active-low input pin is used to insert wait states in a bus cycle, for interfacing low speed peripherals. This signal is neglected during HLDA cycle.

HOLD and HLDA This pair of pins is used by external bus masters to request for the control of the system bus (HOLD) and to check whether the main processor has granted the control (HLDA) or not, in the same way as it was in 8086.

INTR: Through this active high input, an external device requests 80286 to suspend the current instruction execution and serve the interrupt request. Its function is exactly similar to that of INTR pin of 8086.

NMI: The Non-Maskable Interrupt request is an active-high, edge-triggered input that is equivalent to an INTR signal of type 2. No acknowledge cycles are needed to be carried out.

PEREG and PEACK (Processor Extension Request and Acknowledgement): Processor extension refers to coprocessor (80287 in case of 80286 CPU). This pair of pins extends the memory management and protection capabilities of 80286 to the processor extension 80287. The PEREQ input requests the 80286 to perform a data operand transfer for a processor extension. The PEACK active-low output indicates to the processor extension that the requested operand is being transferred.

BUSY and ERROR: Processor extension BUSY and ERROR active-low input signals indicate the operating conditions of a processor extension to 80286. The BUSY goes low, indicating 80286 to suspend the execution and wait until the BUSY become inactive. In this duration, the processor extension is busy with its allotted job. Once the job is completed the processor extension drives the BUSY input high indicating 80286 to continue with the program execution. An active ERROR signal causes the 80286 to perform the processor extension interrupt while executing the WAIT and ESC instructions. The active ERROR signal indicates to 80286 that the processor extension has committed a mistake and hence it is reactivating the processor extension interrupt.

CAP: A 0.047 μf, 12V capacitor must be connected between this input pin and ground to filter the output of the internal substrate bias generator. For correct operation of 80286 the capacitor must be charged to its operating voltage. Till this capacitor charges to its full capacity, the 80286 may be kept stuck to reset to avoid any spurious activity.

Vss: This pin is a system ground pin of 80286.
Vcc: This pin is used to apply +5V power supply voltage to the internal circuit of 80286. RESET The active-high RESET input clears the internal logic of 80286, and reinitializes it RESET The active-high reset input pulse width should be at least 16 clock cycles. The 80286 requires at least 38 clock cycles after the trailing edge of the RESET input signal, before it makes the first opcode fetch cycle.

Real Address Mode

- Act as a fast 8086
- Instruction set is upwardly compatible
- It address only 1 M byte of physical memory using A0-A19.
- In real addressing mode of operation of 80286, it just acts as a fast 8086. The instruction set is upward compatible with that of 8086.

The 80286 addresses only 1Mbytes of physical memory using A0- A19. The lines A20-A23 are not used by the internal circuit of 80286 in this mode. In real address mode, while addressing the physical memory, the 80286 uses BHE along with A0- A19. The 20-bit physical address is again formed in the same way as that in 8086.

The contents of segment registers are used as segment base addresses. The other registers, depending upon the addressing mode, contain the offset addresses. Because of extra pipelining and other circuit level improvements, in real address mode also, the 80286 operates at a much faster rate than 8086, although functionally they work in an identical fashion. As in 8086, the

physical memory is organized in terms of segments of 64Kbyte maximum size. An exception is generated, if the segment size limit is exceeded by the instruction or the data. The overlapping of physical memory segments is allowed to minimize the memory requirements for a task. The 80286 reserves two fixed areas of physical memory for system initialization and interrupt vector table. In the real mode the first 1Kbyte of memory starting from address 0000H to 003FFH is reserved for interrupt vector table. Also the addresses from FFFF0H to FFFFFH are reserved for system initialization.

The program execution starts from FFFFH after reset and initialization. The interrupt vector table of 80286 is organized in the same way as that of 8086. Some of the interrupt types are reserved for exceptions, single-stepping and processor extension segment overrun, etc.

When the 80286 is reset, it always starts the execution in real address mode. In real address mode, it performs the following functions: it initializes the IP and other registers of 80286, it prepares for entering the protected virtual address mode.

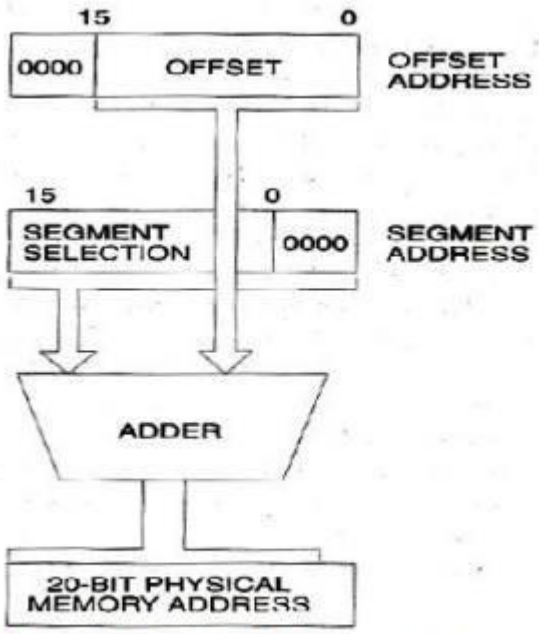


Fig 2.20 Real Address calculation

Protected Virtual Address Mode (PVAM)

80286 is the first processor to support the concepts of virtual memory and memory management. The virtual memory does not exist physically it still appears to be available within the system. The concept of VM is implemented using Physical memory that the CPU can directly access and secondary memory that is used as a storage for data and program, which are stored in secondary memory initially.

The Segment of the program or data required for actual execution at that instant is fetched from the secondary memory into physical memory. After the execution of this fetched segment, the next segment required for further execution is again fetched from the secondary memory, while the results of the executed segment are stored back into the secondary memory for further references. This continues till the complete program is executed

During the execution the partial results of the previously executed portions are again fetched into the physical memory, if required for further execution.

The procedure of fetching the chosen program segments or data from the secondary storage into physical memory is called swapping. The procedure of storing back the partial results or data back on the secondary storage is called unswapping. The virtual memory is allotted per task.

The 80286 is able to address 1 G byte (230 bytes) of virtual memory per task. The complete virtual memory is mapped on to the 16Mbyte physical memory. If a program larger than 16Mbyte is stored on the hard disk and is to be executed, if it is fetched in terms of data or program segments of less than 16Mbyte in size into the program memory by swapping sequentially as per sequence of execution.

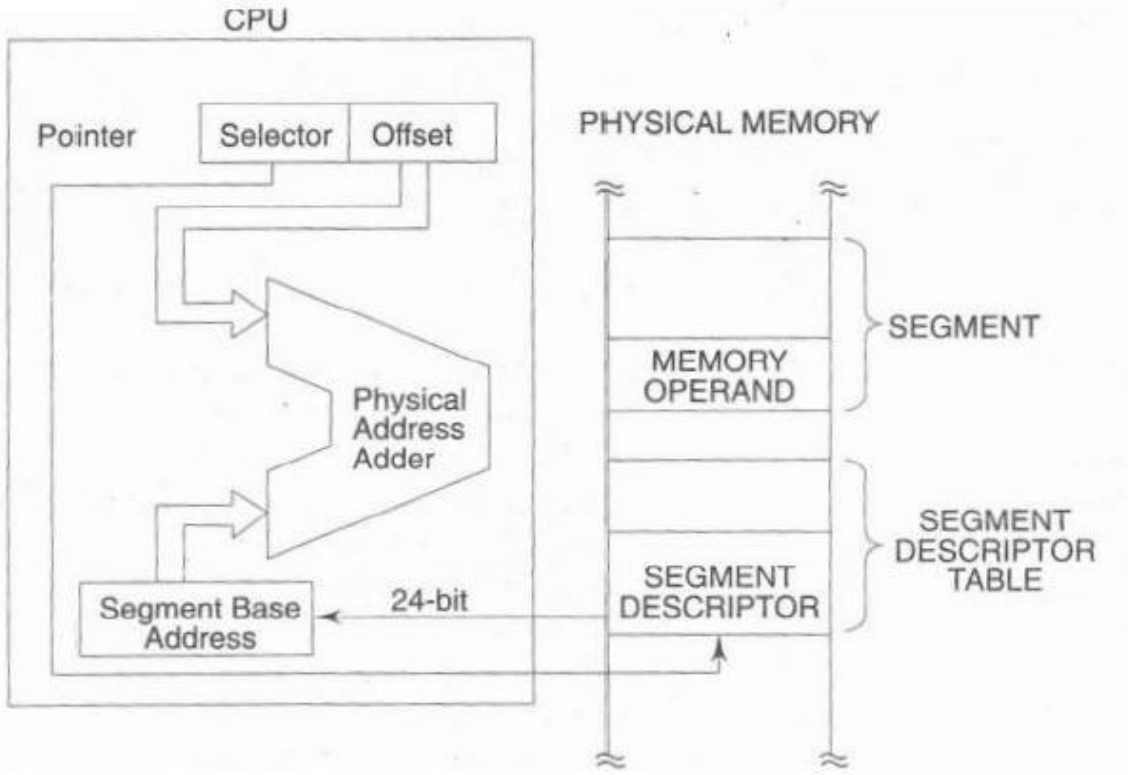


Figure: Physical Address Calculation in PVAM

Whenever the portion of a program is required for execution by the CPU, it is fetched from the secondary memory and placed in the physical memory is

called swapping in of the program. A portion of the program or important partial results required for further execution, may be saved back on secondary storage to make the PM free for further execution of another required portion of the program is called swapping out of the executable program.

80286 uses the 16-bit content of a segment register as a selector to address a descriptor stored in the physical memory. The descriptor is a block of contiguous memory locations containing information of a segment, like segment base address, segment limit, segment type, privilege level, segment availability in physical memory, descriptor type and segment use another task.

Multitasking

Multitasking has the same meaning of multiprogramming but in a more general sense, as it refers to having multiple (programs, processes, tasks, threads) running at the same time. This term is used in modern operating systems when multiple tasks share a common processing resource (e.g., CPU and Memory). At any time the CPU is executing one task only while other tasks waiting their turn. The illusion of parallelism is achieved when the CPU is reassigned to another task (i.e. *process* or *thread context switching*). There are subtle differences between multitasking and multiprogramming. A *task* in a multitasking operating system is not a whole application program but it can also refer to a "thread of execution" when one process is divided into sub-tasks. Each smaller task does not hijack the CPU until it finishes like in the older multiprogramming but rather a fair share amount of the CPU time called quantum.Just to make it easy to remember, both multiprogramming and multitasking operating systems are **(CPU) time sharing** systems. However, while in multiprogramming (older OSs) one program as a whole keeps running until it blocks, in multitasking (modern OSs) time sharing is best manifested because each running process takes only a fair quantum of the CPU time.

Privilege Level:

6. Explain Privilege level.

Ans There are four types of privilege levels

1. 00 - kernel level (highest privilege level)
2. 01 - OS services
3. 10 - OS extensions
4. 11 - Applications (lowest privilege level)

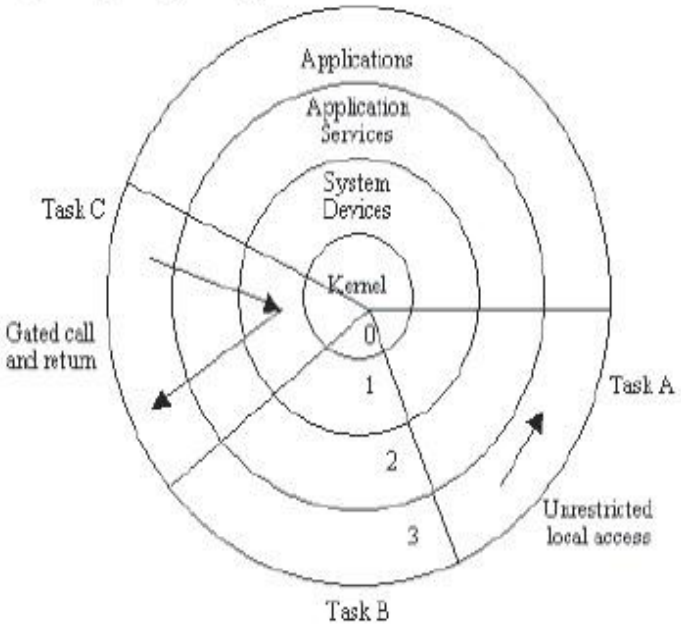


Figure: Privilege Level

- Each task assigned a privilege level, which indicates the priority or privilege of that task.
- It can only changed by transferring the control, using gate descriptors, to a new segment.
- A task executing at level 0, the most privileged level, can access all the data segment defined in GDT and LDT of the task.
- A task executing at level 3, the least privileged level, will have the most limited access to data and other descriptors.
- The use of rings allows for system software to restrict tasks from accessing data.
- In most environments, the operating system and some device drivers run in ring 0 and applications run in ring 3.

Descriptor:

7. What is Descriptor table? What is its use? Differentiate between GDT and LDT.

Ans

- Descriptor is a identifier of a program segment or page.
- A segment cannot be accessed, if its descriptor does not exist in either LDT or GDT.
- Set of descriptor (descriptor table) arranged in a proper sequence describes the complete program.
- The descriptor is a block of contiguous memory location containing information of a segment, like
  i. Segment base address
  ii. Segment limit

iii. Segment type
iv. Privilege level – prevents unauthorized access
v. Segment availability in physical memory
vi. Descriptor type
vii. Segment use by another task

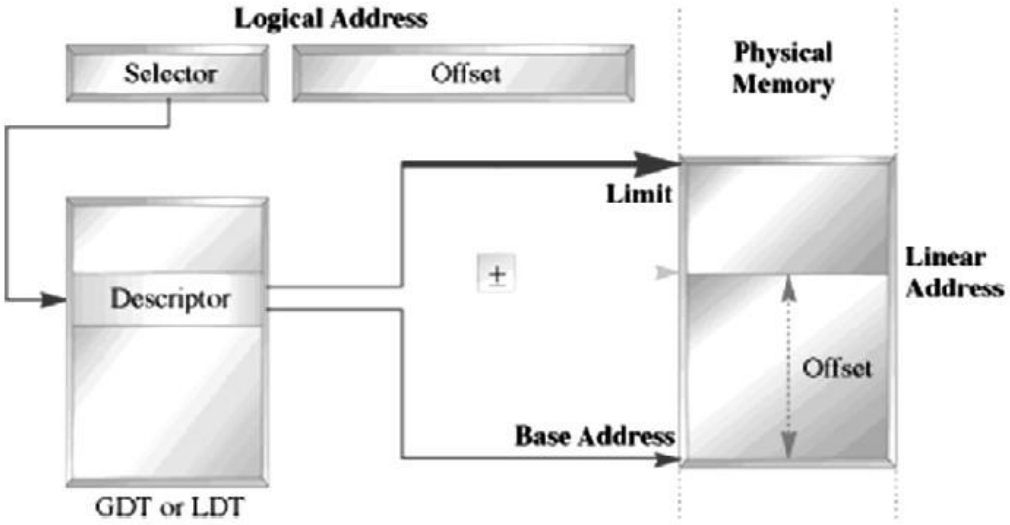


Figure: Protected Mode Addressing with Descriptor Table

Requirement of Descriptor Table:

- The descriptor describes the location, length, and access rights of the segment of memory.
- The selector, located in the segment register, selects one of descriptors from one of two tables of descriptors.

GDT & LDT:

7. Explain LDT, GDT and IDT. Differentiate LDT and GDT

Ans Global Descriptor Table (GDT):

- The 80286 has a single Global Descriptor Table (**GDT**) which is shared between all tasks and addresses up to 512MB of the virtual address space.
- The **Global Descriptor Table** or GDT is a data structure used by Intel x86-family processors starting with the 80286 in order to define the characteristics of the various memory areas used during program execution, including the base address, the size and access privileges like execute-ability and write-ability.

Local Descriptor Table (LDT):

- Each task will have its own Local Descriptor Table (LDT) which is a private 512MB of address space.
- LDT is essential to implement separate address spaces for multiple processes.
- The operating system will switch the current LDT when scheduling a new process, using the LDT machine instruction.

Descriptor Table (LDT):

- IDT used to store interrupt gates and task gates.

- LIDT instruction is used to Load Interrupt Descriptor table.

Differentiate LDT and GDT:

- LDT is actually defined by a descriptor inside the GDT, while the GDT is directly defined by a linear address.
- The lack of symmetry between both tables is underlined by the fact that the current LDT can be automatically switched on certain events, notably if TSS-based multitasking is used, while this is not possible for the GDT.
- The LDT also cannot store certain privileged types of memory segments.
- The LDT is the sibling of the Global Descriptor Table (GDT) and similarly defines up to 8191 memory segments accessible to programs.
- LDT (and GDT) entries which point to identical memory areas are called ***aliases***.
- Instruction to load GDT is LGDT(Load Global Descriptor Table) and instruction to load LDT is LLDT(Load Global Descriptor Table). Both are privileged instructions.

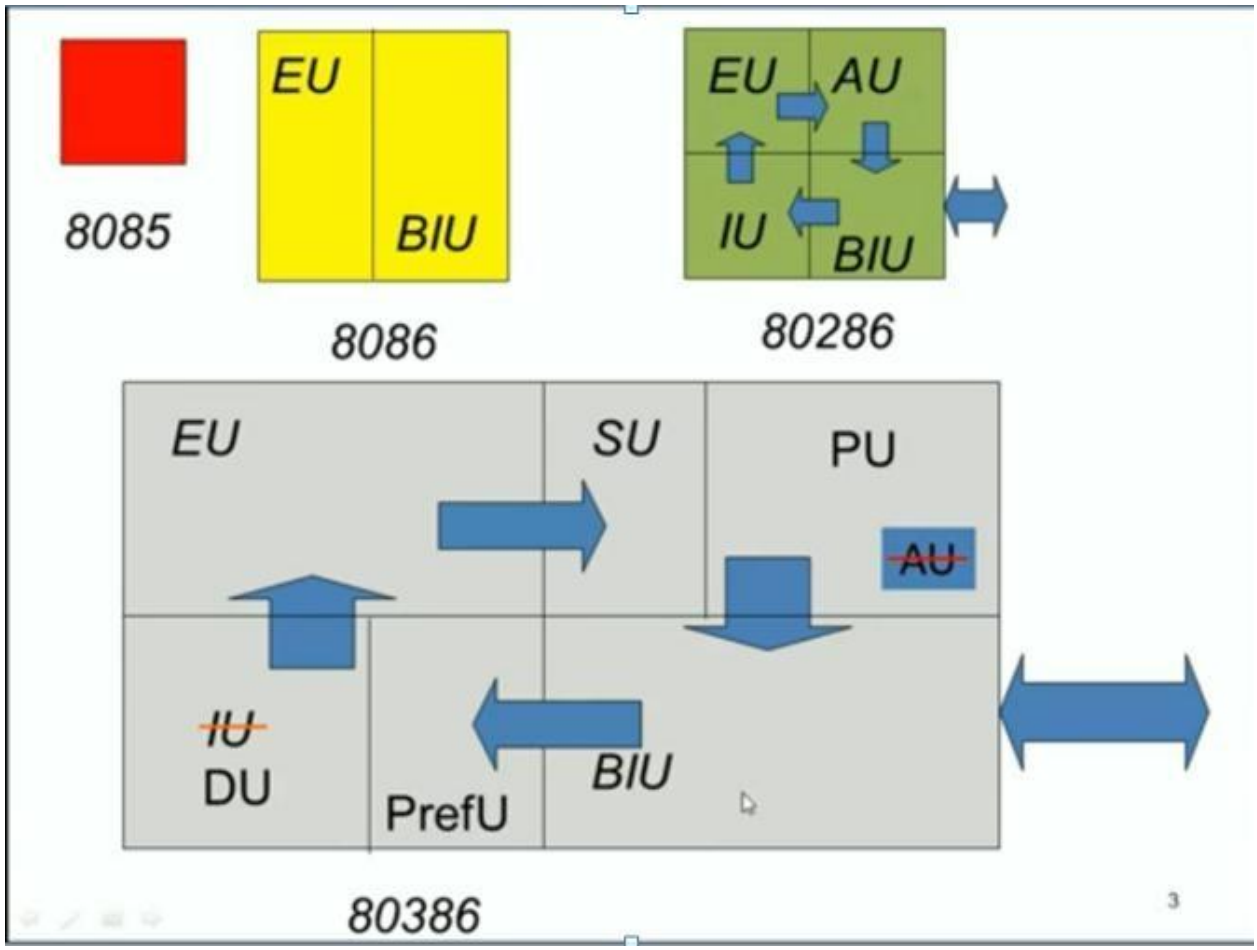
7.2 80386 Microprocessor

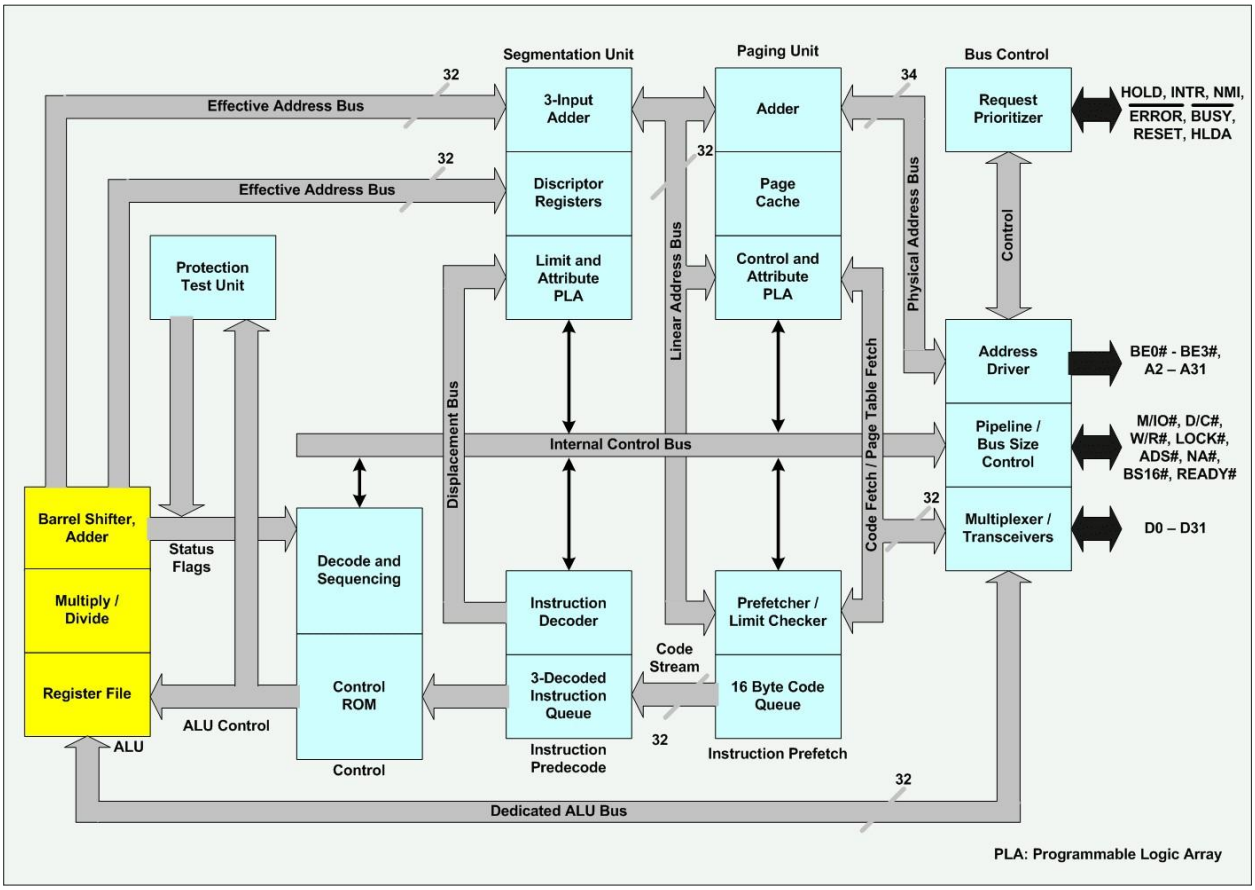
✓ The Internal Architecture of 80386 is divided into 3 sections.

 Central processing Memory management Bus interface unit unit unit

✓ Central processing unit is further divided into Execution unit and Instruction unit Execution unit has 8 General purpose and 8 Special purpose registers which are either used for handling data or calculating offset addresses.

Architecture of 80386





The internal architecture of 80386 is divided into following sections:

- Central processing Instruction decode Segmentation unit unit unit Paging unit
- Execution unit Memory Bus control unit management unit

The central processing unit is further divided into **Execution unit** and **Instruction unit**. The memory management unit consists of a segmentation unit and a paging unit. These unit operate in parallel. Fetching, decoding, memory management and bus access for several instruction are performed simultaneously. This parallel operation is called pipelined instruction processing.

Execution unit: The execution unit read the instruction from the instruction queue and executes the instructions. It consists of three subunit: control unit, data unit and protection test unit.

Control unit: It contains microcode and special hardware. The microcode and special hardware allow 80386DX to reduce time required for execution of multiply and divide instruction. It also speeds up the effective address calculation.

Data unit: The data unit contain the ALU, eight 32-bit general purpose registers and a 64-bit barrel shifter. The barrel shifter is used for multiple bit shifts in one clock. Thus it increases the speed of all shift and rotate operation. The multiply/divide logic implement the bit shift rotate algorithm to complete the operation in minimum time. The entire data unit is responsible for data operation requested by the control unit.

Protection test unit: the protection test unit check for segmentation violations under the control of the microcode.

Instruction decode unit: The instruction decode unit takes the instruction bytes from the code prefetch queue and translates them into microcode the decoded. The decoded instruction are then stored in the instruction queue.
They are passed to control section for deriving the necessary control signals.

Segmentation unit: The segmentation unit translates logic addresses into linear addresses at request of the execution unit. The segmentation unit compares the effective address for the length limit specified in the segment descriptor. The segment unit adds the segment base and the effective address

to generate linear address. Before calculation of linear address it also check for access rights. It provides a 4-level protection mechanism for protecting and isolating the system code and data from those of the application program.

Paging unit: When the 80386DX paging mechanism is enabled, the paging unit translates linear addresses generated by the segmentation unit or the code prefetch unit into physical addresses. If paging unit is not enabled, the physical address is the same as the linear address, and no translation is necessary. The paging unit gives physical address to the bus interface unit to perform memory and I/O accesses. It organizes the physical memory in term of pages of 4 Kbytes size each.

The control and attribute PLA check the privileges at the page level. Each of the page maintain the paging information of the task. The limit and attribute PLA checks segment limits and attributes at the segment level to avoid invalid accesses to code and data in the memory segments.

Bus control unit: The bus control unit is the 80386DX's communication with the outside world. It provide a full 32-bit bi-directional data bus and 32-bit address bus. The bus control unit is responsible for the following operations: It accepts internal request for code fetch and data transfer from the code fetch unit and from the execution unit. It then prioritize the request with the help of prioritize and generate signal to perform bus cycles.

It sends address, data and control signal to communicate with memory and I/O devices. The address driver drives the bus enable and address signal A0A31 and the transceiver interface the internal data bus with the system bus.

It control the interface to the external bus masters and coprocessors.
It also provides the address relocation facility.

Instruction prefetch unit:

The instruction prefetch unit fetch sequentially the instruction byte stream from the memory. It uses bus control unit to fetch instruction bytes when the bus control unit is not performing bus cycle to execute an instruction. These prefetched instruction bytes are stored in the 16-byte code queue. A 16-byte code queue holds these instruction until the decoder needs them the prefetcher always fetches instruction in the order in which they appear in the memory. It fact, the prefetcher simply reads code one double word at the time, not caring whether it's bringing in complete instruction are executed, the contents of the prefetched and decode queues are cleared out. In this case, prefetcher again starts filling its queue.

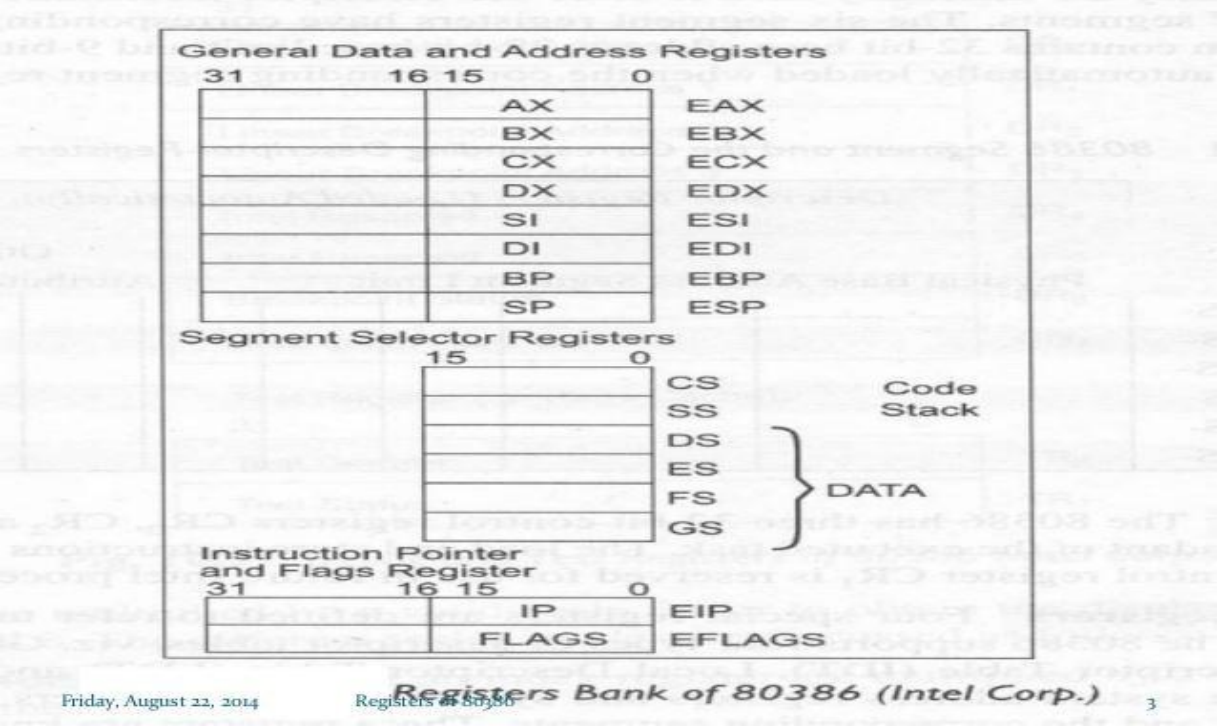
Instruction predecode unit: The instruction predecode unit takes instruction bytes from the instruction prefetch queue and translate them into microcode the decoded instruction are then stored in instruction queue.

Register Organization

Types of Registers

- Flag register
- Segment descriptor register
- Control register
- System address register
- Debug and test register

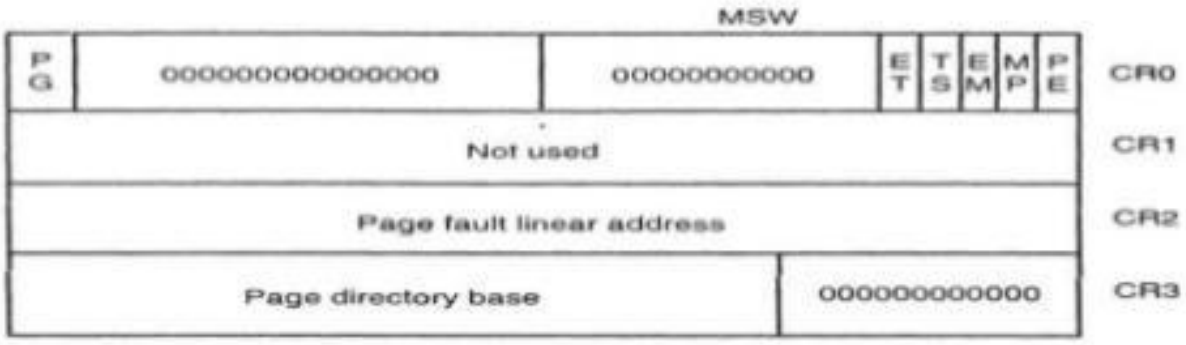
- The 80386 has eight 32 - bit general purpose registers which may be used as either 8 bit or 16 bit registers. A 32 - bit register known as an extended register, is represented by the register name with prefix E.
- Example: A 32 bit register corresponding to AX is EAX, similarly BX is EBX etc.
- The 16 bit registers BP, SP, SI and DI in 8086 are now available with their extended size of 32 bit and are names as EBP, ESP, ESI and EDI.
- AX represents the lower 16 bit of the 32 bit register EAX.
- BP, SP, SI, DI represents the lower 16 bit of their 32 bit counterparts, and can be used as independent 16 bit registers.
- The six segment registers available in 80386 are CS, SS, DS, ES, FS and GS.
- The CS and SS are the code and the stack segment registers respectively, while DS, ES, FS, GS are 4 data segment registers.
- A 16 bit instruction pointer IP is available along with 32 bit counterpart EIP.



Registers Bank of 80386 (Intel Corp.)

Control Registers:

- The 80386 has three 32 bit control registers CR0, CR2 and CR3 to hold global machine status independent of the executed task.
- Load and store instructions are available to access these registers.



• CR2 :- The control register CR2 is used to store the 32-bit linear address at which the previous page fault was detected(PFLA).

• CS3 :- Used when virtual addressing is enabled, hence when the PG bit is set in CR0. CR3 enables the processor to translate linear addresses into physical addresses by locating the page directory and page tables for the current task. Typically, the upper 20 bits of CR3 become the page directory base register (PDBR), which stores the physical address of the first page directory entry

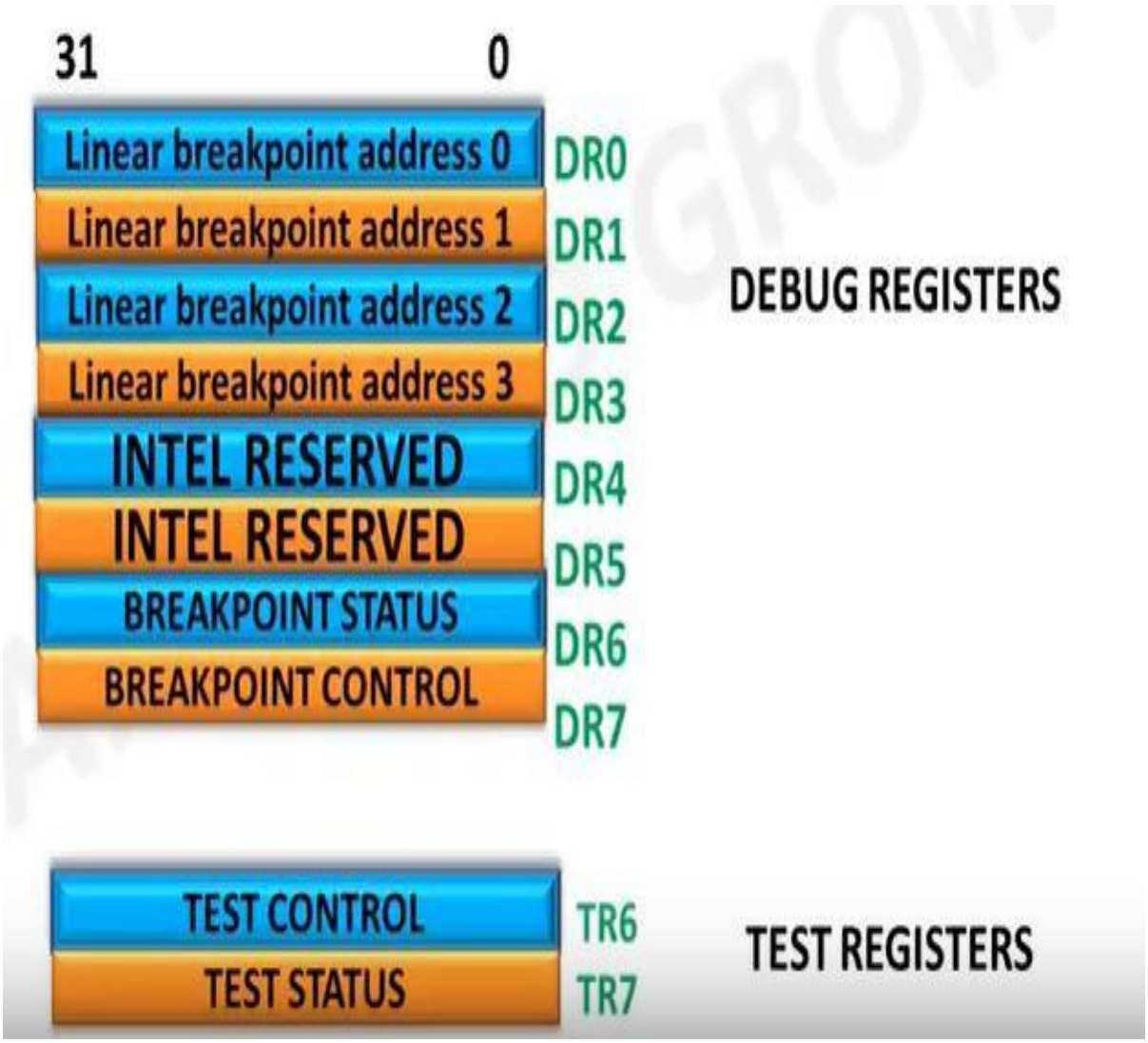
•CR0 :- Register CR0 contains a number of special control bits that are defined as follow ....

1) PG (pagging) :- if PG is set ,it enables paging and use the CR3 register, else disable paging.
2) ET (Extension Type):- On the 80386 , it allowed to specify whether the external math coprocessor was an 80287 or 80387. if ET=0 than it is 80287, else 80387.
3) TS (Task Switch) :- The processor sets TS with every task switch and tests TS when interpreting coprocessor instructions.
4) EM(Emulation) :- EM indicates whether coprocessor functions are to be emulated.
5) MP(Monitor) :- MP controls the function of the WAIT instruction, which is used to coordinate a coprocessor.
6) PE(Protected Mode Enables):-If 1,System is in protected mode, else system is in real mode.

Debug and Test Registers:

- Intel has provide a set of 8 debug registers for hardware debugging.
- Out of these eight registers DR0 to DR7, two registers DR4 and DR5 are Intel reserved.
- The initial four registers DR0 to DR3 store four program controllable breakpoint addresses
- while DR6 and DR7 respectively hold breakpoint status and breakpoint control information.

•Two more test register are provided by 80386 for page caching namely test control and test status register



System Address Registers:

Four special registers are defined to refer to the descriptor tables supported by 80386.

The 80386 supports four types of descriptor table, viz.

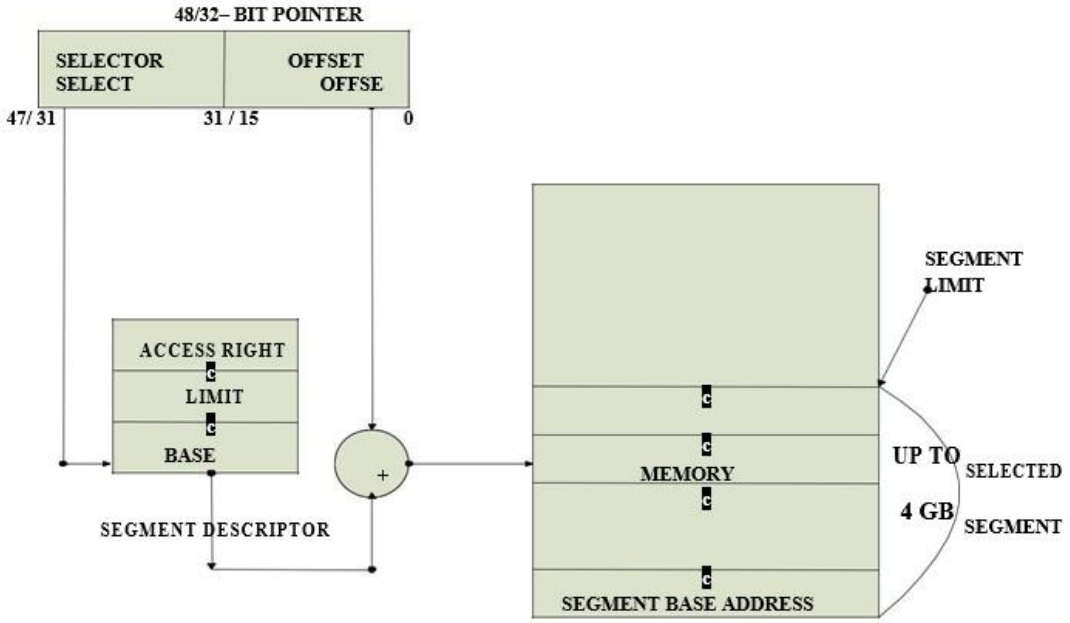
- global descriptor table (GDT),
- interrupt descriptor table (IDT),
- local descriptor table (LDT) ,
- task state segment descriptor (TSS).

Segment Descriptor Registers:

- This registers are not available for programmers, rather they are internally used to store the descriptor information, like attributes, limit and base addresses of segments.
- The six segment registers have corresponding six 73 bit descriptor registers. Each of them contains 32 bit base address, 32 bit base limit and 9 bit attributes. These are automatically loaded when the corresponding segments are loaded with selectors.

Memory Access in Protected Mode of 80386

- All the capabilities of 80386 are available for utilization in its protected mode of operation.
- The 80386 in protected mode support all the software written for 80286 and 8086 to be executed under the control of memory management and protection abilities of 80386.
- The protected mode allows the use of additional instruction, addressing modes and capabilities of 80386.



Protected Mode Addressing Without Paging Unit

ADDRESSING IN PROTECTED MODE:

In this mode, the contents of segment registers are used as selectors to address descriptors which contain the segment limit, base address and access rights byte of the segment.

- The effective address (offset) is added with segment base address to calculate linear address. This linear address is further used as physical address, if the paging unit is disabled, otherwise the paging unit converts the linear address into physical address.
- The paging unit is a memory management unit enabled only in
- protected mode. The paging mechanism allows handling of large segments of memory in terms of pages of 4Kbyte size.
- The paging unit operates under the control of segmentation unit. The paging unit if enabled converts linear addresses into physical address, in protected mode.

PAGING:

Paging is one of the memory management techniques used for virtual memory multitasking operating system. The segmentation scheme may divide the physical memory into a variable size segments but the paging divides the memory into a fixed size pages. The segments are supposed to be the logical segments of the program, but the pages do not have any logical relation with the program. The pages are just fixed size portions of the program module or data.

The advantage of paging scheme is that the complete segment of a task need not be in the physical memory at any time. Only a few pages of the segments, which are required currently for the execution need to be available in the physical memory.

Thus the memory requirement of the task is substantially reduced, relinquishing the available memory for other tasks. Whenever the other pages of task are required for execution, they may be fetched from the secondary storage. The previous page which are executed, need not be available in the memory, and hence the space occupied by them may be relinquished for other tasks. Thus paging mechanism provides an effective technique to manage the physical memory for multitasking systems.

Paging Unit:

The paging unit of 80386 uses a two level table mechanism to convert a linear address provided by segmentation unit into physical addresses. The paging unit converts the complete map of a task into pages, each of size 4K. The task is further handled in terms of its page, rather than segments. The paging unit handles every task in terms of three components namely page directory, page tables and page itself.

Paging Descriptor Base Register:

The control register CR2 is used to store the 32-bit linear address at which the previous page fault was detected.

The CR3 is used as page directory physical base address register, to store the physical starting address of the page directory. The lower 12 bit of the CR3 are

always zero to ensure the page size aligned directory. A move operation to CR3 automatically loads the page table entry caches and a task switch operation, to load CR0 suitably.

Page Directory:

This is at the most 4Kbytes in size. Each directory entry is of 4 bytes, thus a total of 1024 entries are allowed in a directory. The upper 10 bits of the linear address are used as an index to the corresponding page directory entry. The page directory entries point to page tables.

- It has 1024 Entries each of 4 Bytes which points towards starting of the paging table.
- In other words, 1K Paging Directory supports 1K Paging Tables. ■ Total size of Paging Directory = 1024 * 4B = 4K

Page Tables:

- It has 1024 Entries each of 4 Bytes which points towards the pages.
- Every Paging table supports 1K Pages.
- Total size of Paging Table= 1024 * 4B = 4K

Each page table is of 4Kbytes in size and many contain a maximum of 1024 entries. The page table entries contain the starting address of the page and the statistical information about the page. The upper 20 bit page frame address is combined with the lower 12 bit of the linear address. The address bits A12- A21 are used to select the 1024 page table entries. The page table can be shared between the tasks.

The hierarchical steps used by the 80386 to generate an address in paged mode.

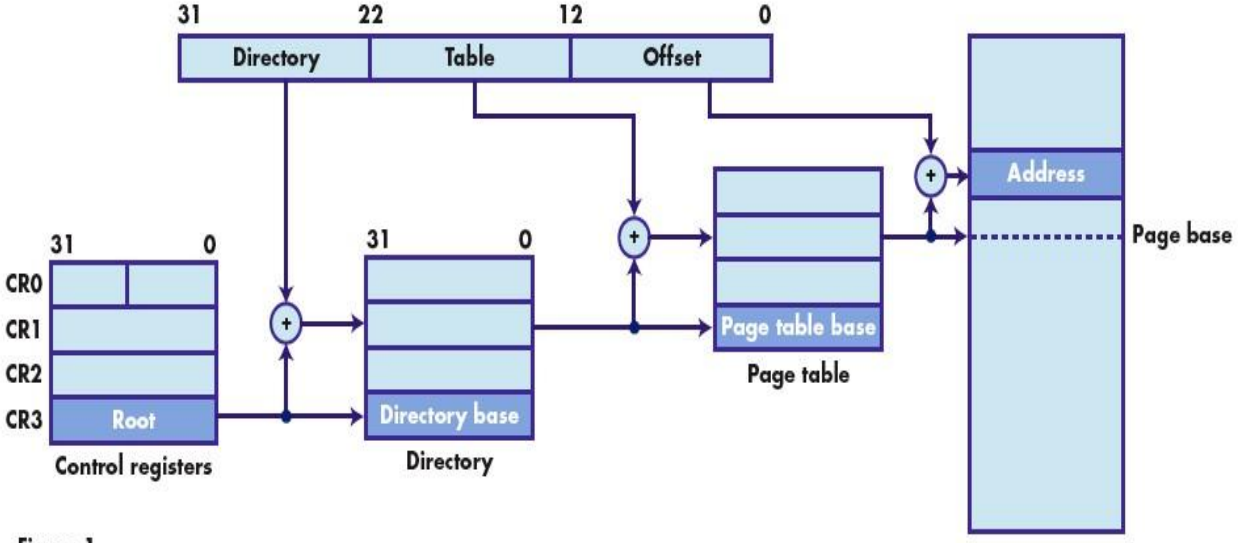
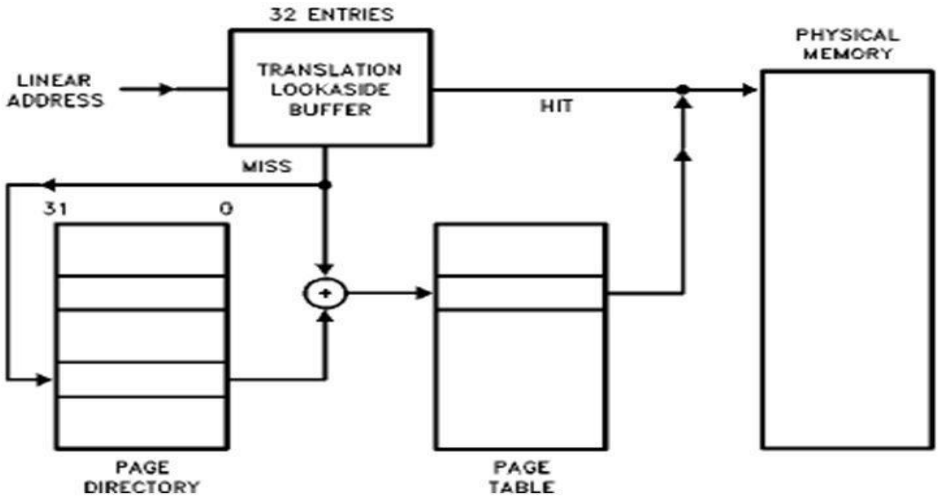


Figure 1

Paging Operation



- The P bit of the above entries indicate, if the entry can be used in address translation.
- If P=1, the entry can be used in address translation, otherwise it cannot be used.
- The P bit of the currently executed page is always high.

- The accessed bit A is set by 80386 before any access to the page. If A=1, the page is accessed, else unassessed.
- The D bit (Dirty bit) is set before a write operation to the page is carried out. The D-bit is undefined for page director entries.
- The OS reserved bits are defined by the operating system software.
- The User / Supervisor (U/S) bit and read/write bit are used to provide protection. These bits are decoded to provide protection under the 4 level protection model.
- The level 0 is supposed to have the highest privilege, while the level 3 is supposed to have the least privilege.
- This protection provide by the paging unit is transparent to the segmentation unit.